

# Programación en Python para Robótica:

De la Teoría a la Práctica con LEGO®  
MINDSTORMS® EV3

Francisco Alejandro Medina Aguirre  
Jimmy Alexander Cortés Osorio  
Hernán Alberto Quintero Vallejo



Colección  
Textos Académicos

UTP  
Universidad Tecnológica  
de Pereira

Editorial UTP



# **Programación en Python para Robótica: de la Teoría a la Práctica con LEGO® MINDSTORMS® EV3**

**Francisco Alejandro Medina Aguirre**

**Jimmy Alexander Cortés Osorio**

**Hernán Alberto Quintero Vallejo**



**Facultad de Tecnología  
Colección Textos Académicos  
2025**

Medina Aguirre, Francisco Alejandro  
Programación en Python para robótica : de la teoría a la práctica  
Con Lego® Mindstorms® EV3 / Francisco Alejandro Medina Aguirre,  
Jimmy Alexander Cortés Osorio y Hernán Alberto Quintero Vallejo. --  
Pereira : Editorial Universidad Tecnológica de Pereira, 2025.  
193 páginas. (Colección Textos académicos).

e-ISBN: 978-628-501-054-5

1. Robots programación 2. Python – Lenguaje de programación 3.  
Robótica

CDD. 629.892

## **Programación en Python para Robótica: de la Teoría a la Práctica con LEGO® MINDSTORMS® EV3**

© Francisco Alejandro Medina Aguirre

© Jimmy Alexander Cortés Osorio

© Hernán Alberto Quintero Vallejo

© Universidad Tecnológica de Pereira

e-ISBN: 978-628-501-054-5

### **Universidad Tecnológica de Pereira**

Vicerrectoría de Investigaciones, Innovación y Extensión

Editorial Universidad Tecnológica de Pereira

Pereira, Colombia

### **Coordinador editorial**

Luis Miguel Vargas Valencia

luismvargas@utp.edu.co

Teléfono (606) 313 7381

Edificio 9, Biblioteca Central Jorge Roa Martínez

Cra. 27 No. 10-02 Los Álamos, Pereira, Colombia

www.utp.edu.co

### **Diagramación y producción**

Jonathan Morales Montes

j.morales9@utp.edu.co

Universidad Tecnológica de Pereira

Pereira, Risaralda, Colombia.



*A mi familia, fuente inagotable de inspiración y apoyo incondicional.*

*Por ser el motor que impulsa mis sueños, la guía en los momentos  
de desafío y la alegría en cada logro alcanzado.*

*Este libro es el resultado del esfuerzo, la pasión por el conocimiento  
y el deseo de compartirlo con quienes buscan aprender y crear.*

*Con gratitud y cariño.*



# Contenido

|                           |    |
|---------------------------|----|
| <b>Introducción</b> ..... | 13 |
|---------------------------|----|

## **Capítulo 1:**

|                                                          |    |
|----------------------------------------------------------|----|
| <b>Programación del Robot LEGO® MINDSTORM® EV3</b> ..... | 17 |
|----------------------------------------------------------|----|

|                                          |    |
|------------------------------------------|----|
| ¿Qué es la Programación? .....           | 19 |
| ¿Qué es Python? .....                    | 20 |
| ¿Qué es LEGO® MINDSTORMS EV3? .....      | 22 |
| Aplicaciones de Python en robótica. .... | 26 |

## **Capítulo 2:**

|                                                        |    |
|--------------------------------------------------------|----|
| <b>Configuración del Ambiente del Desarrollo</b> ..... | 33 |
|--------------------------------------------------------|----|

|                                                                      |    |
|----------------------------------------------------------------------|----|
| Software requerido. ....                                             | 35 |
| Configuración e instalación del LEGO®                                |    |
| MINDSTORMS EV3 con PyBricks .....                                    | 37 |
| ¿Qué es PyBricks? .....                                              | 37 |
| Preparación del entorno de desarrollo. ....                          | 38 |
| Configurando comunicación EV3 y PC. ....                             | 39 |
| Conexión USB .....                                                   | 39 |
| Conexión Bluetooth .....                                             | 40 |
| Conexión Wifi .....                                                  | 40 |
| El firmware del EV3 .....                                            | 41 |
| Firmware oficial LEGO® MINDSTORMS® EV3 (versión 1.10E o superior) .. | 41 |
| EV3 MicroPython (versión 1.2 o superior) .....                       | 41 |
| Pasos para configurar y ejecutar un programa usando Pybricks. ....   | 42 |
| Recursos de aprendizaje PyBricks. ....                               | 44 |

### Capítulo 3:

#### Fundamentos del Python . . . . . 47

|                                                          |    |
|----------------------------------------------------------|----|
| Sintaxis Básica. . . . .                                 | 50 |
| Variables y Tipos de Datos . . . . .                     | 51 |
| Operadores y Expresiones de Asignación . . . . .         | 52 |
| Estructuras de Control. . . . .                          | 56 |
| <i>Condicional if</i> . . . . .                          | 56 |
| <i>Ciclo for</i> . . . . .                               | 57 |
| <i>Ciclo while</i> . . . . .                             | 58 |
| <i>Instrucciones break, continue, pass</i> . . . . .     | 58 |
| Funciones. . . . .                                       | 59 |
| ¿Qué es una función? . . . . .                           | 59 |
| Sintaxis de funciones. . . . .                           | 60 |
| Parametrizando funciones . . . . .                       | 60 |
| Valores por defecto . . . . .                            | 61 |
| Devolviendo valores. . . . .                             | 61 |
| Tipos de funciones Python . . . . .                      | 62 |
| Clases y Objetos. . . . .                                | 65 |
| ¿Qué es una clase? . . . . .                             | 65 |
| Creando objetos . . . . .                                | 66 |
| Atributos. . . . .                                       | 66 |
| Métodos. . . . .                                         | 66 |
| <i>Bibliotecas en Python</i> . . . . .                   | 68 |
| Tipos de bibliotecas en Python . . . . .                 | 69 |
| <i>Bibliotecas estándar de Python</i> . . . . .          | 69 |
| <i>Bibliotecas de terceros en Python</i> . . . . .       | 70 |
| Cómo instalar bibliotecas de terceros en Python. . . . . | 70 |
| Ventajas de las bibliotecas en Python. . . . .           | 70 |

### Capítulo 4: Programación del LEGO®

#### MINDSTORMS® EV3 Usando la Biblioteca . . . . . 73

|                                                 |    |
|-------------------------------------------------|----|
| El ladrillo del LEGO® MINDSTORMS® EV3 . . . . . | 75 |
| Componentes del EV3 . . . . .                   | 76 |
| Motores del LEGO® MINDSTORMS® EV3 . . . . .     | 78 |

|                                                                 |     |
|-----------------------------------------------------------------|-----|
| Tipos de motores EV3. . . . .                                   | 79  |
| Control de motores usando la biblioteca Pybricks . . . . .      | 80  |
| Consideraciones de diseño. . . . .                              | 81  |
| La clase Motor de la biblioteca PyBricks. . . . .               | 83  |
| <i>Sensores EV3. . . . .</i>                                    | 85  |
| Sensor de color . . . . .                                       | 86  |
| <i>Especificaciones . . . . .</i>                               | 87  |
| <i>Programación con PyBricks . . . . .</i>                      | 88  |
| Sensor de ultrasonido . . . . .                                 | 90  |
| <i>Especificaciones . . . . .</i>                               | 91  |
| <i>Programación con PyBricks . . . . .</i>                      | 91  |
| Sensor Giroscópico del LEGO® MINDSTORMS® EV3. . . . .           | 94  |
| <i>Especificaciones . . . . .</i>                               | 96  |
| <i>Programación en PyBricks . . . . .</i>                       | 97  |
| Sensor Táctil del LEGO® MINDSTORMS® EV3. . . . .                | 97  |
| <i>Especificaciones . . . . .</i>                               | 98  |
| <i>Programación en PyBricks . . . . .</i>                       | 99  |
| Sensor infrarrojo . . . . .                                     | 100 |
| <i>Especificaciones Técnicas . . . . .</i>                      | 101 |
| <i>Programación en PyBricks . . . . .</i>                       | 102 |
| Pantalla LCD del LEGO® MINDSTORMS® EV3. . . . .                 | 104 |
| <i>Especificaciones . . . . .</i>                               | 104 |
| <i>Programación con PyBricks . . . . .</i>                      | 104 |
| Bocina e Indicadores Sonoros del LEGO® MINDSTORMS® EV3. . . . . | 107 |
| <i>Especificaciones . . . . .</i>                               | 107 |
| <i>Programación con PyBricks . . . . .</i>                      | 108 |
| Leds del Lego Mindstorms EV3 . . . . .                          | 110 |
| <i>Especificaciones . . . . .</i>                               | 110 |
| <i>Programación en PyBricks . . . . .</i>                       | 110 |

## Capítulo 5:

|                                                 |     |
|-------------------------------------------------|-----|
| <b>Proyectos Prácticos . . . . .</b>            | 115 |
| Programa para Recorrido de 10 Cuadros . . . . . | 118 |
| Programa para Recorrido en Espiral. . . . .     | 120 |

|                                                                             |            |
|-----------------------------------------------------------------------------|------------|
| Programa de Recorrido Aleatorio . . . . .                                   | 122        |
| Programa con Sensor de Tacto. . . . .                                       | 124        |
| Programa con Sensor Ultrasónico . . . . .                                   | 126        |
| Programa con El Sensor Infrarrojo . . . . .                                 | 128        |
| Programa Seguidor de Línea. . . . .                                         | 129        |
| <b>Capítulo 6:</b>                                                          |            |
| <b>Testing y Depuración de Programas con Pybricks . . . . .</b>             | <b>135</b> |
| Probando Programas . . . . .                                                | 137        |
| Errores comunes y soluciones . . . . .                                      | 139        |
| Conclusiones. . . . .                                                       | 142        |
| Glosario . . . . .                                                          | 143        |
| <b>Capítulo 7:</b>                                                          |            |
| <b>Una Referencia Pedagógica para la Enseñanza de la Robótica . . . . .</b> | <b>147</b> |
| Algunas técnicas de aprendizaje activo . . . . .                            | 151        |
| Aprendizaje Basado en Problemas (ABP) . . . . .                             | 152        |
| Aprendizaje cooperativo . . . . .                                           | 153        |
| Flipped Classroom o Aula Invertida. . . . .                                 | 156        |
| Aprendizaje basado en proyectos (ABP) . . . . .                             | 157        |
| Una propuesta de ABP usando el robot LEGO®                                  |            |
| MINDSTORMS® EV3 . . . . .                                                   | 164        |
| Propuesta ABP en su fase de planificación. . . . .                          | 166        |
| Propuesta de evaluación. . . . .                                            | 170        |
| <b>Anexos . . . . .</b>                                                     | <b>173</b> |
| <b>Bibliografía . . . . .</b>                                               | <b>202</b> |
| <b>Autores . . . . .</b>                                                    | <b>209</b> |

# Agradecimientos

*Como docente de la Universidad Tecnológica de Pereira, deseo expresar mi profunda gratitud y reconocimiento a los ingenieros Carlos Augusto Meneses y Osiel Arbeláez, visionarios directores de los programas de Ingeniería de Sistemas y Computación e Ingeniería Mecatrónica respectivamente.*

*Su excelente labor, al frente de estas carreras tan relevantes para la región, ha permitido formar verdaderos profesionales integrales, técnicamente competitivos, pero sobre todo, éticamente responsables. Gracias a líderes educativos como ustedes nuestra casa de estudios se ha consolidado como referente de educación tecnológica de alta calidad.*

*Agradecimiento especial al ingeniero físico y especialista en Gerencia de Proyectos, Jeiner Orlando Modero Machado, por su valiosa colaboración en la corrección de estilo de este documento.*





# Prefacio

La tecnología evoluciona a un ritmo acelerado, y la programación es un arte que nos permite dotar a las máquinas de comportamientos autónomos e inteligentes. Aprender a programar robots abre un sinfín de puertas al desarrollo profesional y la innovación.

Este documento busca introducir de forma amigable el apasionante mundo de la programación de robots usando Python, un lenguaje de programación reconocido por su sintaxis clara y sencilla. El lector aprenderá conceptos de programación mientras desarrolla conductas autónomas en los robots LEGO® MINDSTORMS®.

Mediante explicaciones paso a paso, ejemplos prácticos y continua retroalimentación, el lector aprenderá a dominar las bases de Python y la programación de robots. Los temas cubiertos incluyen Fundamentos de Python, y la programación de robots Ev3. Al finalizar, el lector podrá dar vida a sus propias creaciones robóticas mediante Python.

Este documento está dirigido a estudiantes de ingeniería, entusiastas de la tecnología y cualquier persona que quiera aprender a programar robots. La metodología de aprender construyendo y practicando busca motivar el interés en estas fascinantes áreas del saber.



# Introducción

La programación, es el proceso de crear instrucciones que pueden ser ejecutadas por una computadora u otro dispositivo para realizar tareas específicas. En la actualidad, la programación se ha vuelto omnipresente en nuestra vida cotidiana, desde aplicaciones móviles y sitios web, hasta electrodomésticos e, incluso, juguetes.

Aprender a programar abre un mundo de posibilidades creativas y permite dar rienda suelta a la imaginación; existen muchos lenguajes de programación para elegir. Este libro se enfocará en Python, que se ha convertido en uno de los más populares y de mayor crecimiento en los últimos años.

Python es un lenguaje interpretado, multiplataforma y multiparadigma, que se distingue por tener una sintaxis clara y legible, además de ser, para principiantes, fácil de aprender. Estas características lo convierten en una excelente opción para introducirnos en el apasionante campo de la programación.

Junto con Python se utilizará el popular robot LEGO® MINDSTORMS® EV3, para aprender programación de una forma práctica y entretenida. Esta combinación permite construir distintos robots y dotarlos de comportamientos autónomos programando sus movimientos, sensores y acciones y, de esta manera, se podrá ver cómo cobra vida el código desarrollado en fascinantes proyectos robóticos.

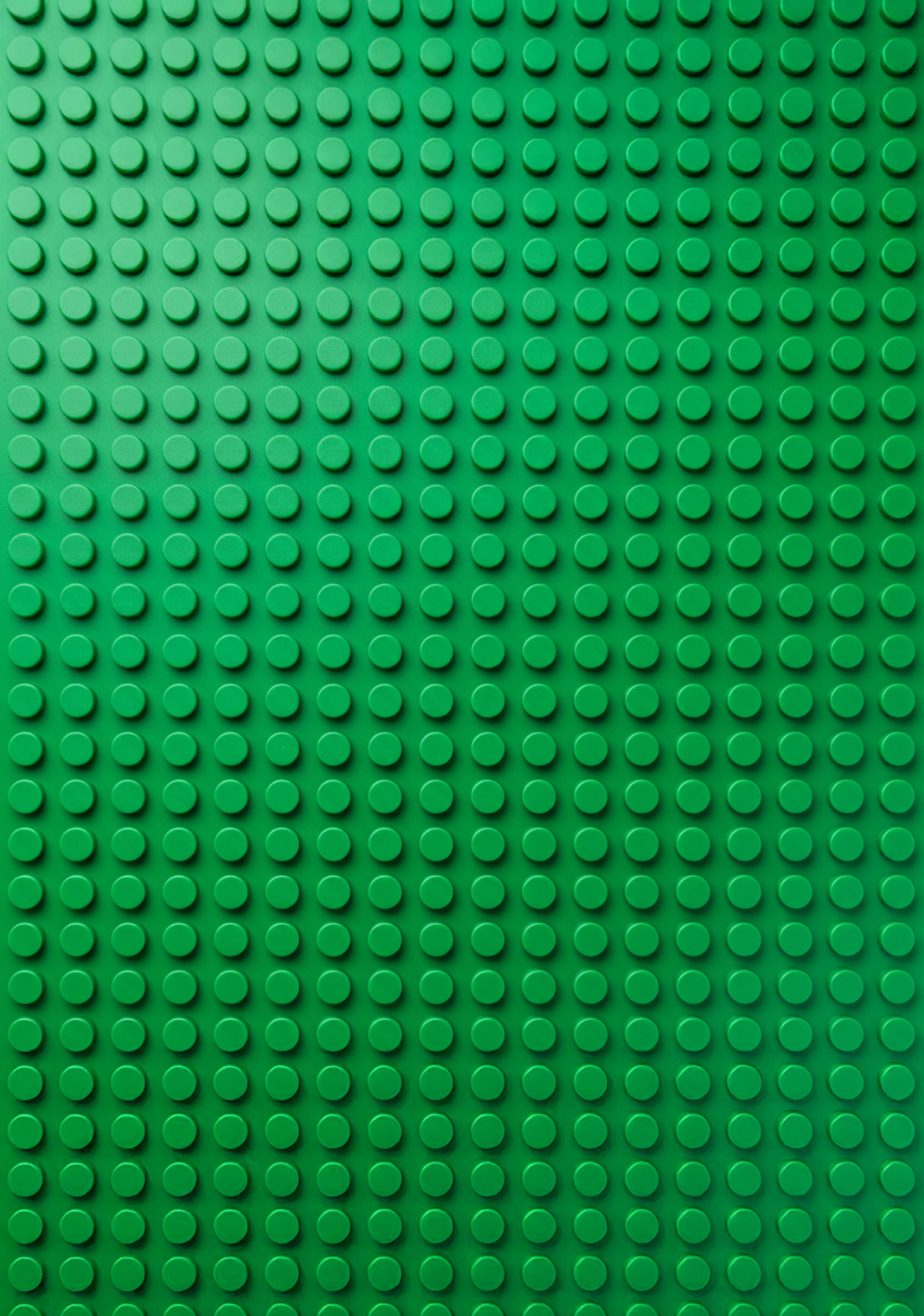
A lo largo de este libro, obtendrás los fundamentos esenciales de Python y programación en general, incluyendo conceptos como variables, operadores, funciones, clases, entre otros. Se utilizarán sencillos ejemplos para ilustrar cada elemento y así facilitar el aprendizaje.

Luego, se aplicarán estos conocimientos para programar un LEGO® MINDSTORMS® EV3 paso a paso, desde hacer simples movimientos hasta crear robots capaces de navegar laberintos y seguir líneas en el suelo.

Al finalizar el libro, habrás adquirido las habilidades para programar robots y otras aplicaciones en Python. La metodología utilizada te permitirá aprender, practicar y aplicar conceptos de una manera activa, constructiva y progresiva. Asimismo, el enfoque práctico sobre proyectos con LEGO® MINDSTORMS® EV3 integrará la programación con una actividad creativa y lúdica.

Los robots LEGO® MINDSTORMS® son accesibles y fáciles de usar, incluso para principiantes o niños, por lo que constituyen un camino ideal para iniciarse en la robótica. La versatilidad de Python y EV3 posibilita crear, desde sencillos programas, hasta complejos sistemas de inteligencia artificial.





## **Capítulo 1**

# **Programación del Robot Lego Midstorn EV3**





## ¿Qué es la Programación?

La programación, o codificación, es el proceso de crear un conjunto de instrucciones estructuradas y organizadas, conocidas como algoritmos, que permiten indicarle a una computadora qué tareas debe realizar. Los algoritmos computacionales se escriben de forma lógica en lenguajes de programación que, tanto las personas como las máquinas pueden entender (Stellman & Greene, 2024).

Un algoritmo es una serie ordenada y finita de pasos u operaciones que permite hacer un cálculo, resolver un problema o ejecutar alguna tarea en particular; sirve para describir el proceso necesario para obtener un resultado deseado. Algunos ejemplos de algoritmos son las recetas de cocina, los mapas para llegar a un destino y los manuales de ensamblaje.

En informática, un algoritmo debe ser preciso, definido, claro, lógico, secuencial y tener un principio y fin determinados. Los algoritmos computacionales son implementados en lenguajes de programación y ejecutados por un dispositivo tecnológico, permiten automatizar tareas, procesar datos, controlar el funcionamiento de máquinas y crear *software*.

Los lenguajes de programación son vocabularios y sintaxis o gramáticas, que permiten escribir algoritmos computacionales en

términos y formatos entendibles tanto para humanos como para las máquinas. Cada lenguaje tiene palabras clave, estructuras y reglas propias para crear programas que solucionen problemas específicos.

Existe una amplia diversidad de lenguajes de programación como Python, *JavaScript*, C++, *Java*, *Swift*, PHP, *Ruby*, etc. Cada uno tiene propósitos, fortalezas y debilidades particulares. Algunos lenguajes son multiparadigma, es decir, soportan distintos estilos y enfoques de programación como programación orientada a objetos, funcional, imperativa, declarativa, entre otros.

### ¿Qué es Python?

Es uno de los lenguajes de programación de mayor crecimiento y popularidad en los últimos años. Fue creado a finales de la década de 1980 por el programador holandés Guido Van Rossum, quien buscaba crear un lenguaje que fuera fácil de usar y aprender, incluso para no programadores.

**Figura 1.**  
*Logo de lenguaje de programación Python*



*Nota.* La imagen muestra el logo de *Phyton*. Fuente: S. F. Python (2021).

Python es multiparadigma, soporta programación orientada a objetos, imperativa y funcional. Se trata de un lenguaje interpretado, es decir, que no necesita compilación y se ejecuta línea por línea, lo que permite ver resultados rápidamente sin procesos intermedios.

También es multiplataforma, puede utilizarse en Windows, Mac, Linux y otros sistemas operativos.

La sintaxis de Python se caracteriza por ser legible y limpia, similar al inglés natural; utiliza sangrías en lugar de corchetes para delimitar bloques de código, lo que le brinda orden visual y favorece la legibilidad y el uso de nombres de variables explicativos. Asimismo, al ser un lenguaje de tipado dinámico, no es necesario declarar variables ni tipos de datos.

Python es un lenguaje interpretado de propósito general, cuenta con tipado dinámico y con una biblioteca estándar que incorpora funciones útiles para distintas tareas de programación, evitando tener que reinventar la rueda. Además, tiene una amplia comunidad que desarrolla y mantiene bibliotecas o *frameworks* adicionales de código abierto para propósitos específicos, como ciencia de datos, web, robótica, inteligencia artificial, entre muchos otros.

Estas características, la sintaxis sencilla de Python y su curva de aprendizaje suave, lo convierten en un excelente primer lenguaje para aprender a programar todo tipo de aplicaciones, ya que permite enfocarse más en la lógica y estructuras de programación antes que en los detalles del lenguaje. Además, Python es utilizado por principiantes como si fueran expertos, dado que es un lenguaje muy versátil y potente.

Desde *scripts* simples hasta complejos programas, cuenta con el poder y escalabilidad para crecer profesionalmente como programador; es un lenguaje confiable utilizado por grandes compañías y aplicaciones web conocidas como Google, YouTube, Instagram, Spotify, entre muchas otras.

Los *scripts* son pequeños fragmentos de código que automatizan tareas específicas, como leer archivos, realizar cálculos o procesar

datos, típicamente sin necesidad de una compilación previa, ejecutándose directamente mediante un intérprete como Python.

Por otra parte, Python sigue mejorando y evolucionando constantemente, la versión estable más reciente es Python 3, que introdujo cambios y mejoras con respecto a su versión anterior Python 2, que ya no se mantiene. La comunidad Python es muy activa en agregar características y actualizaciones por medio de PEPs (*Python Enhancement Proposals*), se adapta así a las tendencias y necesidades cambiantes en el desarrollo de *software*.

Python se ha consolidado como uno de los lenguajes imperdibles y esenciales para todo desarrollador (S. F. Python, 2021), su creciente adopción se debe a características clave como legibilidad, versatilidad, facilidad de uso, amplia biblioteca, comunidad y ecosistema activo. Tiene las cualidades ideales para introducirse en la programación y permite el crecimiento profesional, por esto, aprender Python abre la puerta a un sinfín de posibilidades en el apasionante mundo de la tecnología.

### **¿Qué es LEGO® MINDSTORMS EV3?**

El robot EV3 es un conjunto de robótica de la línea MINDSTORMS® de LEGO®, que fue lanzado en 2013. EV3 proviene de “Evolution 3” (Evolución 3), por ser la tercera generación de estos kits de robótica programable de Lego (LEGO®, 2021b). Este sistema está orientado a aficionados, estudiantes e incluso profesionales que desean adentrarse en el mundo de la robótica y programación de forma práctica e intuitiva. En la Figura 2, se observa el empaque del robot LEGO® MINDSTORMS® EV3.

**Figura 2.**  
*Empaque de robot LEGO® MINDSTORMS EV3*



*Nota.* La imagen representa el empaque comercial del robot LEGO® MINDSTORMS EV3.  
Fuente: LEGO® (2021b).

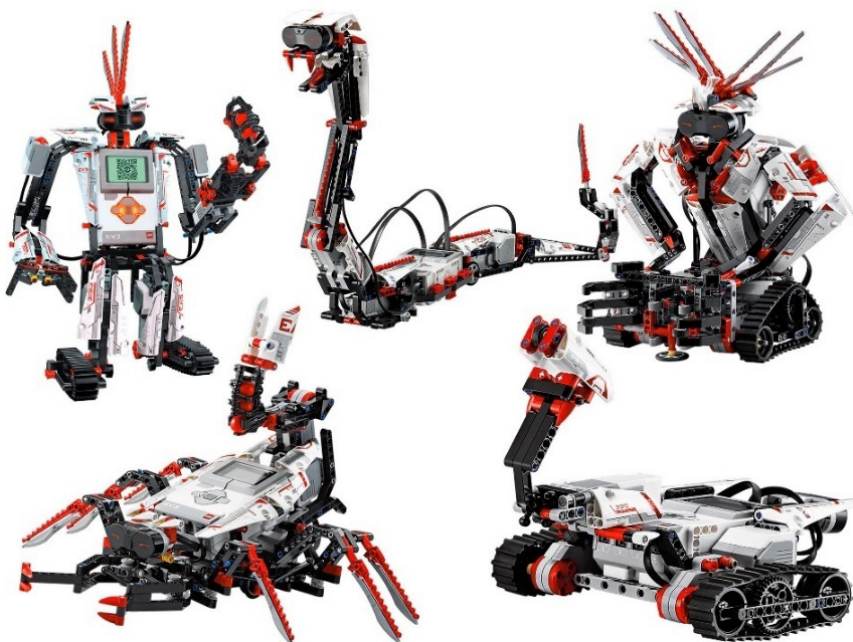
El kit básico de EV3 contiene aproximadamente 600 piezas construibles de Lego, incluyendo bloques, ejes, engranajes y ruedas, así como una unidad programable llamada “ladrillo inteligente” o *brick* EV3, la cual hace las veces de computadora (LEGO®, 2016). Este *brick* cuenta con un procesador ARM9 a 300MHz, memoria RAM de 64 MB, almacenamiento de 16 MB ampliable con tarjeta SD, pantalla LCD monocromática, altavoz y conectividad USB, Bluetooth y Wi-Fi (Rollins, 2014).

Además del ladrillo, el kit incluye tres servomotores interactivos, dos de tamaño grande y uno mediano, así como varios sensores para obtener información del entorno, entre ellos el sensor de tacto, color, ultrasonido y giroscopio. Estos componentes, de entrada y

salida, son fundamentales para que el robot pueda ejecutar acciones y reaccionar ante estímulos mediante la programación desarrollada (LEGO®, 2016).

Uno de los grandes atractivos de LEGO® MINDSTORMS® EV3 es la posibilidad de ensamblar distintos modelos funcionales como vehículos, humanoides, grúas, animales, o lo que la imaginación disponga, utilizando las diversas piezas de Lego. Posteriormente, se programa su comportamiento a través del ladrillo EV3 con movimiento, luces, sonidos, acciones y reacciones, de acuerdo con las entradas de sus sensores (LEGO®, 2021a); en la Figura 3 se observa variantes de los modelos que pueden ser creados a partir de este kit.

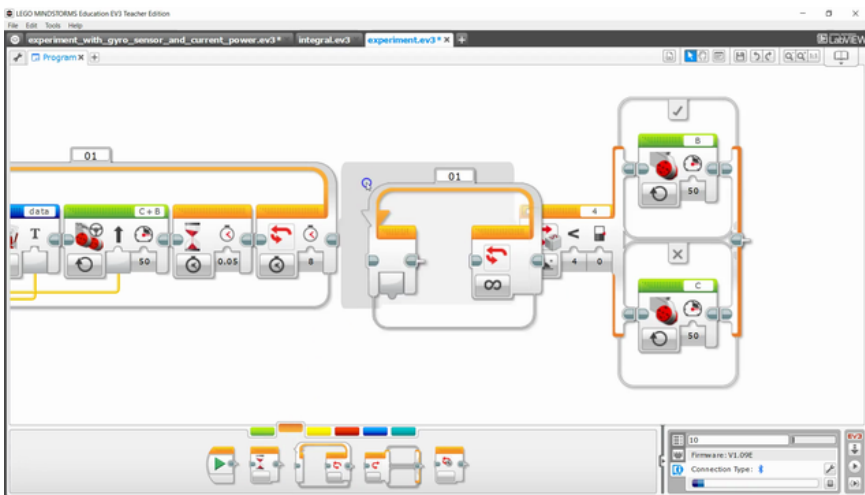
**Figura 3.**  
*Modelos para armar con LEGO® MINDSTORMS® EV3*



*Nota.* La imagen muestra posibilidades de modelos para construir con el LEGO® MINDSTORMS® EV3. Fuente: LEGO® (2021a).

Para crear los programas que controlarán los robots EV3 existen varias alternativas de *software*, por ejemplo, la opción oficial es Lego Mindstorms EV3 Home Edition, un entorno de desarrollo visual que utiliza bloques tipo rompecabezas para definir el comportamiento deseado (LEGO®, 2016). También existen opciones avanzadas como EV3Basic o RobotC, que permiten programación textual en lenguajes como BASIC, C, C++, Python, entre otros. En la Figura 4, se observa la pantalla del *software* oficial de LEGO.

**Figura 4.**



Una de las principales ventajas de MINDSTORMS EV3 es su gran versatilidad para construir desde modelos básicos hasta robots muy complejos controlados por sofisticados programas. Su sistema modular con piezas ensamblables lo diferencia de otros kits que traen un solo modelo predeterminado (LEGO®, 2016). Además, al ser producido por una empresa reconocida como Lego, cuenta con alta calidad en los materiales y buen soporte.



EV3 permite aprender sobre construcción, mecánica, electrónica, motores, sensores, estructuras Lego y, sobre todo, introduce conceptos esenciales de programación y robótica (LEGO®, 2021a). Mediante la creación de distintos proyectos, los usuarios aprenden cómo darle comportamientos autónomos a una máquina, algo que de otra forma sería muy abstracto.

En el área educativa, EV3 es utilizado ampliamente para la enseñanza de STEM (*Science, Technology, Engineering and Math*) dado que de forma práctica e intuitiva enseña sobre ciencia, tecnología, ingeniería y matemáticas (LEGO®, 2021b). Incluso universidades como el MIT (*Massachusetts Institute of Technology*) y Harvard University han incorporado robots de la línea Mindstorms, en sus cursos de robótica e inteligencia artificial para experimentación. Lego Mindstorms EV3 es una excelente plataforma de iniciación en el apasionante mundo de la robótica y la programación, permite construir, programar y dar vida a fascinantes creaciones mecánicas de forma sencilla e interactiva. EV3 conjuga lo mejor del divertido sistema Lego con tecnología de punta en robótica modular.

### **Aplicaciones de Python en robótica**

Python se ha convertido en uno de los lenguajes de programación más utilizados y versátiles para desarrollar aplicaciones en robótica, dada su facilidad de uso y las poderosas librerías especializadas disponibles (LEGO®, 2016). Una de las principales ventajas de Python es que permite controlar de forma simple componentes robóticos como motores, sensores y cámaras desde el lenguaje de programación, mediante el uso de bibliotecas específicas para robótica; se puede interactuar con hardware de diferentes fabricantes sin necesidad de lidiar con los complejos detalles de bajo nivel (Downey, 2012).

Por ejemplo, *Robot Operating System* (ROS) provee una amplia infraestructura en Python para crear todo tipo de comportamientos y funcionalidades en robots (Quigley et al., 2009), además, permite control de movimiento, percepción sensorial, navegación, visión por computadora y manipulación de objetos, entre muchas otras capacidades. En la Figura 5, se observa el logo de ROS.

**Figura 5.**  
*Logo de Robot Operating System*



*Nota.* La imagen representa el logo de ROS. Fuente: Robotics (2021).

Asimismo, OpenCV es una reconocida biblioteca de Python enfocada en visión artificial y procesamiento de imágenes, indispensable en robótica para que un robot pueda “ver” e interpretar su entorno (Bradski & Kaehler, 2017). Python facilita la integración de OpenCV en proyectos de robótica para agregar funciones avanzadas de visión. En la Figura 6, se observa el logo de OpenCV.

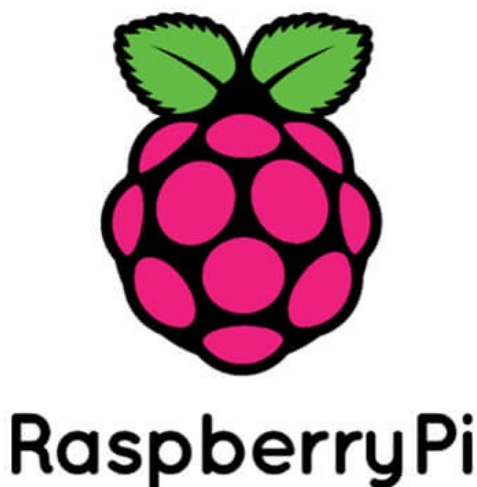
**Figura 6.**  
*Logo de OpenCV*



*Nota.* La imagen representa el logo de OpenCV. Fuente: OpenCV (2008).

En cuanto a robótica educativa, Python ha ganado popularidad gracias a los recientes avances en kits de *hardware/software* como Raspberry Pi, MicroPython y CircuitPython (Richardson et al., 2021). Estas herramientas permiten programar de forma sencilla robots y dispositivos del movimiento maker (proyectos tecnológicos creados por aficionados o estudiantes) mediante Python, sin requerir grandes conocimientos previos. En la Figura 7 se observa el logo de Raspberry Pi.

**Figura 7.**  
*Logo de RaspberryPi*



*Nota.* La imagen representa el logo de RaspberryPi. Fuente: Ltd (2012).

Un ejemplo ampliamente utilizado en el ámbito educativo es el robot Lego Mindstorms EV3, el cual puede ser programado en Python de varias formas (Gouws et al., 2013). Por un lado, instalando el firmware EV3Dev que corre sobre Linux y permite ejecutar código MicroPython, por el otro, transmitiendo programas en Python desde un PC hacia el EV3 usando bibliotecas como EV3Python o PyBricks. A continuación, se observa en la Figura 8 el logo de LEGO Mindstroms EV3.

**Figura 8.**  
*Logo de LEGO Mindstorms EV3*



*Nota.* La imagen representa el logo del robot Minstorms de LEGO®. Fuente: LEGO® (2021a).

En cuanto a robots más avanzados, Python es el lenguaje de programación oficial y recomendado para los populares robots humanoides de SoftBank Robotics, Pepper y NAO (Gouaillier et al., 2009). Python proporciona una API (*Interfaz de Programación de Aplicaciones*) simple y bien documentada para controlar de forma amigable todos los comportamientos y características de estos robots. En la Figura 9, se presentan los robots Pepper y Nao de SoftBank Robotics.

**Figura 9.**  
*Robots Pepper y NAO de SoftBank Robotics*



*Nota.* La imagen representa una muestra de los robots Pepper y NAO. Fuente: SoftBank Robotics America (2025).

A nivel industrial, Python es ampliamente adoptado en aplicaciones de automatización y robótica en manufactura gracias a *frameworks* como Robot Framework, que facilitan crear *scripts* de pruebas automatizadas para robots en este contexto (Robot Framework, 2025). El uso de Python permite mayor productividad y calidad en entornos complejos de fabricación. A continuación, en la Figura 10, se presenta el logo de Robot Framework.

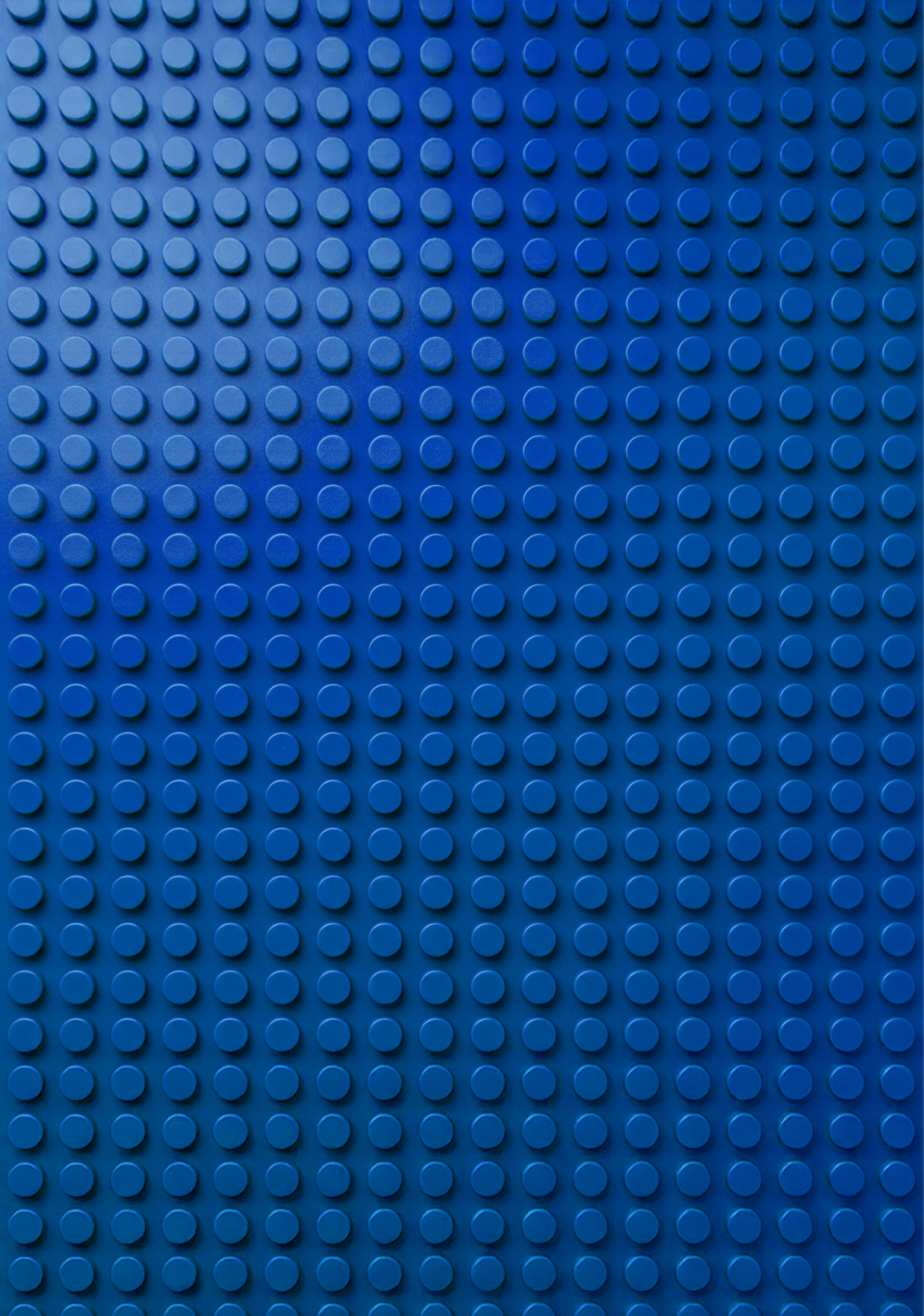
**Figura 10.**  
*Logo de Robot Framework*



*Nota.* La imagen representa el logo de Robot Framework. Fuente: Robot Framework (2025).

Python se ha consolidado como un lenguaje ideal para programar todo tipo de robots y proyectos de robótica en distintos niveles, desde sencillos *scripts* para pequeños robots educativos hasta complejos sistemas de inteligencia artificial en humanoides. La versatilidad y potencia de Python, sumado a sus numerosas librerías especializadas, brindan las herramientas necesarias para dar vida a fascinantes creaciones robóticas.





## **Capítulo 2**

# **Configuración del Ambiente de Desarrollo**





Para empezar a programar el robot Lego Mindstorms EV3 utilizando el lenguaje Python, es necesario configurar adecuadamente el ambiente de desarrollo, lo cual involucra instalar el *software* requerido, establecer la conexión entre la PC y el EV3, y agregar las bibliotecas de Python específicas para interactuar con el robot.

### ***Software requerido***

Lo primero que se debe tener es un *brick* o ladrillo programable EV3, el cual hace las veces de cerebro del robot. El kit básico de LEGO® MINDSTORMS EV3 viene con el *brick* EV3, motores, sensores y piezas mecánicas (LEGO®, 2021a). A continuación, se observa el contenido del kit.

**Figura 11.**  
*Kit de Lego Mindstorms EV3*



*Nota.* La imagen representa los elementos que contiene el kit. Fuente: LEGO® (2021a).

Luego, se requiere un cable USB – mini USB para conectar físicamente el EV3 con la computadora y transferir los programas. Es recomendable un cable USB 2.0 estándar de al menos 1 metro de longitud y de buena calidad para una transmisión estable (LEGO®, 2016).

El siguiente *software* imprescindible es un editor de texto o *Integrated Development Environment* (IDE), con el fin de escribir los programas en Python en la computadora, para esto utilizaremos *Visual Studio Code*, este IDE es gratuito, liviano y con muchas extensiones útiles para Python (Sweigart, 2020), y para programar el LEGO® MINDSTORMS® EV3.

**Figura 12.**  
*Logo Visual Studio Code*



*Nota.* La imagen representa el logo de *Visual Studio Code*. Fuente:(Microsoft, 2020).

También se debe descargar e instalar un intérprete de Python en el equipo o computador. Las versiones recomendadas actualmente son Python 3.7 o superior, siendo 3.10 o 3.11 ideales para aprovechar las últimas características del lenguaje (S. F. Python, 2021). Se puede utilizar la distribución oficial de Python disponible en [www.python.org](http://www.python.org).

### ***Configuración e instalación del LEGO® MINDSTORMS EV3 con PyBricks***

Una excelente opción para programar el EV3, es utilizar el lenguaje Python y la biblioteca PyBricks, la cual permite controlar el robot de manera sencilla e intuitiva. En esta parte del libro, se presentará cómo instalar y configurar PyBricks para programar el EV3 utilizando Visual Studio Code como entorno de desarrollo integrado.

### ***¿Qué es PyBricks?***

PyBricks es una biblioteca de Python *Open Source* creada por LEGO

en colaboración con expertos de la comunidad Python, enfocada en la facilidad de uso para programar robots de LEGO (LEGO®, 2016). Provee una API de alto nivel para interactuar con los ladrillos programables de Lego como los robots EV3, NXT, y boots.

La biblioteca PyBricks presenta las siguientes características clave (Valk, 2021):

- Comunicación inalámbrica con los ladrillos Lego mediante Bluetooth o Wi-Fi sin necesidad de cables.
- Abstracción de los detalles de bajo nivel, enfocándose en objetos de alto nivel como motores, sensores y controles remotos.
- Sintaxis limpia y minimalista parecida al inglés para maximizar legibilidad.
- Excelente documentación con amplios ejemplos y tutoriales.
- Posee una licencia de código abierto MIT, para promover su uso y contribución a la biblioteca.
- Compatibilidad con múltiples ladrillos programables de Lego.

El mismo código Python puede controlar EV3, NXT, Boost, etc.

PyBricks está diseñada desde cero específicamente para proveer la mejor experiencia de programación de robots Lego en Python, tanto para principiantes como para usuarios avanzados.

### **Preparación del entorno de desarrollo.**

Para poder utilizar PyBricks se necesita preparar el entorno de desarrollo, el cual incluye el siguiente *software*:

- **Visual Studio Code:** es el editor de código que utilizaremos para escribir los programas *Python* que controlarán el EV3 (Microsoft, 2020).
- **Python 3.7 o superior:** el intérprete de Python necesario para

ejecutar los programas. Se recomienda la versión 3.10 o 3.11 (S. F. Python, 2021).

- Extensión EV3 MicroPython para VS Code, esta extensión agrega soporte para MicroPython y PyBricks en VS Code.
- Biblioteca PyBricks: se instala mediante el comando `pip install pybricks`.

Una vez se tiene instaladas estas herramientas se puede comenzar a escribir código PyBricks en VS Code para controlar el EV3.

### **Configurando comunicación EV3 y PC.**

Existen dos formas principales de conectar e intercambiar datos entre la computadora y el ladrillo programable EV3: mediante USB y a través de conexión inalámbrica por Bluetooth o la red Wifi, si se le integra al EV3 una USB Wifi compatible con este robot (LEGO®, 2021b).

Los adaptadores que cumplen estos requisitos permitirán configurar el EV3 para programación vía Wi-Fi de forma estable y confiable. Las marcas recomendadas son Edimax, TP-Link, Asus o D-Link.

### **Conexión USB.**

Esta es la forma más común y simple de conectarse. Requiere un cable USB a Mini USB de cualquier marca. Luego se conecta un extremo al puerto USB-host del EV3, y el otro extremo a un puerto USB disponible en la PC.

Posteriormente, se enciende el EV3 y en la PC se detectará como un dispositivo conectado, esto permite transferir programas Python entre ambos. La conexión USB es ideal para las primeras pruebas y muy confiable en entornos con muchas señales inalámbricas.

## Conexión Bluetooth.

Para conectarse de forma inalámbrica se debe vincular el EV3 con la PC mediante Bluetooth, primero se activa el modo Bluetooth en el EV3 desde el menú *Wireless and Networks*. Luego, en la PC se busca el EV3 desde el administrador de dispositivos Bluetooth y se enlaza.

Al estar vinculados, PyBricks detectará el EV3 automáticamente. La conexión Bluetooth es más cómoda al evitar cables, pero puede presentar interrupciones en entornos con muchas interferencias. Requiere que la PC tenga Bluetooth o un adaptador USB.

## Conexión Wifi.

El robot Lego Mindstorms EV3 no tiene como tal un módulo Wifi incorporado, por lo que requiere un adaptador USB Wifi para poder conectarse a redes inalámbricas. Los requisitos del adaptador Wifi para programar el EV3 son:

- **Compatibilidad con el EV3:** el adaptador Wifi debe ser compatible con el hardware y sistema operativo del ladrillo EV3. Los más recomendados son los basados en el chip Realtek RTL8188CUS.
- **Soporte para modo AP:** el adaptador debe poder configurarse en modo de Punto de Acceso (AP) para crear una red Wifi directa con el EV3. Esto facilita la conexión inicial.
- **Seguridad WPA2:** debe soportar el cifrado WPA2 (no sólo WEP) para conectarse de forma segura a redes modernas.
- **Protocolo 802.11b/g/n:** es recomendable que soporte las bandas 2.4GHz en los estándares 802.11b/g/n para máxima compatibilidad.

- **Interfaz USB 2.0:** requiere conexión USB 2.0 para una transmisión estable con el puerto USB del EV3.
- **Bajo consumo energético:** debe consumir poca energía para no agotar rápido la batería del EV3.
- **Antena flexible:** una antena flexible y extensible permite mejorar la señal en diferentes posiciones.

Los adaptadores que cumplen estos requisitos permitirán configurar el EV3 para programación vía Wi-Fi de forma estable y confiable. Las marcas recomendadas son Edimax, TP-Link, Asus o D-Link.

### ***El firmware del EV3***

Para poder ejecutar los programas Python en el EV3, este debe tener instalado un *firmware* compatible con PyBricks (LEGO®, 2021b). Hay dos opciones principales de *firmware* que funcionan:

#### ***Firmware oficial LEGO® MINDSTORMS® EV3 (versión 1.10E o superior).***

Este es el *firmware* por defecto del EV3. Contiene un intérprete de MicroPython especialmente adaptado para correr código PyBricks de forma remota mediante USB o conexión inalámbrica, ya sea por Bluetooth Wifi si se tiene un USB Wifi compatible con el robot. Este *firmware*, por su facilidad de uso, es la opción recomendada para principiantes.

#### ***EV3 MicroPython (versión 1.2 o superior).***

Este *firmware* carga una imagen completa de MicroPython en el EV3 permitiendo ejecutar código Python directamente en el ladrillo sin



necesidad de conexión con la PC. Es más avanzado y permite programación directa en el EV3, no sólo remota; sin embargo, requiere más conocimientos para aprovechar todo su potencial.

### ***Pasos para configurar y ejecutar un programa usando Pybricks.***

#### **Paso 1:** Preparar el ladrillo EV3

1. Descarga la imagen EV3 MicroPython desde el sitio web de LEGO Education.
2. Extrae el archivo de imagen a tu ordenador.
3. Inserta una tarjeta microSD en el lector de tarjetas de tu ordenador.
4. Usa una herramienta de flasheo, como Win32DiskImager, para flashear (grabar imagen) la imagen EV3 MicroPython en la tarjeta microSD.

#### **Paso 2:** Conectar el ladrillo EV3 al ordenador

1. Retira la tarjeta microSD del lector de tarjetas de tu ordenador.
2. Inserta la tarjeta microSD en la ranura para tarjetas SD del ladrillo EV3.
3. Enciende el ladrillo EV3.

**Paso 3:** Instalar la extensión LEGO MINDSTORMS EV3 Micropython para Visual Studio Code.

1. Abre Visual Studio Code.
2. Ve a la pestaña Extensiones.
3. Busca la extensión LEGO MINDSTORMS EV3 Micropython.
4. Haz clic en el botón Instalar.
5. Haz clic en el botón Activar.

#### **Paso 4:** Conectar el ladrillo EV3 a Visual Studio Code.

1. Conecta el ladrillo EV3 a tu ordenador usando un cable USB.

2. Haz clic en el icono EV3 Brick en la barra lateral de Visual Studio Code.

**Paso 5:** Escribir tu primer programa Python

1. Abre un nuevo archivo Python en Visual Studio Code.

Escribe el siguiente código:

```
#!/usr/bin/env pybricks-micropython
from pybricks.hubs import EV3Brick
from pybricks.ev3devices import Motor
from pybricks.parameters import Port
from pybricks.tools import wait

'''Este programa es un ejemplo simple de cómo con-
trolar un ladrillo EV3 usando Python y la biblio-
teca EV3 MicroPython. Hace que el ladrillo EV3
emita un pitido, imprima un mensaje en la pantalla
y luego espera 5 segundos.
'''

#-----

def main():
    # Crea tus objetos aquí.
    ev3 = EV3Brick()

    # Escribe tu programa aquí.
    ev3.speaker.beep()
    ev3.screen.print("Mi primer programa")
    wait(5000)

#-----

if __name__ == "__main__":
    main()
```

2. Guarda el archivo como `my_first_program.py`.

**Paso 6:** Ejecutar tu programa Python

1. Haz clic en el botón Ejecutar en la barra de herramientas de Visual Studio Code.

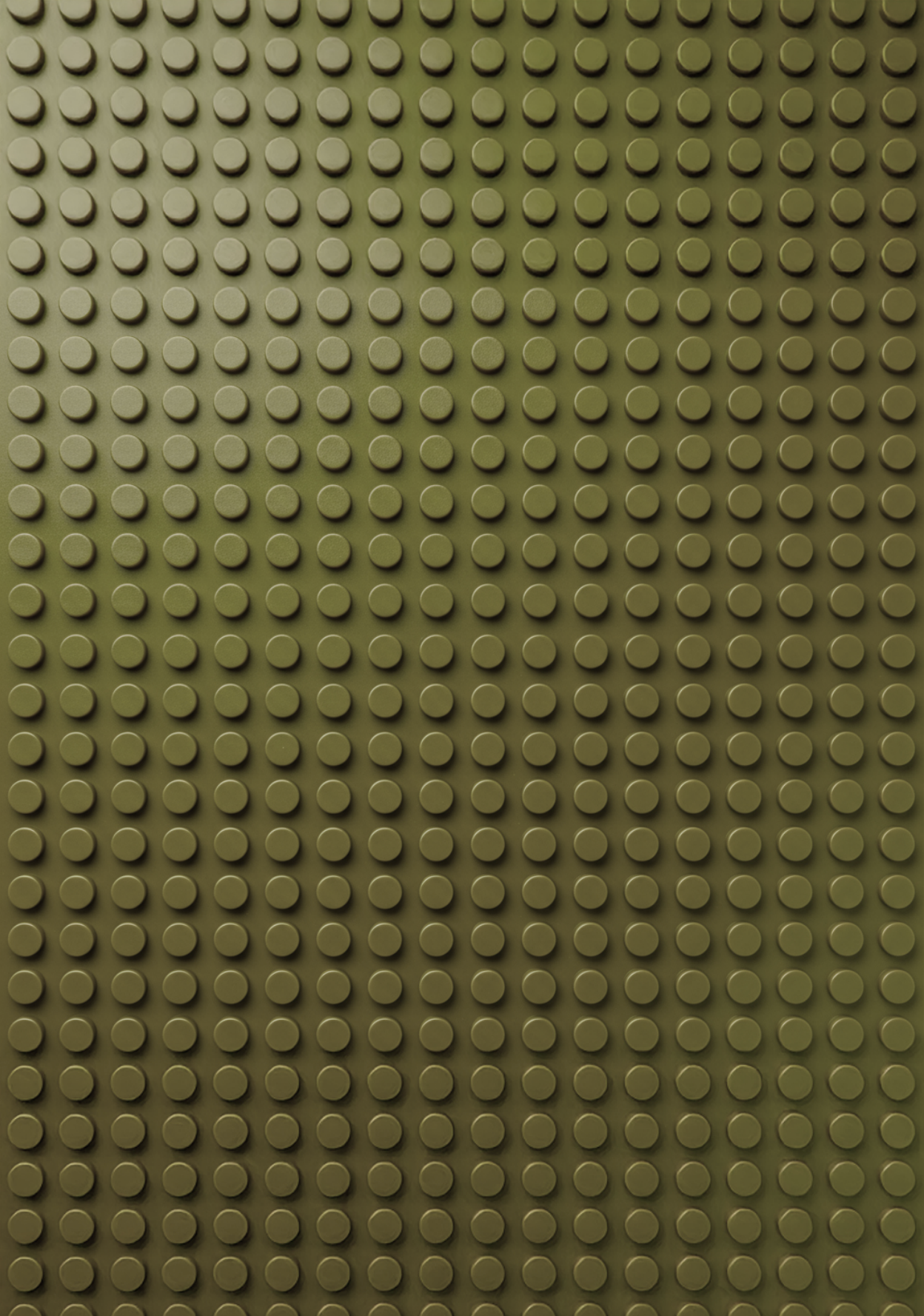
Tu robot LEGO® MINDSTORMS® EV3 se moverá hacia adelante durante 5 segundos.

***Recursos de aprendizaje PyBricks.***

Para dominar el uso de PyBricks en proyectos con el EV3, los siguientes recursos son muy recomendados:

- Sitio web oficial: contiene excelente documentación con explicación detallada de todos los módulos, clases y métodos disponibles en PyBricks: <https://pybricks.com>
- Tutorial oficial de Lego: provee una amplia guía paso a paso para construir y programar un robot tanque con PyBricks: <https://education.lego.com/en-us/support/mindstorms-ev3/python-for-ev3>
- Libro “Python for EV3” de Daniele Benedettelli: un completo manual para aprender a programar el EV3 con Python y PyBricks.
- EV3Dev: sitio web con gran información sobre programación del EV3 con MicroPython. <https://www.ev3dev.org/docs/programming-languages/python/>.
- Foros de la comunidad: sitios como EV3Dev, Stack Overflow y Reddit tienen foros activos para hacer preguntas y resolver dudas.





## **Capítulo 3**

# **Fundamentos de Python**



Python es un lenguaje de programación multiparadigma, muy popular por su facilidad de uso y sintaxis clara y legible. Fue creado por Guido van Rossum a principios de la década de 1990 con el objetivo de ser un lenguaje accesible para todo tipo de programadores, incluso principiantes.

Python es interpretado, dinámicamente tipado y multiplataforma. Cuenta con una amplia biblioteca estándar incorporada y una gran comunidad que desarrolla módulos y paquetes adicionales de código abierto, para expandir sus capacidades en distintas áreas de aplicación como ciencia de datos, desarrollo web, sistemas operativos, entre muchas otras (S. F. Python, 2021).

Python se ha consolidado como uno de los lenguajes de programación más populares y versátiles de los últimos tiempos. Desde aplicaciones web, videojuegos, inteligencia artificial hasta administración de sistemas, tiene un amplio uso en la industria tecnológica actual (Downey, 2012).

Su creciente adopción se debe en gran parte a que cuenta con una curva de aprendizaje suave para principiantes, pero con el poder de escalar a aplicaciones complejas. A continuación, se exploran los elementos fundamentales del lenguaje que todo programador Python debe conocer.



## Sintaxis Básica

La sintaxis de un lenguaje de programación comprende las reglas que gobiernan la estructura y elementos que componen los programas. Python se destaca por priorizar la legibilidad y facilidad de escritura mediante el uso de sangrías e indentaciones para delimitar bloques de código, en lugar de símbolos como paréntesis o llaves. Esto le brinda un formato visual muy ordenado y limpio (Sweigart, 2020).

Otra característica importante es que distingue entre mayúsculas y minúsculas (*case sensitive*), por lo que debemos mantener consistencia al nombrar variables y funciones. Los comentarios se indican con el símbolo de numeral (`#`), lo cual permite documentar y explicar diferentes partes del código sin afectar la ejecución. También se pueden hacer comentarios de varias líneas para explicar un bloque de código más grande, se inician con tres comillas simples (`'''`) o dobles (`"""`) al principio y al final del bloque.

Los programas de Python consisten en *scripts* o módulos almacenados en archivos con extensión `.py` que contienen las instrucciones interpretadas por el intérprete de Python.

La instrucción `Print` es una de las más básicas y utilizadas en el lenguaje de programación Python, se usa para mostrar salidas de datos en la consola de comandos. Las salidas de datos pueden ser mensajes que se escriben entre comillas simples o dobles (Lutz, 2013).

Un ejemplo de un script simple es:

```
'''  
    Este es un comentario de varias líneas  
    Explica el funcionamiento de este bloque de código  
'''
```

```
# Programa Hola Mundo  
print('Hola Mundo') # Imprime string  
print("Bienvenido a Python") # Otro string  
nombre = "Juan" # Variable nombre  
print(nombre) # Imprime variable
```

## Variables y Tipos de Datos

Las variables son contenedores que almacenan valores o información para ser referenciados y manipulados en un programa. Python es un lenguaje dinámicamente tipado, lo que significa que no es necesario declarar explícitamente el tipo de dato al definir una variable, esto se determina automáticamente al asignarle un valor (Sweigart, 2020). Python es sensible a las mayúsculas y minúsculas, lo que quiere decir que distingue entre letras mayúsculas y minúsculas, por lo que debemos tener cuidado al nombrar variables, funciones, etc.

Los nombres de variables no pueden iniciar con números, pero sí con guion bajo (\_). Se recomienda utilizar la nomenclatura “*snake case*” con guiones bajos, para separar palabras en nombres de variables y funciones.

Los principales tipos de datos que se usarán en el código Python son:

- Enteros (int): para valores numéricos sin decimales. Ejemplo: 25
- Flotantes (float): para valores numéricos con decimales. Ejemplo: 3.1416
- Booleanos (bool): para valores True o False.
- Strings (str): para cadenas de texto. Ejemplo: “Hola”

- Listas (list): colecciones de valores entre corchetes. Ejemplo: [1, 2, 3]
- Tuplas (tuple): listas inmutables entre paréntesis. Ejemplo: (1, True, "Hola")
- Diccionarios (dict): pares de clave-valor entre llaves. Ejemplo: {"nombre":"Juan","edad":20}

Un ejemplo de declaración de variables es:

```
# Variables en Python
entero = 15 # int
flotante = 10.5 # float
texto = "Bienvenido" # string
booleano = True # boolean
lista = [1, 2, 3] # list
tupla = ("a", "b", "c") # tuple
diccionario = {"nombre":"Ana", "edad":19} # dict
```

## Operadores y expresiones de asignación

Los operadores son símbolos que representan acciones aplicables a valores y variables para realizar cálculos u operaciones. Los principales tipos de operadores en Python son:

- Aritméticos: +, -, \*, /, %, \*\*, // para realizar operaciones matemáticas.
- Asignación: =, +=, -=, \*=, /= para asignar valores.
- Comparación: ==, !=, >, <, >=, <= para comparar expresiones.
- Lógicos: and, or, not para combinar expresiones booleanas.
- Bitwise: &, |, ^, >>, << para operar a nivel de bits (Downey, 2012).
- Algunos ejemplos de uso:

```
# Operadores aritméticos
5 + 3 # Suma
10 - 4 # Resta
4 * 5 # Multiplicación
16 / 4 # División da un float
18 % 7 # Módulo o resto
5 ** 3 # Potencia
15 // 4 # División entera

# Operadores comparación
5 == 5 # Igualdad
10 != 5 # Desigualdad
15 > 10 # Mayor que

# Operadores lógicos
True and False # Y lógico
True or False # O lógico
not True # Negación

# Operadores bitwise
8 & 5 # AND a nivel de bits
9 | 3 # OR a nivel de bits
11 ^ 5 # XOR a nivel de bits
```

Estos operadores permiten realizar todo tipo de operaciones matemáticas, lógicas y a nivel de bits en Python. Se pueden combinar para crear expresiones un poco más complejas dentro del código, por ejemplo:

```
precio = 25
descuento = 12
precio_final = precio - (precio * descuento / 100)
print(precio_final) # Imprime 22.0
```

Python respeta las reglas convencionales de precedencia de operadores: paréntesis, exponenciación, multiplicación/división, suma/resta. Esto determina el orden en que se evalúan las expresiones para obtener el resultado correcto, estos operadores permiten realizar todo tipo de manipulaciones y cálculos en un programa de Python según se necesite.

En Python existen diferentes operadores de asignación que permiten realizar una operación y asignación de forma simplificada en una sola expresión, el operador básico es el signo igual (=), que asigna el valor de la derecha a la variable de la izquierda (Sweigart, 2020):

```
x = 5 # asigna 5 a x
```

Además, están los operadores combinados como +=, -=, \*=, /=, que combinan la operación con la asignación (Downey, 2012):

```
x += 3 # x = x + 3
x -= 2 # x = x - 2
x *= 4 # x = x * 4
x /= 2 # x = x / 2
```

Estos permiten acortar y simplificar expresiones donde se quiere aplicar una operación a una variable existente.

También se puede encadenar múltiples asignaciones en una línea:

```
x = p = z = 0 # x, p y z asignados a 0
```

Donde los valores se asignan de derecha a izquierda.

En las asignaciones con expresiones se puede utilizar expresiones arbitrarias complejas del lado derecho:

```
x = (3 * 4) + (10 / 5)
```

Python evalúa la expresión, obtiene el resultado y luego lo asigna a la variable.

Las asignaciones múltiples permiten asignar varias variables en una sola expresión:

```
a, b, c = 10, 20, 30
```

Esto asigna 10 a 'a', 20 a 'b' y 30 a 'c' respectivamente.

Un uso muy práctico de la asignación múltiple es para intercambiar los valores de dos variables:

```
a = 10  
b = 20  
a, b = b, a # intercambia a y b  
# a ahora es 20, b ahora es 10
```

Las expresiones de asignación en Python permiten realizar operaciones y asignaciones simplificadas para escribir código más conciso y legible (Lutz, 2013), tienen varios usos para inicializar, modificar y manipular variables.

## Estructuras de Control

Las estructuras de control de flujo permiten controlar el orden en que se ejecutan las instrucciones en un programa Python, de modo que se pueda construir código más complejo y versátil que se adapte a distintas situaciones y requerimientos lógicos.

Python incluye varias estructuras de control que se verá en el apartado siguiente.

### Condicional *if*

La sentencia *if* permite ejecutar código sólo si se cumple una condición booleana específica. Su sintaxis es:

```
if condicion:
    # código a ejecutar
```

Donde condición puede ser cualquier expresión que evalúe a *True* o *False*. Por ejemplo:

```
x = 10
if x > 5:
    print("x es mayor a 5")
```

Esto imprimirá el mensaje porque la condición  $x > 5$  se cumple.

Se puede encadenar múltiples condiciones con *elif* (abreviación de *else if*):

```
color = "rojo"
if color == "verde":
    print("El color es verde")
```

```
elif color == "rojo":  
    print("El color es rojo")  
elif color == "azul":  
    print("El color es azul")  
else:  
    print("Color no reconocido")
```

Si ninguna condición *if* / *elif* se cumple, se ejecuta el bloque *else* opcional.

### **Ciclo for**

El ciclo *for* itera sobre los elementos de una secuencia que se repite, ejecutando el código en cada iteración. Por ejemplo, para iterar sobre una lista:

```
frutas = ["manzana", "banana", "kiwi"]  
for fruta in frutas:  
    print("La fruta es:", fruta)  
  
#0 sobre un rango numérico con range():  
  
for i in range(5):  
    print(f"i vale {i}")
```

Esto imprime los números del 0 al 4.

La letra "f" antes de las comillas indica que esta es una *f-string* (cadena formateada), lo que permite incluir variables directamente dentro de la cadena usando llaves {}. En este caso, se imprimirá el valor de "i" en cada iteración del ciclo.

También se puede iterar sobre caracteres de un *string* con *for*:



```
for letra in "Python":  
    print(letra)
```

## Ciclo while

El ciclo *while* ejecuta un bloque de código mientras se cumpla una condición booleana. Por ejemplo:

```
i = 0  
while i < 5:  
    print(f"i vale {i}")  
    i += 1 # incrementa i
```

Esto imprime el valor de "i" mientras 'i' sea menor que 5, actualizando la variable en cada iteración.

## Instrucciones break, continue, pass

Estas instrucciones brindan mayor control sobre los ciclos:

- *break*: Termina completamente el ciclo actual
- *continue*: Vuelve al comienzo del ciclo
- *pass*: Equivale a no hacer nada, útil como marcado de posición o *placeholder* ó sea un espacio en blanco o un símbolo que indica dónde se insertará una información particular más adelante

Por ejemplo:

```
# Rompe el ciclo si i es 3  
for i in range(10):  
    if i == 3:  
        break
```

```
print(i)
```

**# Pasa a la siguiente iteración si i es impar**

```
for i in range(10):  
    if i % 2 != 0:  
        continue  
    print(i)  
# Hace "nada"  
for i in range(10):  
    pass # placeholders
```

Dominar el uso de estructuras de control en Python permite crear programas versátiles capaces de hacer frente a todo tipo de situaciones y requerimientos lógicos; estas construcciones de flujo son parte integral de cualquier lenguaje de programación.

## Funciones

Las funciones son una característica esencial en cualquier lenguaje de programación. Una función agrupa un bloque de código para realizar una tarea o cálculo específico, lo cual promueve la reutilización, organización y modularidad del código (Lutz, 2013).

En Python se puede reconocer fácilmente las funciones porque su nombre va seguido de paréntesis, veamos en detalle el concepto, sintaxis, tipos y usos de funciones.

### ***¿Qué es una función?***

Una función es un bloque de código identificado por un nombre, que puede ser ejecutado invocando dicho nombre y que permite

encapsular una secuencia de sentencias para realizar una tarea específica. Las funciones pueden recibir entrada en forma de argumentos y parámetros, procesarlos de algún modo, y devolver una salida o resultado, esto permite reutilizar las mismas instrucciones repetidamente sin tener que escribir el código desde cero (Sweigart, 2020).

### ***Sintaxis de funciones***

La sintaxis básica para definir una función en Python es:

```
def nombre_funcion(parametros):  
    # cuerpo de la función  
    return resultado # opcional
```

Donde:

- *def*: indica que se está definiendo una función.
- *nombre\_funcion*: es el identificador de la función.
- *Parámetros*: son los nombres para los datos de entrada. (Opcionales).
- *return* devuelve un resultado opcional al terminar.

Para ejecutar o llamar a la función se escribe su nombre seguido de paréntesis ():

```
nombre_funcion() # ejecuta la función
```

### ***Parametrizando funciones***

Se puede definir funciones que reciban parámetros o argumentos como entrada, estos se incluyen entre paréntesis en la definición

```
def suma(a, b):  
    total = a + b  
    return total  
  
resultado = suma(5, 8) # pasa 5 y 8 a los parámetros  
  
print(resultado) # Imprime 13
```

Los parámetros actúan como variables locales dentro de la función.

### Valores por defecto.

Es posible asignar valores por defecto a los parámetros para cuando no se pasen argumentos:

```
def saludar(nombre="Amigo"):  
    print(f"Hola {nombre}")  
  
saludar("Ana") # Salida: Hola Ana  
saludar() # Salida: Hola Amigo
```

Esto hace la función más flexible y reusable.

### Devolviendo valores.

Con *return* se puede devolver o retornar un valor desde la función:

```
def suma(a, b):  
    return a + b  
  
resultado = suma(10, 5) # se asigna el return a resultado
```

Si no hay *return*, la función devuelve *None* por defecto.

Dependiendo de si una función devuelve un valor mediante *return* o no, se pueden clasificar en dos tipos:

- Funciones fructíferas: aquellas que devuelven un valor utilizando la sentencia *return*.
- Funciones no fructíferas (también llamadas procedimientos): aquellas que no devuelven ningún valor explícito; en su lugar, retornan *None* por defecto.

Esta clasificación es útil para comprender el propósito y el uso de cada tipo de función dentro de un programa (Downey, 2012).

### **Tipos de funciones Python.**

En Python se encuentran distintos tipos de funciones:

- Funciones integradas: ya vienen incorporadas con Python como `print()`, `len()`, `int()`, etc.
- Módulos de la librería estándar: funciones agrupadas para realizar tareas específicas, como matemáticas, entrada/salida, web, OS, etc.
- Funciones definidas por el usuario: las que se crean en los programas para reutilizar código.
- Funciones anónimas o lambda: funciones sin nombre creadas en una sola línea con la palabra reservada `lambda`.
- Funciones recursivas: que se llaman a sí mismas para dividir problemas grandes en partes más pequeñas.

Veamos algunos ejemplos prácticos del uso de funciones en Python:

- Función para calcular el cubo de un número:

```
def cubicar(x):  
    return x ** 3  
  
resultado = cubicar(3)  
print(resultado) # 27}
```

- Función para saludar pasando nombre:

```
def saludar(nombre):  
    print(f"¡Hola, {nombre}!")  
  
saludar("Ana") # ¡Hola, Ana!  
saludar("Pedro") # ¡Hola, Pedro!
```

- Funciones recursivas: las funciones recursivas son un tipo de función en Python que se caracteriza por llamarse a sí misma dentro de su definición. Esto permite dividir problemas complejos en problemas más pequeños cada vez, hasta llegar a un caso base sencillo (Downey, 2012).

La recursividad es una técnica muy útil para problemas que se pueden descomponer naturalmente en partes más simples del mismo tipo. Un ejemplo clásico son los algoritmos para calcular números factoriales:

```
def factorial(x):  
    if x == 1:  
        return 1  
    else:  
        return x * factorial(x-1)  
print(factorial(5)) # 120
```

Analicemos este ejemplo:

- La función factorial() se llama a sí misma dentro del *else*.
- El caso base es cuando  $x == 1$ , ahí se devuelve 1 y se termina la recursión.
- La llamada recursiva es factorial( $x - 1$ ) que reduce el problema en 1 cada vez
- De esta forma se calcula:
  - factorial(5) = 5 \* factorial(4)
  - factorial(4) = 4 \* factorial(3)
  - factorial(3) = 3 \* factorial(2)
  - factorial(2) = 2 \* factorial(1)
  - factorial(1) = 1 (caso base)
- Y se resuelve el problema combinando las llamadas recursivas.
- La recursión requiere:
  - Un caso base que termine la recursión.
  - Una llamada recursiva que acerque al caso base.

Ventajas:

- Soluciones elegantes y simples para problemas complejos.
- Código más corto y fácil de entender que iteraciones.

Desventajas:

- Alto consumo de memoria por las múltiples llamadas.
- Riesgo de recursión infinita si no hay caso base.

La recursión es una técnica avanzada que con práctica permite escribir algoritmos muy eficientes en Python. Debe usarse con precaución para evitar problemas de rendimiento o infinitos.

- Función lambda para sumar dos números:

```
sumar = lambda x, y: x + y
```

```
resultado = sumar(10, 15)

print(resultado) # 25
```

Las funciones son bloques de código reutilizables que permiten crear programas Python más modulares, organizados y fáciles de mantener. Existen muchos tipos de funciones que se pueden aprovechar en los proyectos que se desarrollen.

## Clases y Objetos

Python es un lenguaje que soporta ampliamente la programación orientada a objetos mediante clases y objetos. Esta es una forma muy útil y poderosa de organizar y diseñar los programas.

### ¿Qué es una clase?

Una clase es como un plano o *blueprint* para crear objetos. Define un conjunto de atributos (datos) y métodos (funciones) que serán compartidos por todos los objetos creados a partir de esa clase (Lott & Banffy, 2022).

Las clases permiten modelar entidades del mundo real, como personas, vehículos, cuentas bancarias, etc. Cada objeto creado a partir de la clase se denomina instancia y contiene sus propios datos.

La sintaxis para definir una clase en Python es:

class NombreClase:

```
# atributos de clase

def __init__(self):
    # constructor initializer
```



```
# métodos de clase

def nombre_metodo(self):
    # código del método
```

Donde `__init__()` es el constructor que se ejecuta al crear un objeto, *self* hace referencia al objeto actual.

### Creando objetos.

Se pueden crear instancias u objetos a partir de una clase usando la notación:

```
obj1 = NombreClase() # crea objeto 1
obj2 = NombreClase() # crea objeto 2
```

Cada objeto tendrá sus propios atributos y compartirán los mismos métodos de clase.

### Atributos.

Los atributos almacenan los datos asociados a un objeto en particular, se definen dentro de la clase y se accede con la notación punto:

```
obj1.atributo = valor # asignar valor
valor = obj1.atributo # acceder valor
```

### Métodos.

Los métodos son funciones que representan comportamientos de los

objetos, se definen en la clase y se acceden también con notación punto:

```
obj1.metodo() # llamar método
```

Dentro de los métodos se utiliza *self* para referir los atributos del objeto actual.

Ejemplo de clase Persona: veamos una clase Persona con atributos nombre y edad, y métodos presentarse y cumplir\_años():

```
class Persona:
```

```
    def __init__(self, nombre, edad):  
        self.nombre = nombre # atributo  
        self.edad = edad # atributo
```

```
    def presentarse(self):  
        print(f"Hola soy {self.nombre}, tengo {self.edad} años")
```

```
    def cumplir_anos(self):  
        self.edad += 1
```

```
# crear objetos
```

```
personal = Persona("Juan", 30)
```

```
persona2 = Persona("Ana", 18)
```

```
# llamar métodos
```

```
personal.presentarse() # Hola soy Juan, tengo 30 años
```

```
persona2.cumplir_anos() # actualiza edad a 19
```

El ejemplo anterior demuestra los conceptos centrales de programación orientada a objetos en Python.

Una característica poderosa de manejar objetos es la herencia entre clases, esto permite crear clases que hereden atributos y métodos de una clase padre, pudiendo definir comportamiento propio, la herencia proporciona reutilización de código y modelado de relaciones entre entidades del mundo real. La clase hija se define indicando entre paréntesis la clase padre de la que hereda.

Las clases y objetos en Python permiten crear programas mejor estructurados, reutilizables y fáciles de mantener, es un paradigma fundamental para desarrollar sistemas complejos de forma organizada y eficiente.

## Bibliotecas en Python

Una biblioteca en Python es un conjunto de funciones y clases que se pueden utilizar en un programa, estas se pueden encontrar en línea o en el repositorio de paquetes de Python (PyPI).

Para utilizar una biblioteca en Python, primero se debe importar, esto se puede hacer usando el comando *import*. Por ejemplo, para importar la biblioteca *math*, se puede usar el siguiente código:

```
import math
```

Una vez que se ha importado una biblioteca, se puede acceder a sus funciones y clases usando el operador punto (.). Por ejemplo, para acceder al valor de pi (*math.pi*), se puede usar el siguiente código:

```
import math  
pi = math.pi  
print(pi)
```

## ***Tipos de bibliotecas en Python***

Las bibliotecas en Python se pueden clasificar en dos tipos principales:

- Bibliotecas estándar: estas bibliotecas forman parte del lenguaje Python, y están disponibles de forma predeterminada.
- Bibliotecas de terceros: estas bibliotecas son desarrolladas por terceros, y deben instalarse por separado.

### **Bibliotecas estándar de Python.**

La biblioteca estándar de Python incluye una amplia gama de funciones y clases que pueden utilizarse para realizar tareas comunes, algunas de las bibliotecas estándar más importantes son:

- Biblioteca *math*: proporciona funciones para el cálculo numérico, como el cálculo de raíces cuadradas, logaritmos y funciones trigonométricas.
- Biblioteca *random*: ofrece funciones para generar números aleatorios.
- Biblioteca *datetime*: suministra funciones para trabajar con fechas y horas.
- Biblioteca *os*: otorga funciones para trabajar con el sistema operativo.
- Biblioteca *sys*: provee funciones para acceder a información sobre el sistema.

## **Bibliotecas de terceros en Python.**

Existen muchas bibliotecas de terceros disponibles para Python, suelen ser elaboradas por comunidades de desarrolladores y proporcionar funciones y clases especializadas para tareas específicas, algunas de las bibliotecas de terceros más populares son:

- Biblioteca *numpy*: ofrece funciones para el cálculo numérico de alto rendimiento.
- Biblioteca *scipy*: suministra funciones para el análisis estadístico y científico.
- Bibliotecas *pandas*: otorga funciones para trabajar con datos tabulares.
- Biblioteca *matplotlib*: provee funciones para la visualización de datos.
- Biblioteca *flask*: proporciona un marco web para el desarrollo de aplicaciones web.

## ***Cómo instalar bibliotecas de terceros en Python.***

Para instalar una biblioteca de terceros en Python, se puede usar el administrador de paquetes de Python (pip). Para instalar una biblioteca de terceros usando pip, es posible usar el siguiente comando:

- `pip install [nombre_de_la_biblioteca]`

Por ejemplo, para instalar la biblioteca *numpy*, se puede usar el siguiente comando:

- `pip install numpy`

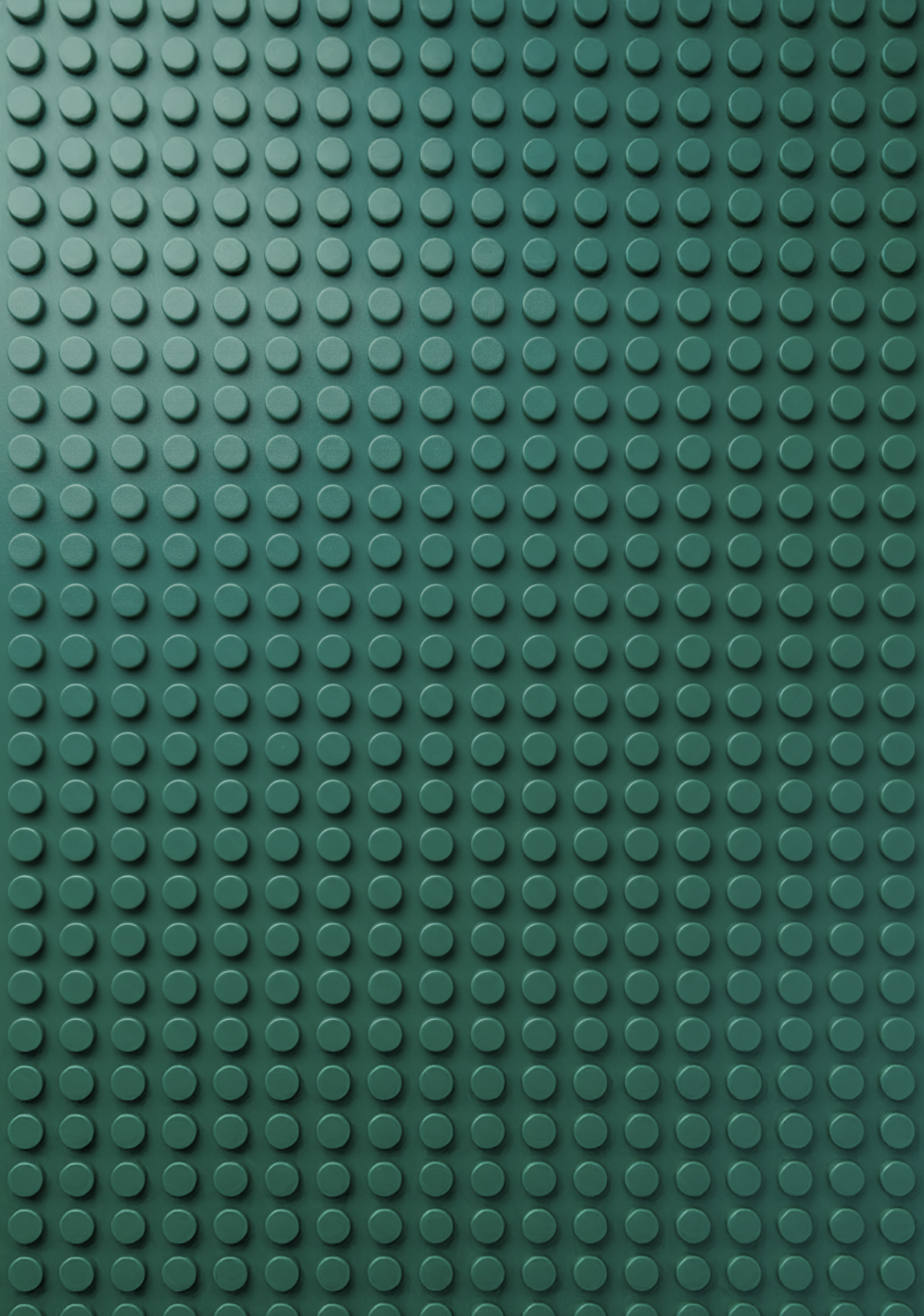
## ***Ventajas de las bibliotecas en Python.***

Las bibliotecas en Python ofrecen una serie de ventajas, entre las

que se incluyen:

- Reutilización de código: las bibliotecas permiten reutilizar código que ya ha sido desarrollado y probado por otros desarrolladores, esto ahorra tiempo y esfuerzo.
- Especialización: las bibliotecas proporcionan funciones y clases especializadas para tareas específicas, esto permite centrarse en la lógica del programa, sin tener que preocuparse por implementar las funciones y clases básicas.
- Documentación: las bibliotecas suelen estar bien documentadas, lo que facilita su uso.

Las bibliotecas en Python son una herramienta esencial para cualquier programador de programa, permiten reutilizar código, especializarnos en tareas específicas y ahorrar tiempo y esfuerzo.



## **Capítulo 4**

# **Programación del Lego® Mindstorms® EV3 Usando la Biblioteca**





La biblioteca PyBricks de Python permite programar el robot LEGO® MINDSTORMS® EV3 de una forma muy sencilla e intuitiva, en este capítulo se verá cómo aprovechar PyBricks para controlar el EV3 mediante código Python escrito en el computador.

### El ladrillo del LEGO® MINDSTORMS® EV3

El ladrillo o *brick* programable EV3 es el cerebro del robot LEGO® MINDSTORMS® EV, se encarga de recibir instrucciones de los programas y coordinar la interacción entre los motores, sensores, luces y otros componentes del robot (LEGO®, 2021b).

**Figura 13.**  
*Ladrillo LEGO® MINDSTORMS® EV3*



*Nota.* La imagen representa el dispositivo Ladrillo. Fuente:LEGO® (2021a).

El EV3 es básicamente una pequeña computadora embebida optimizada para la robótica educativa, cuenta con un procesador ARM9 a 300 MHz, 64 MB de memoria RAM y 16 MB de almacenamiento flash ampliable con tarjeta SD (LEGO®, 2016).

El sistema operativo que corre internamente en el EV3 es Linux, lo que le brinda gran potencia y flexibilidad. El *firmware* oficial de Lego contiene el *software* EV3-G, basado en iconos para programar mediante arrastrar y soltar comandos. Una solución más sencilla es usar la biblioteca PyBricks de Python, esta se instala directamente en el computador e interactúa de forma inalámbrica con el EV3, sin necesidad de modificar el sistema operativo del robot (Valk & Pybricks, 2022).

Sin embargo, también es posible reemplazar el *firmware* por alternativas como EV3Dev o EV3 micropython, que permiten programar el robot en lenguajes de texto como Python, esto le da al usuario mucho más control y funcionalidades avanzadas.

### **Componentes del EV3**

- El ladrillo EV3 contiene los siguientes componentes principales:
- Puerto USB: para conectar a la PC mediante cable USB.
- Puertos de entrada: para conectar sensores y obtener datos del entorno.
- Puertos de salida: para conectar motores y actuadores.
- Pantalla LCD: pequeña pantalla para mostrar texto, imagen y datos.
- Botones: botones de navegación arriba, abajo, izquierda, derecha, central y atrás.
- Bocina: para reproducir sonidos y efectos de audio.
- Luz de estado: led indicador de encendido, apagado, batería, etc.

Todos estos componentes pueden ser controlados y programados utilizando PyBricks para ejecutar las más diversas operaciones y tareas en el robot, veamos un programa muy simple para encender un led en el EV3 usando PyBricks en Python:

```
#!/usr/bin/env pybricks-micropython
from pybricks.hubs import EV3Brick
from pybricks.tools import wait
from pybricks.parameters import Color
'''
    Este programa controla la luz del ladrillo EV3
    para mostrar una luz roja durante un segundo.
'''
def main():
    # Inicializar el EV3
    ev3 = EV3Brick()

    # Encender una luz roja
    ev3.light.on(Color.RED)

    # Esperar un 5 segundos
    wait(5000)

    # Apagar la luz
    ev3.light.off()

#-----
if __name__ == "__main__":
    main()
```

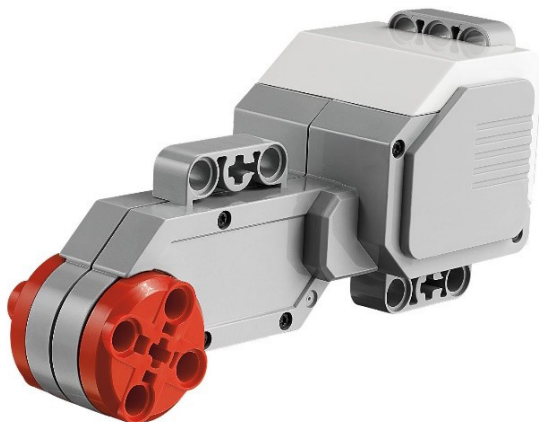
Sólo se necesita importar la clase EV3Brick, luego obtener una referencia al ladrillo e indicar al led que se encienda, esta es precisamente la ventaja de PyBricks, la simplicidad y facilidad de uso, ideal para introducir a nuevos usuarios en la robótica con Python de forma rápida e intuitiva.

Los robots Lego Mindstorms EV3 pueden ser equipados con una gran variedad de motores, sensores y otros componentes que les permiten moverse, percibir su entorno y ejecutar tareas muy sofisticadas. Veamos cómo programar estos dispositivos usando Python y la biblioteca PyBricks.

### ***Motores del LEGO® MINDSTORMS® EV3***

Uno de los componentes más importantes del robot Lego Mindstorms EV3 son sus motores interactivos, los cuales permiten dotar de movimiento y acción a los modelos que se construyan, existen diferentes tipos de motores EV3, cada uno con propiedades y características únicas.

**Figura 14.**  
*Motor L Servo Grande LEGO® MINDSTORMS® EV3*



*Nota.* La imagen representa el dispositivo Motor L Servo Grande. Fuente: LEGO® (2021a).

### **Tipos de motores EV3.**

El kit básico del EV3 contiene 3 servomotores cuyo funcionamiento se basa en un motor eléctrico de corriente directa combinado con un *encoder* o sensor de rotación incremental, que permite monitorear con gran precisión parámetros como velocidad, dirección y posición (LEGO®, 2016).

- Estos 3 motores son idénticos en su funcionamiento excepto por su tamaño y forma:
- Motor grande: tiende una velocidad sin carga de 160 aproximadamente y torque de 20 N-cm (más lento, pero más potente).
- Motor mediano: velocidad de 240 sin carga y torque de 8 N-cm. Es más rápido que el grande, pero menos potente (más rápido, pero menos potente).
- Motor mini: alcanza 390 sin carga y tiene un torque de 1.5 N-cm. Para proyectos pequeños por su reducido tamaño.

**Figura 15.**

*Motor L Servo Mediano LEGO® MINDSTORMS® EV3*



*Nota.* La imagen representa el Motor L Servo Mediano. Fuente:LEGO® (2021a).

También existen otras opciones, como el motor eléctrico, el servo motor y el turbo motor, que se venden por separado y amplían las capacidades del EV3 (LEGO®, 2016).

### Control de motores usando la biblioteca Pybricks.

Esta librería es una excelente opción, enfocada en facilidad de uso, permite controlar el EV3 vía Bluetooth desde Python en el PC utilizando una sintaxis muy clara y minimalista, veamos un ejemplo de control de motores con PyBricks:

```
#!/usr/bin/env pybricks-micropython
from pybricks.hubs import EV3Brick
from pybricks.ev3devices import Motor
from pybricks.parameters import Port
from pybricks.tools import wait

def main():
    # Inicializar el EV3
    ev3 = EV3Brick()

    # Inicializar un motor en el puerto A
    test_motor = Motor(Port.A)

    # Mueve el motor a una velocidad rotacional de
    # 500 grados por segundo, durante 5 segundos
    test_motor.run(500)
    wait (5000)
    # Detiene el motor
```

```
test_motor.stop()  
  
#-----  
  
if __name__ == "__main__":  
    main()
```

Se puede hacer operaciones más avanzadas como:

- Encender el motor hacia atrás o adelante.
- Rampas de aceleración/desaceleración.
- Rotar a un ángulo absoluto o relativo específico.
- Leer precisamente la posición y velocidad actual.

Los motores le dan “vida” a los robots permitiéndoles desplazarse y realizar trabajo físico de forma autónoma mediante la programación en Python.

### Consideraciones de diseño.

Al integrar los motores en un robot EV3 se debe tomar en cuenta ciertas consideraciones de diseño (LEGO®, 2016):

- Tracción: relación entre torque y fricción con el piso, importante en robots móviles.
- Holgura: juego mecánico en ruedas y ejes que puede afectar precisión.
- Centro de masa: debe estar bien ubicado para estabilidad.
- Relación de transmisión: velocidad de rotación de las ruedas respecto al motor.
- Fricción en las ruedas: superficies con mayor tracción proveen mejor control.

Combinando una construcción mecánica adecuada con una



programación rigurosa en Python, es posible crear robots LEGO® MINDSTORMS® EV3 verdaderamente asombrosos y competitivos.

Los motores son el corazón del sistema robótico del EV3 y permiten dar rienda suelta a la creatividad e imaginación del programador para construir todo tipo de máquinas y vehículos integrados con Python. No solo existen tres servomotores estándar, sino otros motores compatibles con el EV3 que permiten expandir las posibilidades de los proyectos:

- Motor eléctrico: similar a los servomotores, pero más económico y sin encoder integrado. Útil para accionar ruedas en robots simples.
- Servo motor: permite control de posición angular exacta entre (0 y 300) grados. Ideal para brazos robóticos u otras articulaciones.
- Turbo motor: tiene engranaje reforzado y mayor torque, hasta 25 N-cm. Para proyectos de alta potencia.
- Motor mediano con tacómetro: versión del motor mediano con sensor de rotación absoluto incorporado. Mayor precisión (LEGO®, 2016).

La biblioteca PyBricks soporta sin problemas estos motores alternativos, permitiendo controlarlos fácilmente con Python al igual que los servomotores estándar, solamente debemos especificar el tipo de motor al crear el objeto:

```
from pybricks.ev3devices import ElectricMotor,
ServoMotor, TachoMotor

motor_electrico = ElectricMotor(Port.A)
servo_motor = ServoMotor(Port.B)
```

```
tacho_motor = TachoMotor(Port.C)
```

De esta forma es posible combinar distintos motores según los requerimientos específicos de cada proyecto de robot EV3.

### ***La clase Motor de la biblioteca PyBricks***

Esta clase incluye varios métodos útiles para controlar los motores del LEGO® MINDSTORMS® EV3 (Valk & Pybricks, 2022):

- `run(speed)`: hace girar el motor a una velocidad constante, donde `speed` es la velocidad rotacional del motor (en grados/s).
- `stop()`: detiene el motor suavemente.
- `brake()`: frena bruscamente el motor.
- `run_time(speed, time, then=Stop.COAST, wait=True)`: hace girar el motor a una velocidad constante durante un tiempo dado, donde `speed` es la velocidad rotacional del motor (en grados/s), `time` (en ms) es la duración del movimiento del motor, `then=Stop` indica que debe hacer el hacer motor después del tiempo de movimiento, y `wait (bool)` se usa de la misma manera que el caso anterior.
- `run_angle(speed, rotation_angle, then=Stop.HOLD, wait=True)`: hace girar el motor a una velocidad constante un ángulo específico, donde `speed` (en grados/s) es la velocidad rotacional del motor, `rotation_angle` (en grados) es el ángulo que el motor debe rotar, `then=Stop` indica que hace el motor después de la rotación, y `wait (bool)` permite al motor esperar a que el movimiento se complete antes de que continúe con el programa.
- `run_target(speed, target_angle, then=Stop.HOLD, wait=True)`:

hace girar el motor a una velocidad constante hasta alcanzar un ángulo absoluto como objetivo, donde *speed* (en grados/s) es la velocidad rotacional del motor (puede ser positiva o negativa), *target\_angle* (en grados) es el ángulo que el motor debe rotar, *then=Stop* indica que hace el motor después de la rotación, y *wait (bool)* indica si el motor espera a que el movimiento se complete antes de que continúe con el programa.

- `track_target(target_speed)`: gira el motor para alcanzar y mantener una velocidad objetivo.
- `hold()`: Mantiene el motor en su posición actual estable.
- `reached_angle(angle)`: retorna *True* si el motor alcanzó cierto ángulo.
- `speed()`: devuelve la velocidad actual en .
- `angle()`: devuelve el ángulo actual del motor en grados.

La clase `Motor` de `PyBricks` proporciona un amplio control sobre los motores del EV3 en Python. A continuación, se presentan algunos ejemplos:

Ejemplo moviendo un motor a una velocidad fija:

```
from pybricks.ev3devices import Motor
from pybricks.parameters import Port

motor = Motor(Port.B)

# mueve el motor a una velocidad
# rotacional de 50 grados por segundo
motor.run(50)
```

Ejemplo rotando el motor un ángulo específico:

```
# mueve el motor a una velocidad  
# rotacional constante de 30 grados sobre  
# segundo en un ángulo de 360 grados  
motor.run_angle(30, 360)
```

Ejemplo deteniendo el motor suavemente:

```
# Detener motor lentamente  
motor.stop()
```

Ejemplo verificando si el motor alcanzó cierto ángulo:

```
if motor.reached_angle(90):  
    ev3.screen.print("Ángulo alcanzado!")
```

Ejemplo leyendo valores del motor:

```
ev3.screen.print(motor.speed()) # Velocidad actual  
ev3.screen.print(motor.angle()) # Ángulo actual
```

La API de motores de PyBricks facilita tareas comunes de control de forma concisa y clara.

Los motores EV3 junto con Python entregan todas las herramientas para hacer realidad las más ambiciosas ideas de proyectos robóticos como la implementación de robots sumo, vehículos todo terreno, robots que navegan entre laberintos, brazos robóticos articulados, drones cuadrúpedos e impresoras 3D, ¡Tu imaginación es el único límite!

## Sensores EV3

Algunos sensores comunes del EV3 son:

- Sensor de color: detecta intensidad y color de objetos mediante luz visible o infrarroja.
- Sensor de ultrasonido: calcula distancia a objetos mediante ondas de sonido, con un alcance máximo 2 metros.
- Sensor giroscópico: mide la orientación y ángulos de inclinación del EV3.
- Sensor táctil: detecta toques y golpes como un interruptor simple.
- Sensor infrarrojo: puede detectar luz infrarroja reflejada por objetos macizos (LEGO®, 2021b).

Para leer un sensor con PyBricks, se crea una instancia de su clase correspondiente y llamamos métodos como `.color()` o `.distance()`:

```
from pybricks.ev3devices import UltrasonicSensor

sonar = UltrasonicSensor(Port.S4)

distancia = sonar.distance() # lectura en mm
if distancia < 200:
    # detener motores si obstáculo cerca
```

### ***Sensor de color***

El sensor de color es uno de los sensores más versátiles del robot Lego Mindstorms EV3, permite detectar colores e intensidad de luz en objetos y superficies. A continuación, se detallan sus capacidades y cómo se puede programar en Python con la biblioteca PyBricks.

**Figura 16.**  
*Sensor de Color LEGO® MINDSTORMS® EV3*



*Nota.* La imagen representa el dispositivo de sensor de color. Fuente:LEGO® (2021a).

El sensor de color del EV3 utiliza un emisor de luz y un sensor CMOS para detectar la intensidad y frecuencia de la luz reflejada, funciona tanto con luz visible como infrarroja (LEGO®, 2021b).

### **Especificaciones.**

Sus especificaciones técnicas son:

- Tipo: sensor de luz ambiental y de color
- Entradas: 6 componentes de luz (rojo, azul, verde, rojo intenso, azul intenso, verde intenso)
- Resolución: 3% de intensidad de luz ambiental
- Rango de medición: de 0 % a 100 % de intensidad de reflexión
- Modos: color, intensidad reflejada, sensor de luz ambiental
- Interfaz: I2C y SMBus
- Alcance: (1 - 2) cm aproximadamente con fuente de luz integrada

El sensor puede identificar 7 colores primarios (negro, azul, verde, amarillo, rojo, blanco y marrón), y detecta intensidad de luz ambiental y niveles de gris; requiere calibración inicial en algunos casos para máxima precisión.

### Programación con PyBricks.

La biblioteca PyBricks de Python provee una clase ColorSensor para interactuar fácilmente con el sensor de color del EV3. Un ejemplo sencillo es:

```
#!/usr/bin/env pybricks-micropython
from pybricks.hubs import EV3Brick
from pybricks.ev3devices import ColorSensor
from pybricks.parameters import Port
from pybricks.tools import wait

'''
    Este programa captura el nombre de color de un
    objeto y la intensidad de luz ambiental en una
    escala. del 0 al 10.
'''

def main():
    # Inicializar el EV3
    ev3 = EV3Brick()

    # Inicializar el sensor de color en el puerto 4
    sensor_color = ColorSensor(Port.S4)
```

```
# Imprimir el color y la intensidad de luz ambiental

# en la pantalla del EV3
ev3.screen.print(sensor_color.color())
ev3.screen.print(sensor_color.ambient())

# Esperar 5 segundos
wait(5000)

#-----
if __name__ == "__main__":
    main()
```

El método `.color()` devuelve el color detectado como valor enumerado (`Color.BLACK`, `Color.BLUE`, etc). Para calibrar el sensor se utiliza el método `.calibrate_white()` colocando una superficie blanca frente al sensor, esto mejora la precisión de detección.

También es posible configurar el sensor en otros modos de operación:

```
# Modo de reflexión de luz
sensor_color.mode = ColorSensor.REFLECT
ev3.screen.print(sensor_color.reflection())

# Modo de detección de luz ambiental
sensor_color.mode = ColorSensor.AMBIENT
ev3.screen.print(sensor_color.ambient())
```

Algunos ejemplos prácticos de uso del sensor de color:

- Seguir una línea negra sobre fondo blanco.



- Identificar y clasificar objetos por color.
- Detectar tipos de superficie por intensidad reflejada.
- Leer códigos de colores o patrones.
- Calibrar con un color de referencia antes de lecturas precisas.

El sensor de color resulta indispensable en una gran variedad de proyectos robóticos donde se requiere que el EV3 obtenga información sobre colores y luz del entorno. Combinado con Python y PyBricks, permitirá implementar comportamientos autónomos fascinantes.

### **Sensor de ultrasonido**

El sensor de ultrasonido es uno de los dispositivos de entrada más útiles en el LEGO® MINDSTORMS® EV3. Permite al robot detectar objetos y calcular distancias en un rango aproximado de (0 a 250) cm. A continuación, más detalle del funcionamiento y cómo programarlo con Python.

**Figura 17.**  
*Sensor de Color LEGO® MINDSTORMS® EV3*



*Nota.* La imagen representa el sensor de color de LEGO®. Fuente:LEGO® (2021a).

## Especificaciones.

El sensor de ultrasonido del EV3 funciona emitiendo ondas de ultrasonido a 40 kHz y midiendo el tiempo que tardan en rebotar en un objeto y regresar (LEGO®, 2021b).

Sus especificaciones técnicas son:

- Tipo: sensor de proximidad por ultrasonido.
- Rango de detección: de (1 a 250) cm aproximadamente.
- Resolución: +/- 1 cm.
- Ángulo efectivo: 15 grados aproximadamente.
- Interfaz: I2C y SMBus.
- Alimentación: (5-6) VDC.

El sensor incorpora un emisor de ultrasonido, un micrófono, circuitos de control y un procesador ARM7 que se encarga de los cálculos para determinar distancia. Requiere calibración ocasional para máximo rendimiento.

## Programación con PyBricks.

La biblioteca PyBricks provee una clase `UltrasonicSensor` para interactuar fácilmente con este sensor en Python. Un ejemplo básico es:

```
#!/usr/bin/env pybricks-micropython
from pybricks.hubs import EV3Brick
from pybricks.ev3devices import UltrasonicSensor
from pybricks.parameters import Port
from pybricks.tools import wait

''' Este programa lee la distancia en mm de un
    sensor de ultrasonido
```

```
'''  
  
def main():  
    # Inicializar el EV3  
    ev3 = EV3Brick()  
  
    # Inicializar el sensor de ultrasonido  
    # en el puerto 2  
    sensor_ultrasonido = UltrasonicSensor(Port.S2)  
  
    # Imprimir la distancia en mm  
    ev3.screen.print(sensor_ultrasonido.distance())  
  
    # Esperar 5 segundos  
    wait(5000)  
  
    #-----  
    if __name__ == "__main__":  
        main()
```

El método `.distance()` dispara un pulso de ultrasonido y calcula la distancia al objeto más cercano detectado en un rango de 0 mm a 2550 mm.

Otros métodos útiles son:

- `.presence()`: devuelve True/False si detecta objeto dentro de 200 mm.
- `.listen()`: retorna el tiempo entre un eco inicial y un eco final en microsegundos.

- Algunos ejemplos de uso del sensor de ultrasonido:
- Evitar obstáculos en un robot móvil midiendo distancia a objetos.
- Calcular distancia desde el EV3 a una pared u otro robot.
- Estimar volumen de objetos sólidos midiendo diferentes lados.
- Crear un seguidor de objetos manteniendo cierto rango de distancia.
- Generar un escaneo 360 grados girando el sensor para mapear entorno.

Por ejemplo, para detectar objetos a menos de 10 cm de distancia:

```
#!/usr/bin/env pybricks-micropython
from pybricks.hubs import EV3Brick
from pybricks.ev3devices import UltrasonicSensor
from pybricks.parameters import Port
from pybricks.tools import wait

'''
Este programa detecta objetos usando el sensor
ultrasónico y emite un pitido cuando detecta un
objeto a menos de 100 mm de distancia
'''

def main():
    # Inicializar el EV3
    ev3 = EV3Brick()

    # Inicializar el sensor de ultrasonido en el
    puerto 2
```

```
sensor_ultrasonido = UltrasonicSensor(Port.S2)

# Repetir 10 veces
for i in range(10):
    d=sensor_ultrasonido.distance()
    ev3.screen.print("Distancia --> ",d)
    # Si la distancia es menor a 100 mm
    if (d < 100):
        # Emitir un pitido
        ev3.speaker.beep()
        # Esperar 0.5 segundos
        wait(500)

#-----
if __name__ == "__main__":
    main()
```

El sensor de ultrasonido abre un mundo de posibilidades de interacción autónoma del EV3 con su entorno físico mediante medición de distancias con el poder de Python.

### ***Sensor Giroscópico del LEGO® MINDSTORMS® EV3***

El sensor giroscópico es un dispositivo clave para dotar al robot LEGO® MINDSTORMS® EV3 de mayor orientación y equilibrio. Permite medir ángulos e inclinación con gran precisión. A continuación, su funcionamiento y aplicaciones.

**Figura 18.**  
*Sensor Giroscópico LEGO® MINDSTORMS® EV3*



*Nota.* La imagen representa el dispositivo del giroscopio. Fuente:LEGO® (2021a).

### **Especificaciones.**

El sensor giroscópico del EV3 posee un giroscopio MEMS de 3 ejes que detecta orientación espacial y rotación del ladrillo de control con respecto a los ejes X, Y y Z (LEGO®, 2021b). Sus especificaciones técnicas son:

- Tipo: sensor MEMS de 3 ejes.
- Rango de medición:  $\pm 7$  g de aceleración lineal,  $\pm 1200$  grados / s de velocidad de rotación.
- Resolución:  $0.01^\circ$  en rotación,  $0.0025^\circ$  de inclinación.
- Frecuencia de muestreo: 100 Hz.
- Interfaz: I2C/SPI.

El sensor incorpora tecnología de auto-test y calibración, mide ángulos absolutos permitiendo conocer la orientación precisa respecto a un eje de referencia. No requiere calibración manual.

## Programación en PyBricks.

La biblioteca PyBricks provee una clase GyroSensor para leer fácilmente los valores del sensor giroscópico en Python:

```
#!/usr/bin/env pybricks-micropython
from pybricks.ev3devices import GyroSensor
from pybricks.parameters import Port

# Inicializar el EV3
ev3 = EV3Brick()

sensor_giroscopio = GyroSensor(Port.S2)

# Ángulo actual
angulo = sensor_giroscopio.angle()

# Velocidad angular actual
vel_angular = sensor_giroscopio.speed()
```

Del ejemplo anterior, el método *.angle()* devuelve la rotación del EV3 en grados en el momento de encender el sensor. El método *.speed()* indica la velocidad angular actual en grados/segundo, los valores positivos para giro son en sentido horario. El método *.reset\_angle(0)* reinicia el ángulo de referencia a cero (0) grados.

Algunos ejemplos de aplicación del sensor giroscópico son los siguientes:

- Mantener el equilibrio en un robot bípedo o cuadrúpedo midiendo inclinación.

- Girar un robot móvil un ángulo determinado midiendo la rotación.
- Detección de caídas o colisiones monitorizando aceleraciones.
- Control de estabilidad y corrección de rumbo en drones, aviones o barcos teledirigidos.
- Sistemas de estabilización de cámara sobre bases móviles o brazos robóticos.

Por ejemplo, para girar un robot exactamente 90 grados:

```
#!/usr/bin/env pybricks-micropython
from pybricks.robotics import DriveBase
robot = DriveBase(...)

# Girar mientras no se alcance objetivo
while sensor_giroscopio.angle() < 90:

    robot.drive(100, 0) # girar
# Detenerse
robot.stop()
```

De esta manera se puede realizar movimientos y navegación precisa utilizando el sensor giroscópico para medición de ángulos y rotación, sus capacidades permiten implementar comportamientos muy avanzados de estabilidad y control de movimiento en una amplia variedad de proyectos robóticos con LEGO® MINDSTORMS® EV3.

### Sensor Táctil del LEGO® MINDSTORMS® EV3

El sensor táctil es ideal para detectar contacto físico y golpes en el robot LEGO® MINDSTORMS® EV3, funciona como un interruptor simple



que activa o desactiva un circuito eléctrico al ser presionado. Veamos cómo aprovecharlo en los proyectos.

**Figura 19.**  
*Sensor Táctil LEGO® MINDSTORMS® EV3*



*Nota.* La imagen representa el dispositivo de sensor táctil. Fuente:LEGO® (2021a).

### **Especificaciones.**

El sensor táctil del EV3 consta de un resorte conductor y un botón interno que hacen contacto al ser presionado cerrando un circuito eléctrico (LEGO®, 2021b), sus especificaciones son:

- Tipo: interruptor táctil de presión por resorte.
- Entradas: botón presionable.
- Salidas: circuito abierto/cerrado.
- Fuerza de operación: (1-3) Newtons aprox.
- Recorrido de botón: 2 mm - 4 mm.
- Material: plástico ABS resistente a impactos.
- Conexión: cable 2-pin.

El sensor táctil no requiere ninguna configuración o calibración, simplemente se conecta a un puerto de entrada y está listo para usarse.

## Programación en PyBricks.

La biblioteca PyBricks provee una clase `TouchSensor` para leer el estado del sensor táctil. Un ejemplo básico es:

```
#!/usr/bin/env pybricks-micropython

from pybricks.ev3devices import TouchSensor
from pybricks.parameters import Port
from pybricks.hubs import EV3Brick
from pybricks.tools import wait

def main():
    # Inicializar el EV3
    ev3 = EV3Brick()

    # Inicializar el sensor de Brick tacto
    # por el puerto 4
    sensor_tactil = TouchSensor(Port.S4)

    while True:
        if sensor_tactil.pressed():
            # Hacer algo cuando se presiona
            ev3.screen.print("Presionado!")
        else:
            # Acción por defecto
```

```
ev3.screen.print("Soltado")

#-----
if __name__ == "__main__":
    main()
```

El método `.pressed()` devuelve `True` mientras el botón esté presionado, y `False` cuando se suelta.

Algunos ejemplos de uso del sensor táctil:

- Detectar cuando el EV3 choca contra un obstáculo en un robot móvil.
- Sensor *bumper* que active un retroceso o cambio de trayectoria.
- Disparar una acción cuando se presiona, como lanzar una pelota.
- Botones sencillos de control manual del EV3.
- Sistema de seguridad con alarma activada al presionar.

La simplicidad y bajo costo del sensor táctil lo hacen ideal para una amplia gama de aplicaciones en proyectos de robótica Lego.

### ***Sensor infrarrojo***

El sensor infrarrojo es un dispositivo electrónico muy útil, que permite al robot LEGO® MINDSTORMS® EV3 detectar obstáculos y medir distancias de forma precisa mediante infrarrojos. A continuación, su funcionamiento y cómo aprovecharlo en los proyectos.

**Figura 20.**  
*Sensor Infrarrojo LEGO® MINDSTORMS® EV3*



*Nota.* La imagen representa el dispositivo de infrarrojo. Fuente:LEGO® (2021a).

### **Especificaciones Técnicas.**

El sensor infrarrojo del EV3 funciona emitiendo luz infrarroja y midiendo la intensidad de la luz reflejada para determinar la proximidad de objetos cercanos (LEGO®, 2021b), sus especificaciones son:

- Tipo: sensor de proximidad infrarrojo activo.
- Rango de detección: de (0 a 100) cm aproximadamente.
- Resolución: +/- 1 cm.
- Ángulo de haz: < 30 grados.
- Salida: valores analógicos entre 0 % y 100 %.
- Interfaz: I2C/SMBus.
- Alimentación en DC: 5 V a 6 V.
- Dimensiones: (43.2 x 30.5) mm.

## Programación en PyBricks.

La biblioteca PyBricks de Python provee la clase *InfraredSensor* para leer fácilmente el sensor infrarrojo:

```
#!/usr/bin/env pybricks-micropython
from pybricks.ev3devices import InfraredSensor
from pybricks.parameters import Port
from pybricks.hubs import EV3Brick
from pybricks.tools import wait

def main():
    # Inicializar el EV3
    ev3 = EV3Brick()

    # Inicializar el sensor de
    # infrarrojo por el puerto 4
    infrarrojo = InfraredSensor(Port.S4)

    distancia = infrarrojo.distance()
    # Lee distancia en cm
    ev3.screen.print("Distancia:", distancia)

    # Esperar 5 segundos
    wait(5000)

#-----
if __name__ == "__main__":
    main()
```

El método `.distance()` devuelve la distancia al objeto más cercano detectado en un rango de (0 a 100) cm.

Otros métodos útiles son:

- `.presence()`: devuelve bool si detecta objeto a  $< 25$  cm.
- `.beacon(channel)`: lectura de baliza infrarroja en canal especificado.
- `.proximity()`: porcentaje de intensidad de luz reflejada.
- Algunos ejemplos de uso del sensor infrarrojo:
  - Medir distancia a pared para que un robot no choque.
  - Detector de proximidad para parar antes de colisionar.
  - Sensor de posición en una baliza infrarroja.
  - Calibración de color de líneas comparando intensidad.
  - Comunicación de baliza a baliza entre robots.
  - Detección de fuentes de calor mediante infrarrojos.

Por ejemplo, para medir distancia a objetos:

```
while True:
    if infrarrojo.distance() < 15:
        # Obstáculo cerca, detenerse
    else:
        # Avanzar
```

De esta manera un robot móvil puede esquivar obstáculos midiendo en todo momento la distancia con el sensor infrarrojo.

El sensor infrarrojo del EV3 es una importante herramienta para dotar a los robots de percepción del entorno, especialmente en tareas de navegación, detección y medición. Mediante infrarrojos, el sensor amplía enormemente las capacidades del robot para interactuar con su entorno físico; utilizando la biblioteca PyBricks podemos

aprovechar fácilmente toda la potencia de este sensor en nuestras aplicaciones.

### ***Pantalla LCD del LEGO® MINDSTORMS® EV3***

La pantalla LCD incorporada en el ladrillo EV3 permite mostrar información visual muy útil para interacción con el usuario o depuración de programas. Analicemos sus capacidades y cómo controlarla en Python.

#### **Especificaciones.**

La pantalla del EV3 es un display LCD monocromo de 178x128 pixels con retroiluminación LED, orientado en formato horizontal (Jordan, J., & Fern, A. , 2019), sus especificaciones técnicas son:

- Tipo: LCD TFT monocromo.
- Resolución: (178 x 128) pixels.
- Área visible: (36 x 24) mm.
- Retroiluminación: 4 LEDs blancos.
- Interfaz: SPI.
- Contraste: control por *firmware*.
- Diagonal: 47 mm.

La pantalla es controlada por el sistema operativo del EV3, el *firmware* de LEGO® contiene fuentes y controladores optimizados para este *display*.

#### **Programación con PyBricks.**

La biblioteca PyBricks de Python provee la clase *screen* para controlar la pantalla del EV3 de forma sencilla:

```
#!/usr/bin/env pybricks-micropython
from pybricks.hubs import EV3Brick
from pybricks.tools import wait

# Inicializar el EV3
ev3 = EV3Brick()

# Mostrar texto en pantalla
ev3.screen.print("Hola Mundo!")

# Esperar 5 segundos
wait(5000)
```

El método `.print()` permite imprimir una cadena especificada. Otros métodos útiles son:

- `.clear()`: limpia la pantalla.
- `draw_text(x, y, "Texto")`: escribe un texto en las coordenadas (x,y)
- `.draw_pixel(x, y)`: dibuja un pixel en las coordenadas (x,y).
- `.draw_box(x, y, ancho, alto)`: dibuja rectángulo.
- `.draw_circle(x,y,radio,relleno)`: dibuja un círculo.
- `.draw_line(x1,y1,x2,y2)`: dibuja una línea.
- `.load_image(imagen)`: dibuja la imagen en la pantalla, algunas imágenes son:

- `ImageFile.ANGRY`
- `ImageFile.NEUTRAL`
- `ImageFile.QUESTION_MARK`
- `ImageFile.PINCH_RIGHT`
- `ImageFile.WARNING`



- ImageFile. AWAKE
- ImageFile. STOP\_1
- ImageFile.EV3\_ICON
- ImageFile. HURT
- ImageFile. ACCEPT
- ImageFile. DECLINE

La pantalla LCD resulta muy útil para:

- Mostrar estado, mensajes, valores de sensores, etc.
- Visualizar imágenes, gráficos, formas simples.
- Interfaz gráfica básica de usuario.
- Animaciones y efectos visuales.
- Depuración y *troubleshooting* de programas.

Por ejemplo, para un juego que muestra vidas restantes:

```
#!/usr/bin/env pybricks-micropython
from pybricks.hubs import EV3Brick
from pybricks.tools import wait
from pybricks.ev3devices import TouchSensor
from pybricks.parameters import Port
from pybricks.media.ev3dev import Image, ImageFile

# Inicializar el EV3
ev3 = EV3Brick()

# Inicializar el sensor de tacto por el puerto 4
sensor_tactil = TouchSensor(Port.S4)
vidas = 5
ev3_img = Image(ImageFile.DECLINE)
while vidas > 0:
```

```
texto_vidas= "Vidas: " + str(vidas)
ev3.screen.draw_text(30,30,texto_vidas)

if sensor_tactil.pressed():
    vidas -= 1
    ev3.screen.clear()
    ev3.screen.load_image(Image(ev3_img))
    wait(500)
    ev3.screen.clear()
```

### ***Bocina e indicadores sonoros del LEGO® MINDSTORMS® EV3***

El ladrillo programable EV3 incorpora una pequeña bocina y la capacidad de reproducir distintos sonidos y efectos, los cuales resultan muy útiles para dotar al robot de capacidad de comunicación sonora.

#### **Especificaciones.**

La bocina del EV3 es un transductor electroacústico que convierte señales eléctricas en ondas sonoras (LEGO®, 2021b), sus especificaciones son:

- Tipo: bocina dinámica de imán móvil.
- Potencia: 1 W aprox.
- Impedancia: 8 ohms.
- Respuesta de frecuencia: (300-20) kHz.
- Tensión DC: 5 V.
- Nivel sonoro: Hasta 85 dB a 10 cm.

Adicionalmente, el *firmware* del EV3 trae canciones pregrabadas y un sistema de efectos sonoros programables. Los indicadores sonoros son especialmente útiles cuando la pantalla no es visible o como

alertas. Por ejemplo, el EV3 emite distintos tonos para encendido, apagado, conexión bluetooth, entre otros estados.

## Programación con PyBricks.

La biblioteca PyBricks de Python provee la clase *Speaker* para control de sonidos:

```
#!/usr/bin/env pybricks-micropython
from pybricks.hubs import EV3Brick
from pybricks.tools import wait
from pybricks.media.ev3dev import SoundFile
from pybricks.media.ev3dev import Image, ImageFile

# Inicializar el EV3
ev3 = EV3Brick()

# Dice "Sonido uno"
ev3.speaker.say("Sonido uno")
# Reproducir sonido de beep
ev3.speaker.beep()
wait(1000)
# Dice "Sonido dos"
ev3.speaker.say("Sonido dos")
# Reproducir sonido de beep con
# frecuencia y duración
ev3.speaker.beep(frequency=500, duration=500)

# Dice "Sonido tres"
```

```
ev3.speaker.say("Sonido tres")  
# Reproduce el sonido de un archivo  
ev3.speaker.play_file(SoundFile.SONAR)  
  
ev3.speaker.say("Sonido cuatro")  
# Reproduce el sonido de un archivo  
ev3.speaker.play_file(SoundFile.DOG_BARK_1)
```

Algunos métodos útiles son:

- `.play_file(Archivo_de_sonido)` Reproduce el archivo de sonido, algunos sonidos son:
  - `SoundFile.AIRBRAKE`
  - `SoundFile.AIR_RELEASE`
  - `SoundFile.BACKUP_ALERT`
  - `SoundFile.BOO`
  - `SoundFile.BRAKE`
  - `SoundFile.CHIME`
  - `SoundFile.CLICK_2`
  - `SoundFile.DOOR_SLAM`
  - `SoundFile.DOWN`
  - `SoundFile.ENGINE_FAST`
  - `SoundFile.ERROR`
  - `SoundFile.EV3`
- `.play_note()` Reproduce una nota musical.
- `.say()` Convierte el texto a voz.
- `.set_speech_options()` Cambia voz y velocidad.

Usos comunes:

- Reproducir efectos sonoros y alertas.
- Hablar mensajes y texto como interfaz.

- Añadir música y sonido en proyectos y juegos.
- Servir como altavoz para reproducir cualquier sonido.
- Debugging al reproducir códigos morse.

### ***Leds del Lego Mindstorms EV3***

Los leds o diodos emisores de luz son componentes electrónicos que permite emitir luz de diferentes colores de forma simple en el robot LEGO® MINDSTORMS® EV3. Se analiza a contrinuación las capacidades de estos indicadores.

#### **Especificaciones.**

El ladrillo EV3 incluye 3 leds que funcionan como luces indicadoras del estado del robot (LEGO®, 2021b), sus especificaciones son:

- Tipo: diodos emisores de luz (LED).
- Colores: rojo, verde, ámbar y azul.
- Corriente: 20mA típica, 30mA máxima.
- Tensión de operación: 4.5V DC aprox.
- Intensidad luminosa: 280-400 mcd típica.
- Ángulo de emisión: 60 grados.
- Tamaño: 5mm de diámetro.

No requieren configuración, están incorporados dentro del ladrillo principal del EV3

#### **Programación en PyBricks.**

La biblioteca PyBricks de Python provee la clase light para controlar los leds del EV3:

```
#!/usr/bin/env pybricks-micropython
from pybricks.hubs import EV3Brick
from pybricks.parameters import Color
from pybricks.tools import wait

# Inicializar el EV3
ev3 = EV3Brick()

# Encender una luz naranja
ev3.light.on(Color.ORANGE)
# Esperar un 5 segundos
wait(500)
# Apagar la luz
ev3.light.off()
```

El método *light.on(color)* permite encender la luz LED del Brick, donde el parámetro color puede ser RED, ORANGE y RED.

EL método *light.off()* apaga el led.

Usos comunes:

- Luces indicadoras de estado (batería, conexión, etc).
- Indicadores luminosos en proyectos y juegos.
- Árbol de navidad parpadeando leds.
- Señales visuales de operación o alertas.
- Simular semáforo o señalización de tráfico.

Por ejemplo, un patrón de leds en un *while*:

```
#!/usr/bin/env pybricks-micropython
from pybricks.hubs import EV3Brick
```

```
from pybricks.parameters import Color
from pybricks.tools import wait

# Inicializar el EV3
ev3 = EV3Brick()
while True:
    # Encender una luz naranja
    ev3.light.on(Color.ORANGE)
    # Esperar un 5 segundos
    wait(500)
    # Apagar la luz
    ev3.light.off()

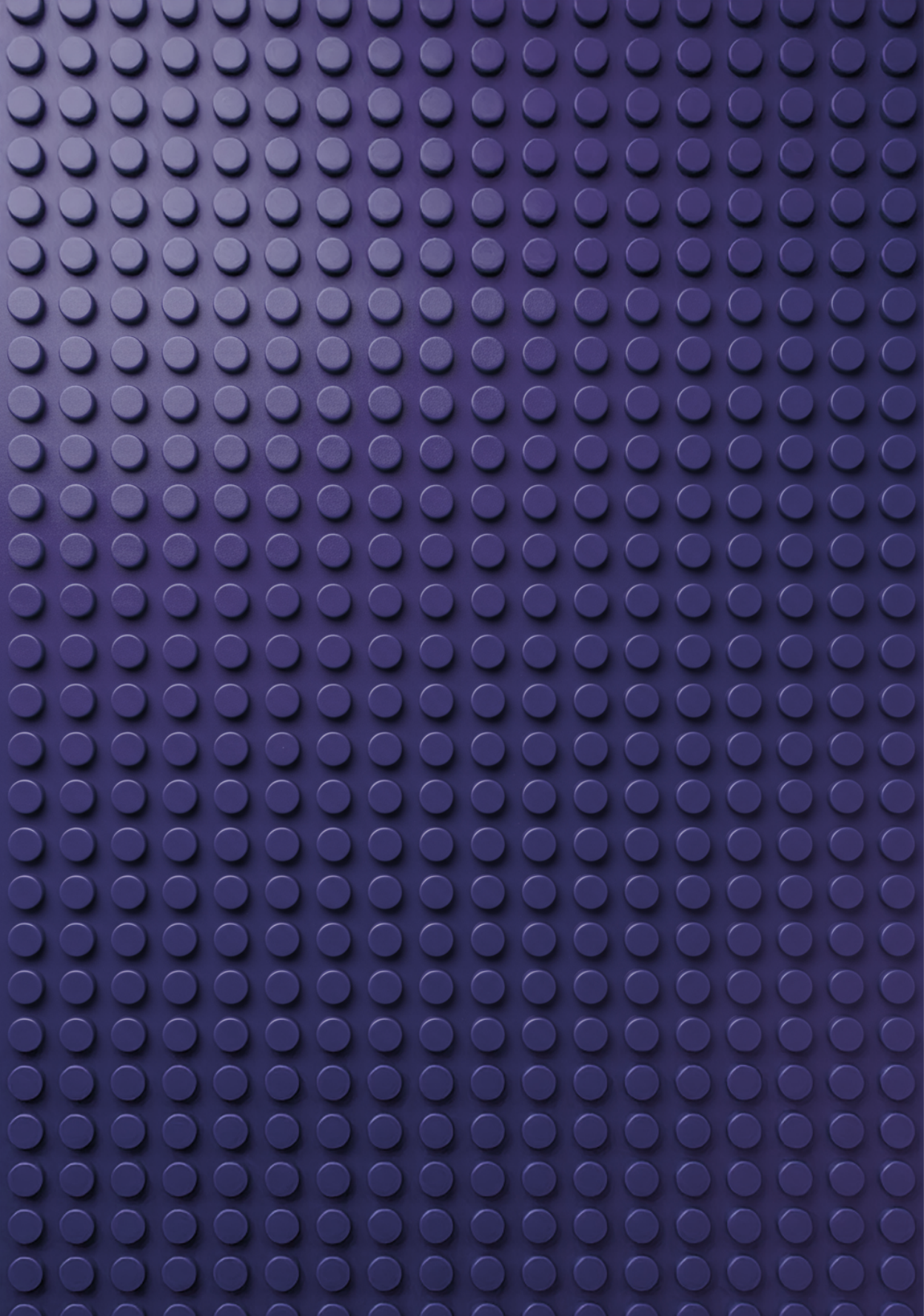
    # Encender una luz Roja
    ev3.light.on(Color.RED)
    # Esperar un 5 segundos
    wait(500)
    # Apagar la luz
    ev3.light.off()

    # Encender una luz Verde
    ev3.light.on(Color.GREEN)
    # Esperar un 5 segundos
    wait(500)
    # Apagar la luz
    ev3.light.off()
```

Los leds pueden añadir efectos visuales interesantes de forma sencilla en todo tipo de proyectos con el EV3. Su versatilidad y bajo costo los hacen ideales para indicadores simples.







## **Capítulo 5**

# **Proyectos Prácticos**

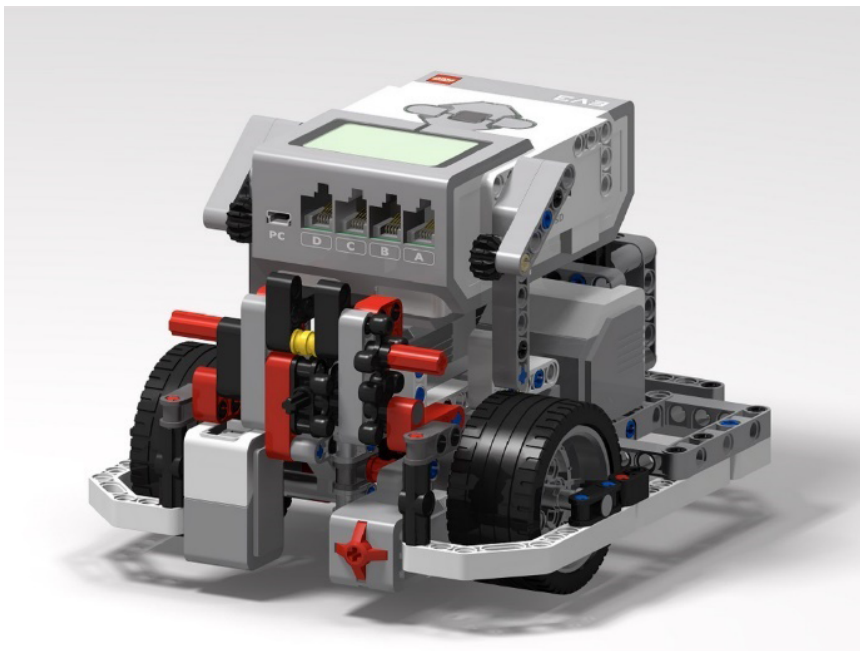


A continuación, se presenta una colección de proyectos prácticos para construir diferentes robots funcionales utilizando el ladrillo programable Lego Mindstorms EV3 y el lenguaje de programación Python con la biblioteca PyBricks.

Los proyectos están pensados para ser abordados de forma incremental, aplicando los conceptos explorados en los capítulos previos para resolver desafíos de robótica y programación cada vez más complejos, comenzando con robots simples que se mueven en patrones básicos y avanzando gradualmente hacia la incorporación de diferentes sensores, lo que permite que los robots perciban y reaccionen a su entorno; la programación culmina con un robot móvil autónomo capaz de navegar su entorno sin supervisión.

Cada proyecto se explica en detalle, analizando la lógica implementada y el código necesario, de modo que al finalizar este capítulo el lector estará capacitado para crear y programar sofisticados robots utilizando el LEGO® MINDSTORMS® EV3.

**Figura 21.**  
*Robot sumo LEGO® MINDSTORMS® EV3*



*Nota.* La imagen representa el robot sumo. Fuente: Seshan & EV3Lessons (2020).

### Programa para recorrido de 10 cuadros

El *programa para recorrido de 10 cuadros* en PyBricks hace que un robot LEGO® MINDSTORMS® EV3 se mueva describiendo un recorrido cuadrado. Ambos motores avanzan durante 1 segundo, detienen el motor izquierdo y giran el derecho durante 1 segundo para lograr un giro de  $\pm 90$  grados; Repiten esta secuencia 10 veces mediante un bucle for, logrando así un movimiento en el que el robot recorre un total de 10 cuadrados gracias a los giros alternados.

```
#!/usr/bin/env pybricks-micropython
```

```
from pybricks.hubs import EV3Brick
from pybricks.ev3devices import Motor
from pybricks.parameters import Port
from pybricks.tools import wait

def main():
    # Inicializa el brick
    ev3 = EV3Brick()

    # Inicializa los motores
    left_motor = Motor(Port.A)
    right_motor = Motor(Port.C)

    for i in range(10):
        for j in range(4):
            # Mueve el motor los motores A y C
            # a una velocidad rotacional 500
            # grados por segundo
            left_motor.run(500)
            right_motor.run(500)
            # Espera 1000 milisegundos
            wait(1000)
            # Detiene los motores A
            left_motor.hold()
            # Mueve el motor C a una velocidad
            # rotacional
            # de -500 grados por segundo
            right_motor.run(-500)
            # Espera 1000 milisegundos
```

```
        wait(1000)

        # Detiene los motores A y C
        left_motor.stop()
        right_motor.stop()

        #-----

if __name__ == "__main__":
    main()
```

### Programa para Recorrido en Espiral

El Programa para Recorrido en Espiral en PyBricks hace que un robot LEGO® MINDSTORMS® EV3 se mueva describiendo una trayectoria en espiral, mediante un bucle for que se repetirá 50 veces. En cada iteración se mueven ambos motores durante un tiempo determinado por la variable *time*, luego se detiene el motor izquierdo y se invierte el derecho durante 1000 milisegundos logrando un giro, al final del bucle se incrementa *time* en 200 ms, de esta forma, en cada iteración el tiempo de avance es mayor, dando como resultado un movimiento en espiral cuyo radio se va agrandando progresivamente.

```
#!/usr/bin/env pybricks-micropython

from pybricks.hubs import EV3Brick
from pybricks.ev3devices import Motor
from pybricks.parameters import Port
from pybricks.tools import wait

def main():
    # Inicializa el brick
```

```
ev3 = EV3Brick()

# Inicializa los motores
left_motor = Motor(Port.A)
right_motor = Motor(Port.C)

# Inicializa el tiempo de espera
time = 200
for i in range(50):
    # Mueve el motor los motores A y C
    left_motor.run(500)
    right_motor.run(500)
    wait(time)

    left_motor.hold()
    right_motor.run(-500)
    wait(1000)

    # incrementa el tiempo de espera
    time += 200

# Detiene los motores A y C
left_motor.stop()
right_motor.stop()

#-----

if __name__ == "__main__":
    main()
```



## Programa de recorrido aleatorio

El *programa de recorrido aleatorio* en PyBricks hace que un robot LEGO® MINDSTORMS® EV3 se mueva de forma aleatoria durante 2 segundos mediante un bucle infinito. En cada iteración, primero hace avanzar a ambos motores juntos durante dos segundos, luego, genera un número aleatorio entre 0 y 9, si es mayor o igual a 5 detiene el motor A y retrocede sólo con el motor C, de lo contrario, detiene el motor C y retrocede el motor A; esto provoca giros y cambios de dirección aleatorios.

Finalmente, vuelve a detener ambos motores después de un segundo antes de volver a repetir. Este proceso en bucle genera un movimiento irregular e impredecible del robot, avanzando, girando y retrocediendo al azar, gracias al uso de la función *randint()* para introducir aleatoriedad en el programa.

```
#!/usr/bin/env pybricks-micropython
from pybricks.hubs import EV3Brick
from pybricks.ev3devices import Motor
from pybricks.parameters import Port
from pybricks.tools import wait, StopWatch
import random

# Generar un número entero aleatorio entre 0 y 9
def aleatorio (n,m):
    numero_aleatorio = random.randint(n, m)
    return numero_aleatorio

#-----
```

```
def main():  
    # Inicializa el brick  
    ev3 = EV3Brick()  
  
    # Inicializa los motores  
    left_motor = Motor(Port.A)  
    right_motor = Motor(Port.C)  
  
    # Inicializa el tiempo de espera  
    while True:  
        # Para los motores A y C  
        left_motor.stop()  
        right_motor.stop()  
        # Mueve el motor los motores A y C  
        left_motor.run(500)  
        right_motor.run(500)  
        # Espera 2000 milisegundos  
        wait(2000)  
        # Genera un número aleatorio entre 0 y 9  
        n = aleatorio(0, 9)  
        if (n>=5):  
            # Para el motor A  
            left_motor.hold()  
            # Mueve el motor C a una velocidad  
            # rotacional  
            right_motor.run(-500)  
        else:  
            # Para el motor C
```

```
        right_motor.hold()

        # Mueve el motor A con una
        # velocidad rotacional
        left_motor.run(-500)
        # Espera 1000 milisegundos
        wait(1000)

        # Detiene los motores A y C
        left_motor.stop()
        right_motor.stop()

#-----

if __name__ == "__main__":
    main()
```

### Programa con sensor de tacto

El *programa con sensor de tacto* en PyBricks utiliza el sensor de tacto del robot LEGO® MINDSTORMS® EV3 para detectar colisiones. Mediante un bucle infinito, se verifica continuamente si el sensor está presionado, en caso positivo, invierte el giro de ambos motores durante 2500 milisegundos, para retroceder, detiene el motor izquierdo y gira el derecho durante 3000 milisegundos. De esta forma, cuando el robot colisiona con un obstáculo, reacciona retrocediendo y girando para sortearlo; en caso de que el sensor no detecte presión, simplemente ambos motores siguen avanzando normalmente.

```
#!/usr/bin/env pybricks-micropython

from pybricks.ev3devices import TouchSensor
```

```
from pybricks.ev3devices import Motor
from pybricks.parameters import Port
from pybricks.hubs import EV3Brick
from pybricks.tools import wait

def main()
    # Inicializar el EV3
    ev3 = EV3Brick()

    # Inicializar el sensor de tacto
    # por el puerto 4
    sensor_tactil = TouchSensor(Port.S4)
    # Inicializar los motores en el puerto A y C
    motor_A = Motor(Port.A)
    motor_C = Motor(Port.C)
    while True:
        if sensor_tactil.pressed():
            # Retroceder y girar al chocar
            motor_A.run(-250)
            motor_C.run(-250)
            wait(2500)
            motor_C.stop()
            motor_A.run(-100)
            wait(3000)
        else:
            # Avanzar normalmente
            motor_A.run(250)
            motor_C.run(250)
```

```
#-----  
  
if __name__ == "__main__":  
    main()
```

Así el EV3 puede reaccionar al contacto físico, el potencial de aplicaciones es muy variado dada la versatilidad del sensor táctil para detectar interacción con objetos.

### Programa con sensor ultrasónico

El *programa con sensor ultrasónico* en PyBricks utiliza el sensor de ultrasónico en el robot LEGO® MINDSTORMS® EV3 para detectar objetos que están al frente del robot. Mediante un bucle infinito, se verifica continuamente la distancia del sensor con el objeto, en caso de que la distancia sea menor 150 milímetros, invierte el giro de ambos motores durante 2500 milisegundos, para retroceder, detiene el motor izquierdo y gira el derecho durante 3000 milisegundos, De esta forma, cuando el robot encuentra un obstáculo por debajo de dicho rango, reacciona retrocediendo y girando para sortearlo; en caso de que el sensor no detecte presencia cercana, simplemente ambos motores siguen avanzando normalmente.

```
#!/usr/bin/env pybricks-micropython  
  
from pybricks.ev3devices import UltrasonicSensor  
from pybricks.ev3devices import Motor  
from pybricks.parameters import Port  
from pybricks.hubs import EV3Brick  
from pybricks.tools import wait
```

```
def main():  
    # Inicializar el EV3  
    ev3 = EV3Brick()  
  
    # Inicializar el sensor de  
    # Ultrasonico por el puerto 2  
    sensor_ultrasonido = UltrasonicSensor(Port.S2)  
    # Inicializar los motores en el puerto A y C  
    motor_A = Motor(Port.A)  
    motor_C = Motor(Port.C)  
    while True:  
        d= sensor_ultrasonido.distance()  
        if (d<150):  
            # Retroceder y girar al chocar  
            motor_A.run(-250)  
            motor_C.run(-250)  
            wait(2500)  
            motor_C.stop()  
            motor_A.run(-100)  
            wait(3000)  
        else:  
            # Avanzar normalmente  
            motor_A.run(250)  
            motor_C.run(250)  
  
#-----  
  
if __name__ == "__main__":  
    main()
```

## Programa con el sensor infrarrojo

El *programa con el sensor infrarrojo* en PyBricks implementa un robot móvil LEGO® MINDSTORMS® EV3 capaz de detectar y esquivar obstáculos utilizando un sensor infrarrojo. Mediante un bucle infinito, lee continuamente la distancia al objeto más cercano, si es menor a 15 centímetros, determina que hay un obstáculo próximo por lo que frena el robot y retrocede para sortearlo, en caso contrario, si el camino está libre, simplemente avanza en línea recta. De esta forma, el robot es capaz de navegar un entorno de manera autónoma, detectando obstáculos y evitándolos, gracias al sensor infrarrojo y el control de motores mediante PyBricks.

```
#!/usr/bin/env pybricks-micropython

from pybricks.ev3devices import InfraredSensor
from pybricks.ev3devices import Motor
from pybricks.parameters import Port
from pybricks.hubs import EV3Brick
from pybricks.tools import wait

def main():
    # Inicializar el EV3
    ev3 = EV3Brick()

    # Inicializar el sensor infrarrojo
    # por el puerto 4
    infrarrojo = InfraredSensor(Port.S4)
    # Inicializar los motores en el puerto A y C
    motor_A = Motor(Port.A)
```

```
motor_C = Motor(Port.C)
while True:
    d= infrarrojo.distance()
    if (d<15):
        # Retroceder y girar al chocar
        motor_A.run(-250)
        motor_C.run(-250)
        wait(2500)
        motor_C.stop()
        motor_A.run(-100)
        wait(3000)
    else:
        # Avanzar normalmente
        motor_A.run(250)
        motor_C.run(250)

#-----

if __name__ == "__main__":
    main()
```

## Programa seguidor de línea

El *programa seguidor en línea* en PyBricks implementa un seguidor de línea usando un sensor de color en un robot LEGO® MINDSTORMS® EV3. Mediante un bucle infinito se lee continuamente el valor de reflexión del sensor, si es muy bajo, significa que detecta una línea negra, por lo que el robot gira hacia la izquierda, si el valor es muy alto, indica que se salió de la línea y debe girar a la derecha para



retomarla; en caso que el valor esté en un rango medio avanza en línea recta, hasta encontrar los valores de reflexión requeridos para tomar de nuevo la línea negra, de esta forma, es capaz de seguir el recorrido de una línea negra en el piso guiando el movimiento del robot.

```
#!/usr/bin/env pybricks-micropython
from pybricks.hubs import EV3Brick
from pybricks.ev3devices import Motor, ColorSensor
from pybricks.parameters import Port

def main()
    ev3 = EV3Brick()
    left_motor = Motor(Port.A)
    right_motor = Motor(Port.C)
    color_sensor = ColorSensor(Port.S1)

    while True:
        reflection = color_sensor.reflection()

        if reflection < 30: #Izquierda
            left_motor.run(-100)
            right_motor.run(100)

        elif reflection > 60: #Derecha
            left_motor.run(100)
            right_motor.run(-100)

        else:
```

```
    left_motor.run(500)
    right_motor.run(500)

#-----

if __name__ == "__main__":
    main()
```

El seguidor de línea también se puede implementar leyendo el color del sensor, si es negro significa que detecta una línea negra, por lo que el robot gira hacia la izquierda, si el color es blanco, indica que se salió de la línea y debe girar a la derecha para retomarla; en caso de que el valor no sea negro o blanco, avanza en línea recta hasta encontrar de nuevo el color negro o blanco.

```
#!/usr/bin/env pybricks-micropython
from pybricks.hubs import EV3Brick
from pybricks.ev3devices import Motor, ColorSensor
from pybricks.parameters import Port, Color
def main():
    ev3 = EV3Brick()
    left_motor = Motor(Port.A)
    right_motor = Motor(Port.C)
    color_sensor = ColorSensor(Port.S1)

    while True:
        color = color_sensor.color()

        if color == Color.BLACK: # Izquierda
```

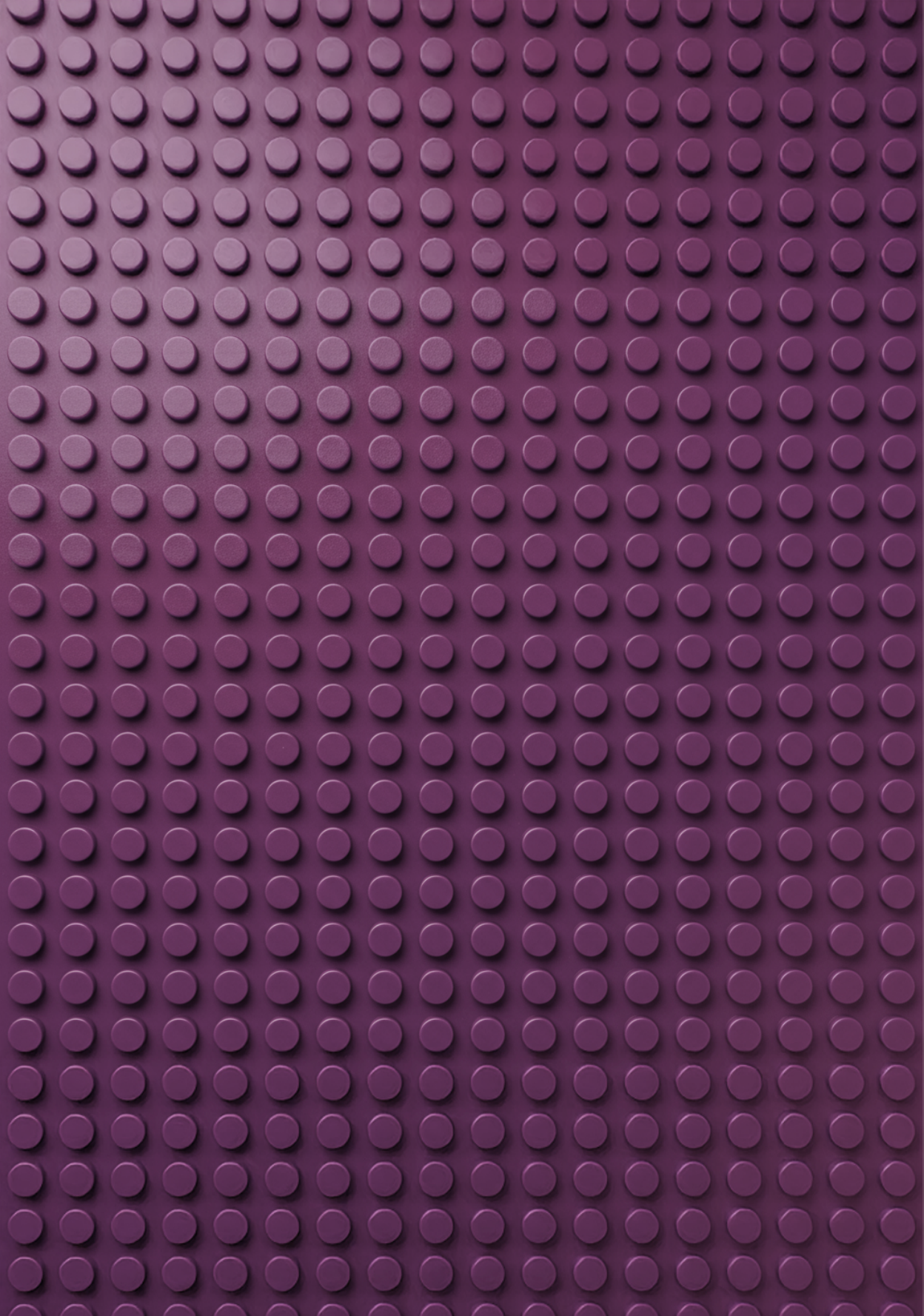
```
        left_motor.run(-100)
        right_motor.run(100)

    elif color == Color.WHITE: # Derecha
        left_motor.run(100)
        right_motor.run(-100)

    else:
        left_motor.run(500)
        right_motor.run(500)

#-----
if __name__ == "__main__":
    main()
```





## Capítulo 6

# Testing y Depuración de Programas con Pybricks



El *testing* o prueba de programas es una parte fundamental del desarrollo de *software*, permite verificar que el código funciona correctamente antes de desplegarlo a un entorno real. En robótica esto cobra especial importancia, ya que errores en el *software* pueden hacer que un robot físico presente comportamientos erráticos, inseguros o dañinos.

En este capítulo verán algunas técnicas y buenas prácticas para probar programas de robots LEGO® MINDSTORMS® EV3 escritos en Python utilizando la biblioteca PyBricks, también se discutirán los errores más comunes y cómo solucionarlos mediante un proceso llamado depuración o *debugging*.

## Probando programas

A continuación, se presentan diversas estrategias para probar el código del robot EV3 con PyBricks (Rollins, 2014):

- Probar segmentos pequeños de código de forma aislada: tratar cada nueva función o característica por separado antes de integrarlas al programa completo. Esto ayuda a identificar errores más fácilmente.
- Probar con diferentes conjuntos de entradas: hay que verificar el código con valores de entrada válidos y no válidos. Por



ejemplo, números positivos y negativos, dentro y fuera de rangos límite.

- Probar múltiples flujos de ejecución: probar cada camino posible en condicionales e iteraciones. Por ejemplo, las cláusulas *if-else* deben validarse tanto para *True* como *False*.
- Simular componentes faltantes: si no tenemos un sensor físico, podemos simularlo pasando valores predefinidos para probar esa parte del código.
- Pruebas unitarias: son pequeños fragmentos de código para probar unidades individuales como funciones, de forma aislada y controlada. Ayudan a probar partes específicas mejor.
- Pruebas de integración: una vez que las unidades individuales funcionan, debemos probar como trabajan en conjunto utilizando pruebas de integración.
- Pruebas de caja negra vs caja transparente: las pruebas de caja negra verifican inputs y outputs sin ver los detalles internos. Las pruebas de caja transparente verifican además el flujo interno paso a paso.
- Pruebas de humo: provocar intencionalmente fallos, como apagar el EV3 durante la ejecución, para verificar la robustez y manejo de errores.
- Pruebas de regresión: volver a probar características existentes tras agregar nuevas, para detectar efectos colaterales o bugs introducidos.

Buenas prácticas para pruebas efectivas:

- Cubrir todos los casos y flujos posibles, no sólo los normales.
- Tener un buen conjunto de datos de prueba representativos.
- Automatizar pruebas repetitivas cuando sea posible.

- Documentar los casos y resultados de prueba.
- Probar frecuentemente a lo largo del ciclo de desarrollo, no sólo al final.
- Que otra persona pruebe el código para encontrar errores nuevos.

## Errores comunes y soluciones

- Algunos errores y excepciones comunes en programas de EV3 con PyBricks son (Sweigart, 2020):
- Errores de sintaxis: errores de tipado, mala indentación o puntuación errónea. Causan que el programa no se ejecute. Revisar cuidadosamente el código y los mensajes de error para detectarlos.
- Nombres indefinidos: uso de variables, funciones o módulos que no han sido definidos. Puede ayudar nombrar de forma explicativa.
- Tipos de datos incorrectos: por ejemplo, tratar de sumar un *string* y un número. Hay que revisar que los tipos coincidan en operaciones.
- Accesos inválidos: tratar de acceder un elemento fuera del rango de un arreglo o una lista. Por ejemplo, revisar índices y rangos.
- Iteraciones infinitas: tener cuidado con *loops* infinitos que pueden congelar el programa. Probar límites y condiciones de salida.
- Errores lógicos: el programa funciona, pero no hace lo que debería. Revisar la lógica de decisiones y secuencias. Depurar con *prints*.

- Errores de integración: piezas individuales funcionan, pero fallan al combinarlas. Probar exhaustivamente la integración de componentes.
- Excepciones: errores que interrumpen el flujo como al dividir por cero. Manejar excepciones con *try-except* donde sea probable que ocurran.
- Problemas de concurrencia: cuando dos procesos o *threads* acceden a un recurso compartido simultáneamente. Utilizar exclusiones mutuas.
- Errores aleatorios o intermitentes: son los más difíciles de diagnosticar. Intentar reproducirlos y aislar casos para entender las causas.

Para identificar y resolver estos y otros errores, podemos aplicar técnicas de depuración:

- Leer detenidamente los mensajes de error y excepciones para entender dónde se origina el problema.
- Usar *prints* para mostrar valores de variables en diferentes puntos y ver dónde se desvía el flujo.
- Revisar en detalle los registros, *traces* y *dumps* que provee el sistema operativo del EV3.
- Utilizar un depurador para ejecutar paso a paso viendo el estado del programa. Visual studio code integra uno.
- Aislar partes del código y datos para replicar el error en un caso mínimo.
- Validar supuestos y teorías en donde puede estar el error sistemáticamente.
- Repasar código recientemente agregado o modificado, donde puede originarse el *bug* nuevamente introducido.

- Consultar con otros programadores para un análisis de código crítico.
- Como último recurso, reescribir partes del código desde cero o reflexionar sobre el diseño.

Un proceso metódico y constante de pruebas ayuda a detectar y corregir errores tempranamente. La depuración efectiva requiere de una mente crítica y creatividad para resolver los inevitables *bugs* de la programación. Estas habilidades fundamentales se desarrollan con la experiencia y son esenciales para todo programador profesional.

## Conclusiones

Python es un excelente lenguaje para iniciarse en robótica, dada su curva de aprendizaje suave y la gran cantidad de bibliotecas especializadas disponibles. Permite desarrollar comportamientos sofisticados en robots de manera simple e interactiva, resultando ideal para propósitos educativos. La posibilidad de controlar fácilmente un *hardware* como el EV3 abre un mundo de posibilidades creativas.

El ladrillo programable LEGO® MINDSTORMS® EV3 proporciona una plataforma de *hardware* versátil y de alta calidad para montar y dar vida a distintos modelos de robots. Su sistema modular con motores, sensores y componentes interconectables mediante programación, permite construir desde robots simples hasta muy complejos, según el nivel de habilidad.

La biblioteca PyBricks de Python brinda una API limpia e intuitiva para controlar el EV3 de forma remota desde el PC, sin necesidad de modificar el sistema operativo del robot. Su sintaxis sencilla y enfocada a principiantes la hace ideal para introducir la programación de robots con Python de manera rápida y práctica.

Los proyectos presentados proveen ejemplos incrementalmente más complejos que permiten aplicar los conceptos explorados para dotar a un robot EV3 de comportamientos autónomos, como seguir líneas, evitar obstáculos y navegar entornos basándose en lecturas de sensores mediante el código Python desarrollado.

Las buenas prácticas de *testing* y depuraciones cubiertas son esenciales para verificar el correcto funcionamiento de los programas y corregir errores. Probar exhaustivamente diferentes casos y manejar excepciones de forma adecuada, es indispensable en el desarrollo de un *software* robótico confiable.

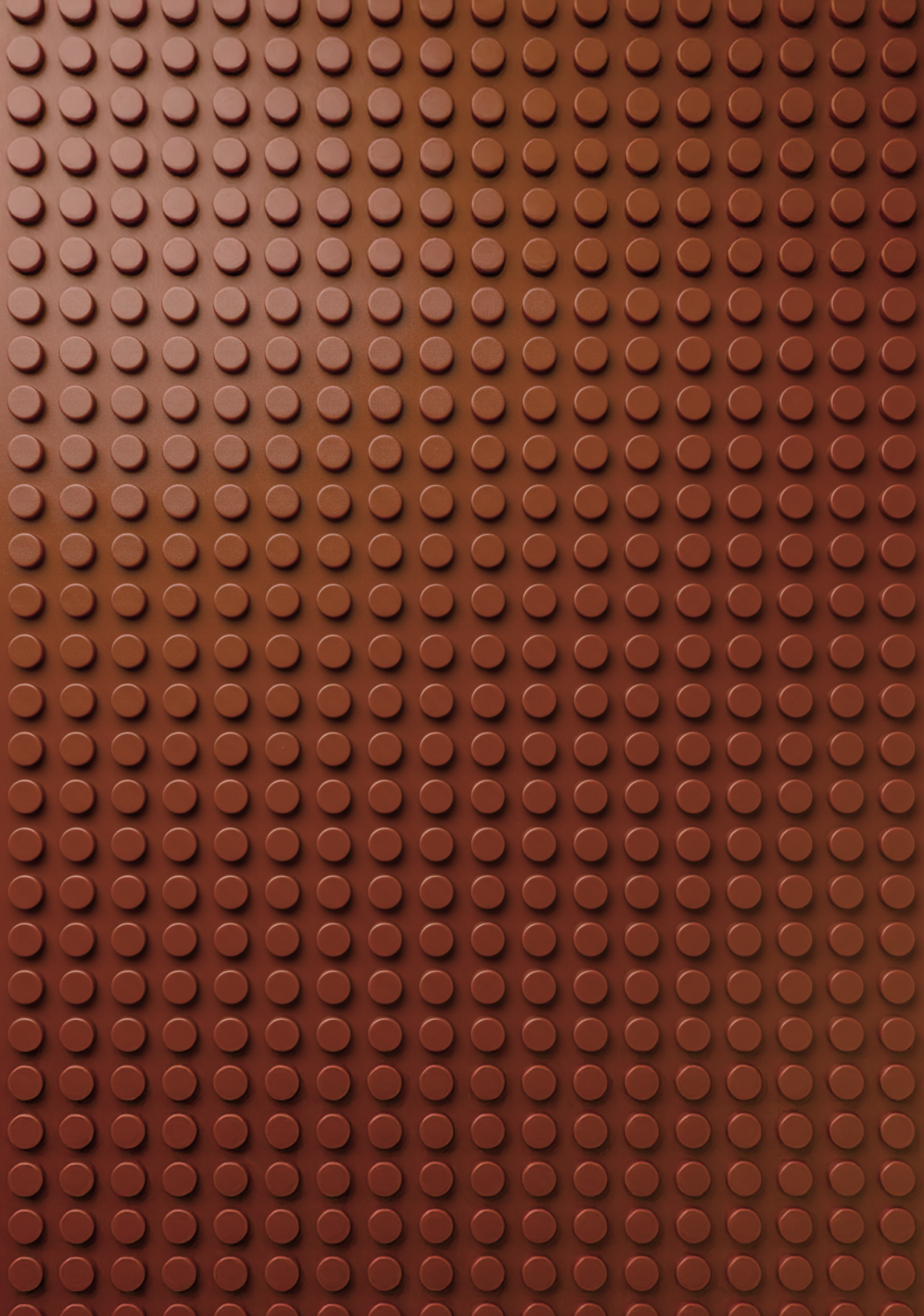
## Glosario

- **API:** interfaz de programación de aplicaciones que expone funciones de un sistema.
- **ARM:** arquitectura de CPU utilizada en el EV3 y otros sistemas embebidos.
- **Assert:** función para verificar resultados esperados en pruebas.
- **Biblioteca:** colección de funciones y clases reutilizables.
- **Brick:** otro nombre para el ladrillo EV3.
- **Debugging:** proceso de encontrar y arreglar defectos en programas.
- **Dumps:** tipos de datos del sistema operativo que contienen una copia de la memoria del kernel del sistema operativo del robot.
- **Embedded:** sistemas computarizados dedicados a una tarea específica integrados en un dispositivo.
- **EV3:** ladrillo programable que hace de computadora del robot Lego Mindstorms EV3.
- **Excepción:** error detectado durante ejecución que altera flujo normal.
- **Firmware:** *software* que controla los componentes electrónicos de un dispositivo.
- **Frameworks:** conjunto de herramientas y librerías para desarrollar aplicaciones.
- **GPIO:** Pines de propósito general para conectar sensores y actuadores.
- **Interpretado:** lenguajes que se ejecutan línea por línea sin compilación previa.
- **LCD:** pantalla de cristal líquido para mostrar información visual.

- **Motor:** actuador que provee movimiento y trabajo mecánico al robot.
- **Multiparadigma:** lenguaje que soporta varios estilos de programación.
- **Packages:** módulos que agrupan bibliotecas relacionadas.
- **Pip:** herramienta para instalar y administrar paquetes Python.
- **PWM:** modulación por ancho de pulso para controlar potencia en motores.
- **PyBricks:** biblioteca de Python para controlar el EV3 de forma remota.
- **PyPI:** repositorio de paquetes Python para instalar bibliotecas.
- **RAM:** memoria de acceso aleatorio volátil para almacenar datos en uso.
- **Robot:** máquina programable capaz de realizar tareas físicas de forma autónoma.
- **Sensor:** dispositivo que permite al robot detectar el entorno.
- **SPI:** bus de comunicación para transferir datos entre componentes.
- **Struct:** estructuras de datos personalizadas compuestas por varios tipos.
- **Testing:** realizar pruebas sistemáticas para verificar comportamiento de código.
- **Thread:** segmento de ejecución concurrente e independiente.
- **Tipado dinámico:** tipo de variable se define en tiempo de ejecución, no explícitamente.
- **Traces:** tipo de datos del sistema operativo que registra eventos que ocurren en el kernel del sistema operativo del robot.
- **Try/except:** manejar excepciones ejecutando código alternativo.







## **Capítulo 7**

# **Una Referencia Pedagógica para la Enseñanza de la Robótica**



En un mundo donde la tecnología avanza a ritmos acelerados, las técnicas del aprendizaje activo se llegan a destacar como un enfoque pedagógico diferente para la formación del estudiante moderno, en el dinámico campo de las ciencias aplicadas.

La robótica, disciplina que ha pasado de ser un sueño futurista a una realidad cotidiana, se encuentra en el corazón de la revolución tecnológica actual, desde los robots industriales que ensamblan productos en fábricas, hasta los dispositivos de asistencia que mejoran la calidad de vida de las personas, es así como la robótica impregna todos los aspectos de la sociedad moderna. Esto se ve reflejado en el ámbito educativo, desde los primeros años de escolaridad hasta los ambientes de formación avanzada; las instituciones dedicadas a esta actividad y cuyo interés es desarrollar en el estudiante habilidades de diseño, programación, construcción mecánica, ensamble de robots y en general, competencias científico-técnicas, se ven en la necesidad de abrir espacios a los que normalmente se nombran “Robótica Educativa”. de acuerdo con J. M. García (2015), Chavarría y Saldaño (2010), Karin et al (2015), y Botelho et al (2012), citados en Ruiz Vicente (2017),:

Un proceso pedagógico creativo basado en el aprendizaje por ensayo-error (J. M. García, 2015), y un proceso tecnológico, basado en

las interacciones entre ciencia, sociedad y tecnología (Chavarría & Saldaño, 2010), que se concretan en la construcción, programación y manipulación de una plataforma robótica (Karim et al., 2015). Esta última parte de la definición es la que se corresponde con la robótica en sí misma, sea o no educativa, es decir, la ciencia que estudia la creación y programación de máquinas que perciben e interactúan con el entorno (Botelho et al., 2012). (p. 58)

Sin embargo, en numerosas situaciones, la robótica educativa simplifica el proceso al reducir la etapa de construcción, dejando solo las fases de programación y manipulación, donde los estudiantes interactúan directamente con el robot. En tal caso, aparecen una serie de empresas que desarrollan kits listos para ser usados; tal es el caso de la empresa LEGO®, que ha desarrollado el robot educativo MINDSTORMS® EV3.

En este contexto, el aprendizaje activo se muestra como una poderosa herramienta para capacitar a los estudiantes y prepararlos para los desafíos del siglo XXI. Al alejarse del modelo tradicional de enseñanza centrado en el profesor, el aprendizaje activo coloca al estudiante en el centro del proceso educativo, fomentando la exploración, la experimentación y el descubrimiento independiente.

Al interior de este capítulo, se explorarán algunas facetas del aprendizaje activo aplicado en el campo de la robótica, se apoyará en el aprendizaje basado en proyectos (ABP) como estrategia de enseñanza aprendizaje y se contextualizará usando el ROBOT LEGO® MINDSTORMS EV3 como instrumento para el desarrollo de las actividades. De esta forma, el propósito de este libro no solo se centra en la producción de un texto guía en la programación del robot LEGO® usando el lenguaje de programación *Python*, sino que

también pretende ofrecerle al docente una referencia para que aplique el ABP dentro de un curso de programación de robots.

### **Algunas técnicas de aprendizaje activo**

En el entorno educativo, el enfoque hacia el aprendizaje activo ha ganado terreno como un método eficaz para involucrar a los alumnos en su proceso de formación, en contraste con las técnicas tradicionales centradas en la transmisión de información, el aprendizaje activo se centra en la participación del estudiante para que sea parte en la construcción de su propio conocimiento. Es importante considerar que el aprendizaje activo no es un enfoque nuevo, esto se señala en “¿PEDAGOGÍA ACTIVA O MÉTODOS ACTIVOS? EL CASO DEL APRENDIZAJE ACTIVO EN LA UNIVERSIDAD”, en donde Espejo Leupin (2016) indica que la evolución del aprendizaje activo se puede agrupar en cuatro periodos, comenzando en 1889 con lo que se llamó “Escuelas nuevas”.

En el proceso evolutivo del aprendizaje activo, se han desarrollado diversas técnicas de aprendizaje que buscan promover la interacción, la colaboración y el compromiso de los alumnos en el aula, en tal sentido, se abordará de manera general el Aprendizaje Basado en Problemas (ABP), el Aprendizaje Cooperativo y el Aula Invertida o *Flipped Classroom*, para, posteriormente, hacer una presentación más detallada del Aprendizaje Basado en Proyectos, el cual será usado dentro de este texto para presentar una propuesta pedagógica de un curso de programación en *Python*, usando el ROBOT LEGO MINDSTORMS EV3.

Es importante destacar que cada una de estas técnicas aporta una perspectiva única para promover un aprendizaje significativo y

duradero, fomentando que los estudiantes asuman un rol activo en su formación. Sin embargo, los autores han seleccionado el aprendizaje basado en proyectos como la metodología principal de esta obra, ya que consideran que esta estrategia no solo enriquece la experiencia de aprendizaje del estudiante, sino que también permite al docente estructurar un ambiente formativo que puede extenderse a lo largo de un periodo académico completo, facilitando así el diseño curricular de toda la actividad.

### **Aprendizaje Basado en Problemas (ABP)**

El aprendizaje basado en problemas (ABP) tuvo su origen en la Escuela de Medicina de la Universidad de McMaster en Canadá, en la década de 1960. Fue desarrollado por Howard S. Barrows y su equipo, como un enfoque innovador para la educación médica. La motivación detrás del desarrollo del ABP fue proporcionar a los estudiantes de medicina una forma más efectiva de aprender a aplicar el conocimiento científico en la resolución de problemas médicos reales (Bueno et al., 2004).

Se caracteriza porque el estudiante se enfrenta a la solución de un problema, que se define para reflejar situaciones auténticas y relevantes del mundo real, lo suficientemente complejas y desafiantes, para que los aprendices integren conocimientos de múltiples áreas y empleen habilidades de pensamiento crítico con el fin de encontrar las soluciones; también se requiere que el problema integre conocimientos y habilidades de múltiples disciplinas, lo que fomenta un enfoque holístico del aprendizaje y refleja la naturaleza interdisciplinaria de muchos problemas del mundo real (Fernández Martínez et al., 2006).

Los problemas en el ABP, deben encontrarse contextualizados dentro del marco de referencia del currículo o del tema de estudio, esto ayuda a los estudiantes a comprender la relevancia y aplicabilidad del conocimiento en situaciones específicas; finalmente, los problemas deben ser diseñados para que motiven y capten el interés de los alumnos, lo que fomenta la motivación intrínseca de ellos para participar en el proceso de aprendizaje (Kolmos et al., 2017).

Algunos trabajos orientados a la robótica que han usando esta metodología de enseñanza aprendizaje han sido divulgados, entre ellos tenemos “Un ABP basado en la robótica para las ingenierías informáticas” de Oliver et al (2009), donde se propone el uso de un robot educativo para un curso de la carrera de Ingeniería Informática de la Universidad Politécnica de Catalunya, cuyo objetivo principal fue el de aumentar la motivación de los alumnos que llegan al primer curso de la titulación.

Por otra parte, Echevarría (2022), propone en su trabajo “Motivados por la robótica” una práctica pedagógica basada en ABP para un colegio de secundaria utilizando un robot educativo. En este caso, la asignatura en la que se realizó fue Tecnología de la Información y las Comunicaciones, en donde el objetivo principal fue abordar el tema “Programación y Robótica” de una forma diferente, en la que el estudiante se enfrentara a la problemática establecida haciendo uso del pensamiento computacional mediante el lenguaje de programación gráfica *Scratch*.

### **Aprendizaje cooperativo**

El aprendizaje cooperativo es un enfoque pedagógico que promueve la colaboración entre los estudiantes para alcanzar metas educativas



comunes, tiene sus raíces más profundas desde los años 1920, sin embargo, la verdadera revolución y los principios que rigen esta técnica de formación surgieron alrededor de 1938, con el estudio de la psicología social, donde se concluyó que, “dos o más individuos resuelven los problemas mejor, trabajando cooperativamente que de forma aislada y que la superioridad de ‘dos cabezas’ sobre una es tan clara que resulta inútil más investigaciones sobre el tema (...) Una situación de aprendizaje cooperativo es aquella en la que los estudiantes pueden conseguir sus objetivos sí y solo si, los demás con los que trabaja cooperativamente consiguen también los suyos” (R. García et al., 2019, p. 23).

De esta forma, se puede indicar que en esta estrategia formativa los estudiantes dependen unos de otros para alcanzar los objetivos, que el aprendiz es responsable tanto de su propio aprendizaje como del éxito del grupo, esto, a su vez promueve un ambiente de apoyo mutuo y refuerzo positivo entre los miembros del grupo y, por lo tanto, fomenta la construcción activa del conocimiento a través de la discusión, la argumentación y la resolución conjunta de problemas, lo que puede llevar a un aprendizaje más profundo y significativo (López & Acuña, 2011).

Es importante recalcar que, de acuerdo con los trabajos de Keyser (2000) y López y Acuña (2011), el aprendizaje cooperativo debe contener los siguientes elementos:

- Interdependencia positiva: el esfuerzo de cada estudiante no solamente lo beneficia a sí mismo, sino que también contribuye positivamente al resto de los miembros del grupo.
- Interacción cara a cara: fomentar el aprendizaje mediante la ayuda mutua, el intercambio de conocimientos, la enseñanza

entre pares, el estímulo y la colaboración en la distribución de recursos y materiales.

- Responsabilidad individual: todos los participantes se comprometen a lograr los objetivos y asumen la responsabilidad de cumplir con el trabajo asignado. Se espera que nadie permanezca inactivo ni se beneficie del esfuerzo de los demás.
- Habilidades interpersonales y de pequeño grupo: liderazgo, la promoción de un ambiente de confianza, la comunicación efectiva, el respeto mutuo, la toma de decisiones colaborativas y la habilidad para resolver conflictos.
- Reflexión individual y grupal: juntos reflexionan y evalúan si están logrando los objetivos establecidos y si las relaciones dentro del grupo son efectivas. Identifican acciones positivas y negativas, determinando qué comportamientos mantener y cuáles eliminar.

Finalmente, algunas recomendaciones que se dan para generar un ambiente propicio para el aprendizaje cooperativo y aprovechar al máximo sus beneficios son: procurar la conformación de grupos heterogéneos, de tal forma que se incluyan una variedad de habilidades y estilos de aprendizaje, lo que puede promover un aprendizaje más rico y brinda oportunidades para que los estudiantes se apoyen mutuamente; También, diseñar tareas donde se necesite que los estudiantes trabajen juntos y dependan unos de otros para alcanzar los objetivos del aprendizaje, animar a escuchar activamente las ideas de sus compañeros y a expresar las propias de manera coherente y fundamentada (López & Acuña, 2011).

Algunos trabajos de robótica, donde se usó la estrategia de enseñanza aprendizaje colaborativo se encuentran en los trabajos de

Chavarría (Chavarría & Saldaño, 2010) y Moscaiza (Moscaiza Torres, 2022), ellos se muestran la manera como la estrategia impactó, de manera positiva, la formación de los estudiantes que participaron en los proyectos.

### ***Flipped Classroom o Aula Invertida***

El concepto de aula invertida, también denominado *Flipped Classroom* en inglés, implica una reestructuración de los roles tradicionales en la enseñanza. En lugar de que el estudiante reciba nuevo contenido durante las clases mediante exposiciones del profesor, para luego realizar tareas en casa, los alumnos acceden al material de aprendizaje fuera del aula, usualmente a través de videos, lecturas u otros recursos, antes de asistir a clase. Durante el tiempo en el aula, los estudiantes participan en actividades interactivas, discusiones, resolución de problemas o proyectos que están relacionados con el contenido que han revisado con anticipación. De acuerdo a Ledo et al (2016):

Un enfoque pedagógico en el que la instrucción directa mueve desde un espacio de aprendizaje colectivo a un espacio de aprendizaje individual al estudiante, y el espacio de aprendizaje colectivo resultante, se transforma en un ambiente de aprendizaje dinámico e interactivo, donde el docente guía a los estudiantes a medida que él aplica los conceptos y participa creativamente en el tema. (p. 687)

De esta forma, el desarrollo tecnológico impacta de forma directa las estrategias de enseñanza y aprendizaje, dado que cada vez es más fácil construir contenido que pueda ser enviado y revisado desde cualquier lugar del mundo. Sin embargo, el método puede verse interrumpido si el estudiante, quien debe realizar el trabajo de revisión y aprendizaje por su propia cuenta y fuera del aula, no lo

hace; frente a esta dificultad, será necesario buscar mecanismo para sobrepasarla.

Sin embargo, si el estudiante participa de manera activa en su proceso de aprendizaje, la metodología puede tener las siguientes ventajas:

- Produce una mayor implicación del estudiante.
- Permite una mayor adaptación al ritmo de cada estudiante.
- Produce un aprendizaje más profundo y significativo, al permitir que se trabajen, en combinación con el aula, todos los niveles cognitivos (Walsh, 2013 citado por Ruiz Vicente, 2017, p. 84).

Para concluir y de acuerdo con Cruz et al (2023), quien indica que a lo largo de los años se han llevado a cabo numerosos estudios sobre la efectividad del aula invertida en comparación con los métodos de enseñanza tradicionales, si bien la investigación ha arrojado resultados mixtos en términos de impacto en el rendimiento académico, muchos educadores han encontrado que el enfoque propuesto mejora la participación de los estudiantes y promueve un aprendizaje más profundo y significativo.

### **Aprendizaje basado en proyectos (ABP)**

El aprendizaje basado en Proyectos (ABP) ha sido definido como:

Un conjunto de tareas que culminan con un producto final y que los estudiantes deben resolver de una manera autónoma mediante procesos de investigación. Para la realización del proyecto, los estudiantes tienen la oportunidad de trabajar de manera autónoma durante la mayor parte del tiempo, que culmina en la realización de un producto final presentado frente a los demás. (Sánchez, 2013 citado por Ruiz Vicente, 2017)

Una definición más escueta del ABP es presentada por Estruch y Silva (2006), donde se indica que es “una metodología activa que toma al grupo como unidad fundamental de trabajo (frente al individuo por sí sólo) para resolver un problema de carácter real y multidisciplinar” (p. 340).

De esta forma, y tal como se ha indicado a lo largo de estas líneas, las metodologías de aprendizaje activo ubican al estudiante como eje central del proceso de formación. En el ABP, no solo ocurre esto, sino que proporciona un ambiente que genera la necesidad de aprender para poder solucionar, lo que permite que el aprendizaje se convierta en un acto intensional y, por lo tanto, duradero. De acuerdo con Vergara (2015), el ABP se convierte en un modelo que se orienta a la vivencia de experiencias educativas, en lugar de modelos de transmisión de contenidos.

Por otra parte, el rol del profesor está definido en el trabajo “Aprendizaje basado en proyectos: una experiencia de innovación docente” de Martí et al (2010), donde indica que “el docente actúa como orientador o guía y, también aprende durante todo el proceso” (p. 340), sin embargo, y complementando lo anterior, el docente también tiene las tareas de: seleccionar el proyecto, planearlo, realizar seguimiento al grupo y evaluar el proceso; esto último, se realiza en conjunto con los estudiantes.

En cuanto a la organización o el orden de actividades que deben ser seguidas dentro del ABP, con el fin de definir una estructura guía para los estudiantes y el profesor, y que permita su implementación en el aula de clase, de acuerdo con Quintero y Prado (2020), se tiene la siguiente distribución:

**Paso 1:** Punto de partida.

Se realiza una introducción del proyecto a los estudiantes,

proporcionándoles una descripción clara del problema o tema a abordar y se les presenta el marco general del proyecto. Es posible presentar también una pregunta guía. Se crean grupos de trabajo lo más heterogéneos posible, máximo de cuatro estudiantes y se induce a los estudiantes a generar ideas sobre la forma en la que se podría resolver el proyecto planteado.

**Paso 2:** Necesidades cognitivas de los estudiantes.

Los estudiantes realizan una reflexión del conocimiento, las habilidades y las fortalezas requeridas para enfrentar el proyecto. Cada estudiante deberá adoptar un rol dentro de su equipo de trabajo y, de acuerdo a esto, definir sus tareas dentro del grupo.

**Paso 3:** Búsqueda de información.

Los estudiantes comienzan a investigar el tema o problema, recopilan información relevante y desarrollan un plan de acción para abordar el proyecto. Esto puede incluir la identificación de preguntas de investigación, la búsqueda de recursos, la planificación de actividades y la distribución de tareas.

**Paso 4:** Gestión y planeación del trabajo.

Una vez identificada la información y los recursos necesarios para el desarrollo del proyecto, el grupo realiza un plan de trabajo donde definen tiempos y se establecen tareas. Normalmente se presenta un cronograma de actividades.

**Paso 5:** Síntesis y análisis de información.

En esta etapa del proceso, los estudiantes de cada grupo comparten sus ideas y conocimientos, contrastan opiniones, resuelven problemas en consenso y toman decisiones.

**Paso 6:** Actuar “la realización del proyecto”.

Los estudiantes llevan a cabo las actividades planificadas y

trabajan en equipos o de manera individual, según sea necesario, aplicando los conceptos y habilidades aprendidos durante la investigación. Durante esta fase, es importante que los estudiantes mantengan un registro de su progreso y realicen ajustes según sea el caso.

**Paso 7:** Presentación de resultados, conclusiones y perspectivas.

Una vez completado el proyecto, los estudiantes presentan sus hallazgos, soluciones o productos al resto de la clase. Esto puede incluir presentaciones orales, demostraciones, informes escritos, exhibiciones o cualquier otra forma de comunicación de los resultados del proyecto.

**Paso 8:** Publicación y retroalimentación (opcional).

Esta etapa es opcional y depende del alcance del proyecto, en ella se busca presentar los resultados del proceso a pares externos, a la sociedad o al sector industrial. Se busca con esto contextualizar el proyecto fuera del aula de clase o del centro de formación.

**Paso 9:** Retroalimentación y evaluación.

Se evalúa el trabajo de los estudiantes de acuerdo con los criterios establecidos al comienzo del proyecto. Además, se fomenta la reflexión sobre el proceso de aprendizaje, incluyendo lo que funcionó bien, los desafíos encontrados y las lecciones aprendidas. Esta fase proporciona la oportunidad de retroalimentación para mejorar futuros proyectos y promover un aprendizaje más profundo.

Es posible encontrar literatura donde haya mayor o menor número de etapas para el desarrollo del proyecto, pero al final, todas tienen la misma equivalencia y lo que ocurre es que, por ejemplo, el paso uno lo dividen en dos etapas: presentación del proyecto y conformación de equipos, incrementándose de esta manera la estructura.

Detrás del trabajo que se realiza, con la guía presentada, existe para los autores una fase cero (Paso 0). Es una etapa ejecutada por el docente, que normalmente comienza antes de la presentación del proyecto al grupo de estudiantes y continua hasta la terminación del proceso de formación. Este paso podría ser llamado “Planificación y diseño”; en él se identifican los objetivos de aprendizaje, se selecciona un tema o problema relevante, se diseña el proyecto que será presentado a la clase, se gestionan los recursos locativos, las herramientas, los insumos, entre otras cosas necesarias para la ejecución del ABP. Esto implica definir el alcance del proyecto, establecer los criterios de éxito y desarrollar un plan de trabajo detallado, propio del docente, además, durante el tiempo que dure el proceso de formación ABP, el profesor se debe mantener alerta para realizar los ajustes “en línea” que considere necesarios para el éxito de la formación.

Para cerrar, es importante precisar que los proyectos de clase en formación tradicional, no corresponden ni se aproximan a la filosofía del ABP, es decir, de acuerdo con Liu y Pedersen (1998), mientras que en los proyectos de aula tradicionales primero se expone la información y luego se busca su aplicación para resolver una situación real, en el aprendizaje por proyectos, primero se presenta el problema, el reto o producto, luego se identifican las necesidades de aprendizaje, se busca la información y finalmente, se resuelve el proyecto.

Otro aspecto importante en el ABP es la evaluación realizada a los estudiantes; se sabe que es un asunto neurálgico dentro de los procesos de formación y que debe ser llevado de forma adecuada para que el resultado sea confiable. En tal sentido, el ABP utiliza una evaluación formativa que debe ser de calidad, justa y eficaz, ajustada a los objetivos del proyecto, con criterios de evaluación claros,



compresibles y manejables (Trujillo, 2016). En esta línea, el mismo autor indica que pueden ser usados otros mecanismos evaluativos, diferentes a los exámenes individuales y grupales, tales como: el diario de aprendizaje, rubricas, diana de evaluación, portafolio, autoevaluación y coevaluación. En todo caso, la evaluación se vuelve un asunto complejo, que debe quedar bien planeada y definida, de tal forma que evalúe los conocimientos adquiridos por los alumnos, su desempeño y aportes al proyecto, y su participación en el grupo de trabajo.

Como se mencionó anteriormente, el ABP es una metodología de enseñanza–aprendizaje que ha crecido en su uso, ya que presenta beneficios importantes para el alumno que se involucra dentro del proceso y entre los cuales podemos destacar los siguientes:

- Prepara a los estudiantes para el campo laboral, ya que al trabajar en proyectos que reflejan situaciones del mundo real, los estudiantes adquieren habilidades y conocimientos que son directamente aplicables en la vida y en el lugar de trabajo, generando una conexión entre la escuela y la realidad; por otro lado, brinda mayor motivación para el estudio.
- El sentido desafiante que tiene el proyecto enriquece habilidades para la solución de problemas, lo que obliga a los estudiantes a pensar de manera crítica, y a desarrollar destrezas para abordar problemas complejos y encontrar soluciones creativas; en el mismo sentido, aumenta las habilidades sociales y de comunicación, debido a la necesidad del trabajo en equipo(Coria Arreola, 2010).

En general, la metodología ABP puede desarrollar una amplia gama de habilidades: las académicas como lectura, escritura y matemáticas, las que son prácticas como programación, diseño y

presentación, y aquellas que hacen parte del individuo, como la autonomía, la responsabilidad, el trabajo en equipo, el liderazgo, entre otras.

En el campo de la robótica, el ABP ha sido utilizado en diferentes trabajos, desde los primeros años de escolaridad hasta los programas universitarios. Un ejemplo de esto lo encontramos en el trabajo titulado “Motivados por la robótica” de Echevarría (2022), donde se aborda el tema “Programación y Robótica” utilizando los recursos que tiene el colegio, como un robot educativo, una sala de informática y un aula móvil con computadoras. Otro ejemplo, es “Una Experiencia de Aprendizaje Basado en Proyectos en una Asignatura de Robótica” trabajo de Aliane et al (2008), en el que presenta un proyecto a nivel universitario, que consiste en el desarrollo de un sistema de juego de damas contra un robot manipulador; el proyecto tiene una connotación multidisciplinar, ya que integra varias tecnologías: la robótica, la visión por ordenador y la inteligencia artificial. Finalmente, encontramos en el trabajo “PequeBot: Propuesta de un Sistema Ludificado de Robótica Educativa para la Educación Infantil de da Silva y González (2017) el diseño de un sistema ludificado para el aprendizaje en la educación infantil, por medio de la robótica educativa (PequeBot), que mediante la utilización de metodologías de aprendizaje basada juegos y en proyectos, facilita a los estudiantes la adquisición de conocimientos en las diferentes áreas del currículo educativo.

Es fundamental considerar que los robots están cada vez más presentes en las aulas, no solo con el propósito de enseñar la robótica en sí misma, sino también para aprovechar su naturaleza multidisciplinaria (mecánica, programación, física, matemáticas, entre otras), y estimular procesos cognitivos que promuevan un aprendizaje

significativo y una introducción al mundo de la ciencia y la tecnología. El robot LEGO MINDSTORMS EV3 es un ejemplo de esto y está siendo empleado extensamente, no solo en el ámbito educativo, sino también en diversos proyectos de investigación.

### **Una propuesta de ABP usando el robot LEGO® MINDSTORMS® EV3**

A continuación, se presenta la planeación de una propuesta de enseñanza basada en el Aprendizaje Basado en Proyectos (ABP) para abordar los temas tratados en este libro. El objetivo es integrar el uso del robot LEGO MINDSTORMS EV3 con la enseñanza de la programación en *Python*, utilizando recursos pedagógicos activos que fomenten el aprendizaje significativo.

Esta metodología busca que los estudiantes participen de manera activa en su propio proceso de formación, promoviendo el desarrollo de habilidades prácticas y el pensamiento crítico, a través de la experimentación y la resolución de los problemas asociados al proyecto propuesto.

La planificación previa que el profesor debe realizar es, sin duda, una de las fases más cruciales de toda la experiencia. En esta etapa, llamada “Fase cero” o “Paso 0”, se definen y consolidan los elementos fundamentales para el desarrollo exitoso del proyecto.

A continuación, se presentan los principales aspectos que conforman esta planificación:

1. Proponer un título al proyecto.
2. Describir el objetivo general del proyecto.
3. Definir los objetivos específicos del proyecto.
4. Estructura del proyecto:
  - a. Describir el proyecto de forma clara, de tal manera que se

identifiquen los requerimientos de operación que tendrá el dispositivo final.

- b. Definir los temas de aprendizaje con mayor relevancia y que serán, de una u otra forma, usados por los estudiantes durante el desarrollo del proyecto.
  - c. De ser necesario, identificar algunas prácticas que le permitan a los estudiantes contextualizar los temas de aprendizaje.
5. Definir tiempos de ejecución y material de evaluación:
- a. Definir un cronograma principal, que sería el cronograma del docente, donde se proponen las fases propias del desarrollo de la metodología ABP y los tiempos de ejecución, también debe aparecer los momentos de evaluación.
  - b. Desarrollar el material de evaluación que se usará en el proceso de formación, en este caso, lo recomendado es el uso de rúbricas con criterios como los siguientes:
    - i. Funcionalidad del código.
    - ii. Eficiencia del código.
    - iii. Integración de sensores.
    - iv. Creatividad en el diseño del recorrido.
    - v. Presentación y documentación.
    - vi. Entre otros más.
6. Realizar un listado de los recursos necesarios que se requieren en la implementación del proceso.

Con base en lo expuesto anteriormente, a continuación, se presenta una propuesta de Aprendizaje Basado en Proyectos (ABP) en su fase de planificación. Esta propuesta está orientada a la enseñanza de los temas relacionados con la programación del robot LEGO

MINDSTORMS EV3 utilizando el lenguaje *Python*.

## **Propuesta ABP en su fase de planificación**

### **1. Título del Proyecto:**

“Diseño e implementación de un explorador Autónomo con el robot LEGO® MINDSTORMS® EV3”

### **2. Objetivo General:**

Desarrollar habilidades en programación mediante el desarrollo de un sistema de navegación autónomo que utiliza el robot LEGO® MINDSTORMS® EV3 como vehículo y Python como lenguaje de programación.

### **3. Objetivos Específicos:**

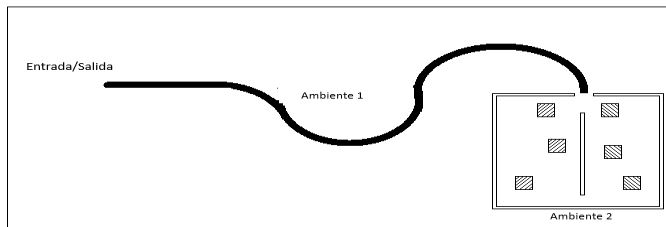
- Implementar secuencias de movimiento utilizando bucles y estructuras condicionales en Python.
- Diseñar algoritmos para trayectorias específicas como recorridos en cuadrados, espirales y movimientos aleatorios.
- Aplicar principios de optimización en la programación de trayectorias.
- Integrar sensores para mejorar la autonomía del robot.

### **4. Estructura del proyecto**

Descripción del proyecto: el proyecto propone el diseño e implementación de un sistema autónomo de navegación para el robot LEGO® MINDSTORMS® EV3, permitiéndole desplazarse de manera eficiente a lo largo de una pista con dos ambientes diferenciados. En la primera sección, el robot seguirá una línea negra que lo guiará a través de un recorrido con curvas de 20, 50 y 90 grados. Una vez el robot salga de esta fase, al frente se encontrará con un entorno más dinámico, donde se desplazará de forma autónoma, sorteando

obstáculos hasta encontrar nuevamente la salida y retomar la línea negra del tramo inicial. Desde allí, seguirá la trayectoria inversa hasta regresar al punto de partida. Finalmente, ejecutará un movimiento en espiral para completar la ruta en su totalidad. En la Figura 22, se muestra la propuesta que tendría la pista que el vehículo debería recorrer.

**Figura 22.**  
*Propuesta de la pista. Fuente: elaboración propia*



Principales temas de aprendizaje:

- Estructura básica de un programa.
- Control de un robot usando como base la biblioteca EV3dev.
- Control básico de motores.
- Implementación de bucles.
- Control de tiempos, control de giros y velocidades de motores.
- Análisis del código y depuración de errores.
- Uso de estructuras condicionales.
- Uso de los sensores (en este caso particular de tacto, uso del sensor ultrasónico).

Información útil:

En general, la documentación que aparece en este texto y más

específicamente las practicas propuestas en el capítulo 5: “Proyectos Prácticos”

#### 5. Definir tiempos de ejecución y material de evaluación

##### Cronograma principal del proyecto

En la Figura 23 se muestra la propuesta de cronograma que el docente podría seguir durante el desarrollo del proyecto mencionado anteriormente. Este cronograma sirve como guía para organizar las actividades y asegurar una adecuada implementación del Aprendizaje Basado en Proyectos (ABP) en la programación del robot LEGO MINDSTORMS EV3 con Python.

En el cronograma, la primera columna hace referencia a los pasos en que se ejecuta la metodología ABP, estos fueron descritos anteriormente y a continuación se enuncian:

- Paso 1: Generación de ideas.
- Paso 2: Necesidades cognitivas del alumno
- Paso 3: Búsqueda de información
- Paso 4: Análisis y síntesis
- Paso 5: Gestión/ Planeación por parte de los estudiantes de sus actividades y tiempos.
- Paso 6: Actuación
- Paso 7: Conclusiones/perspectivas
- Paso 8: Publicación/ Presentación pública del proyecto
- Paso 9: Retroalimentación

**Tabla 1.**  
*Cronograma*

| Pasos   | Actividad                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|---------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1, 2    | Presentación del proyecto, conformación de grupos y lluvia de ideas inicial<br>Presentación del Cronograma por parte de cada grupo, para el desarrollo de las diferentes actividades que lleva el proyecto. Definición de las necesidades de conocimiento.                                                                                                                                                                                                                                                                                                                                                                                    |
| 3, 4, 5 | Búsqueda de información y presentación por parte del docente de los conceptos que fueran necesarios para que el estudiante desarrolle las actividades requeridas en la ejecución del proyecto<br>Desarrollo del programa que permita seguir la línea al robot LEGO<br>Evaluación del programa que realice el seguimiento de la línea.<br>Desarrollo del programa que permita ejecutar un movimiento en espiral al robot LEGO<br>Integración del programa seguidor de líneas con el programa que mueve en espiral al robot.<br>Evaluación de la integración entre los programas seguidor de línea y el programa que mueve en espiral al robot. |
| 6       | Desarrollo del programa que permita evadir obstáculos al robot LEGO<br>Evaluación del programa que realiza la evasión de obstáculos y su integración con el seguimiento de la línea<br>Pruebas Y ajustes en la operación del robot sobre la pista final que se usará en la prueba                                                                                                                                                                                                                                                                                                                                                             |
| 7       | Entrega del proyecto operando de acuerdo con las especificaciones que se indicaron al inicio<br>Reunión con los estudiantes donde se realice una retrospectiva y se concluya sobre el desarrollo del proyecto.                                                                                                                                                                                                                                                                                                                                                                                                                                |
| 8       | Presentación pública del proyecto mediante una competencia. En dicho evento, se premiará a los dos sistemas que mayor eficiencia presenten. (Recorrido de la pista vs tiempo)                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| 9       | Retroalimentación por parte de jurados externos invitados al evento.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |



## Propuesta de evaluación

Dentro del proceso de evaluación del proyecto, es común utilizar la rúbrica como una herramienta para diagnosticar los logros alcanzados por los estudiantes. No obstante, el docente tiene la libertad de elegir los modelos evaluativos que considere más adecuados para el proceso de formación. En la Figura 24 se presenta una propuesta de rúbrica diseñada para evaluar la primera actividad establecida en el cronograma, cuyo objetivo es verificar que el robot sea capaz de seguir una línea de manera eficiente.

**Tabla 2.**  
*Propuesta de Rábica*

| Objetivo: Evaluar el programa que realiza el seguimiento de la línea. |                                                                                    |                                                             |                                                        |                                                          |                                                              |             |      |
|-----------------------------------------------------------------------|------------------------------------------------------------------------------------|-------------------------------------------------------------|--------------------------------------------------------|----------------------------------------------------------|--------------------------------------------------------------|-------------|------|
| Criterio                                                              | Descripción                                                                        | Nivel 1 (1-2 puntos)                                        | Nivel 2 (3 puntos)                                     | Nivel 3 (4 puntos)                                       | Nivel 4 (5 puntos)                                           | Ponderación | Nota |
| Puntualidad en la entrega                                             | Entrega del programa dentro del tiempo establecido.                                | No entrega o lo hace fuera del plazo sin justificación.     | Entrega fuera del plazo con justificación.             | Entrega dentro del plazo, pero con retraso menor a 24h.  | Entrega en el plazo estipulado.                              | 30%         |      |
| Funcionalidad del código                                              | El programa debe permitir que el robot siga la línea correctamente.                | El robot no sigue la línea o el código presenta errores.    | El robot sigue la línea de manera inconsistente.       | El robot sigue la línea con algunos errores menores.     | El robot sigue la línea de manera precisa y estable.         | 25%         |      |
| Documentación del código                                              | Uso de comentarios y buenas prácticas de codificación.                             | No hay comentarios ni estructura en el código.              | Comentarios insuficientes o poco claros.               | Comentarios adecuados pero podrían mejorarse.            | Comentarios claros y bien estructurados.                     | 15%         |      |
| Integración de sensores                                               | Correcta configuración e integración de los sensores para el seguimiento de línea. | No se integraron sensores o su configuración es incorrecta. | Sensores integrados pero con errores funcionales.      | Sensores bien integrados con ajustes menores requeridos. | Sensores correctamente integrados y funcionando óptimamente. | 15%         |      |
| Entrega de documentación                                              | Presentación de un informe detallado del desarrollo del programa.                  | No se entrega documentación.                                | Documentación incompleta o con errores significativos. | Documentación adecuada pero falta detalle.               | Documentación clara, completa y bien estructurada.           | 15%         |      |

### 6. Listado de recursos necesarios

A continuación, se presentan los recursos mínimos que el docente debe considerar para desarrollar el proyecto bajo la metodología ABP. Es importante tener en cuenta que esta lista puede ajustarse a medida que avanza la ejecución del proyecto, especialmente cuando

se implementa por primera vez. No obstante, proporciona una muy buena base para iniciar sin contratiempos.

- Kit LEGO Mindstorms EV3 con firmware EV3dev.
- Computador con entorno de desarrollo Python (Visual Studio Code, Thonny, u otro).
- Sensores adicionales (color, ultrasonido, giroscopio) para la optimización de movimientos.
- Espacio delimitado para pruebas de recorrido.
- Pista con obstáculos.
- Pista para desarrollar el seguidor de línea.

Se ha propuesto el desarrollo de un curso de programación en Python utilizando el robot LEGO® MINDSTORMS® EV3, empleando una metodología de enseñanza-aprendizaje activa. Aunque la propuesta incluye una rúbrica y un cronograma general del proyecto, en el anexo C se presentan herramientas y recursos adicionales para ser usados dentro del proyecto “Diseño e implementación de un explorador Autónomo con el robot LEGO® MINDSTORMS® EV3”, tales como:

- Las rúbricas para evaluar los cuatro momentos en que se entregan los trabajos prácticos indicados en el cronograma general.
- El formato de una bitácora para cada clase, la cual es diligenciada por cada uno de los grupos que conforman el curso.
- La bitácora donde se consigna la información más relevante, una vez culminada cada una de las etapas del proyecto.
- Un formato para la coevaluación del grupo.
- Una plantilla para la autoevaluación de cada estudiante.
- Una plantilla con los ítems más importantes que deben aparecer en el informe final presentado por cada grupo.

Por otra parte, algunos recursos adicionales que se encuentran en la web y que podrían ser útiles, son los siguientes:

- Lista de videos con simulaciones prácticas, como seguimiento de línea básico y movimientos simples en EV3. En el siguiente enlace (Code Robots, 2020):

<https://www.youtube.com/playlist?list=PLOO2WubedeopKstvOh5k3nTMI5yzaP4Jt>

- Uso de algoritmos PID para seguidor de línea con robot EV3. En el siguiente enlace (Medium, R. & W., 2019):

<https://medium.com/kidstronics/lego-pid-the-ultimate-line-follower-45d4e517572b>

- Desarrollo de un robot llamado “The Launcher”, útil para ilustrar integración práctica de hardware y programación como actividad de ABP. En el siguiente enlace (LEGORobotics Mr. Hino, 2018):

[https://www.youtube.com/watch?v=Kl\\_tZvwmZew&t=20s](https://www.youtube.com/watch?v=Kl_tZvwmZew&t=20s)

Toda la información descrita anteriormente, es presentada como base inicial para que docentes y lectores interesados la adapten a sus procesos, y la fortalezcan con nuevos elementos. De esta manera, la propuesta se configura como un punto de partida sólido para futuras experiencias educativas que integren programación, robótica y metodologías activas en el aula.

Finalmente, en el anexo B, se presenta dos propuestas adicionales de proyectos organizadas en un formato de planeación ABP con elementos claros que permiten una guía para la implementación de un curso.

## ANEXO A. GUÍA DE ESTILO PEP 8 – BUENAS PRÁCTICAS EN PYTHON

### ***¿Qué es PEP 8?***

PEP 8 (Python Enhancement Proposal 8) es el documento oficial que establece las convenciones de estilo para escribir código en el lenguaje de programación Python de forma clara, coherente y fácil de mantener. Fue elaborado por Guido van Rossum (creador de Python), junto a Barry Warsaw y Nick Coghlan, y se ha convertido en la referencia fundamental para todos aquellos que desarrollan *software* en este lenguaje (P. Python, 2025).

Aplicar PEP 8 tiene como objetivo facilitar la lectura del código, mejorar la colaboración entre desarrolladores y fomentar la calidad del *software*. En proyectos educativos, personales o profesionales, seguir estas normas ayuda a mantener un estilo uniforme, independientemente del autor del código o el tamaño del proyecto.

### ***Formato general***

- Longitud de línea: se recomienda que cada línea de código tenga como máximo 79 caracteres. Esto mejora la visualización en editores divididos o en pantallas pequeñas.
- Indentación: utilizar exactamente 4 espacios por nivel de indentación. No se deben usar tabuladores para evitar inconsistencias entre editores de texto.
- Líneas en blanco:
  - Dejar 2 líneas en blanco antes de definir funciones o clases de nivel superior.
  - Dejar 1 línea en blanco entre métodos dentro de una clase

para mejorar la claridad.

Estos detalles, aunque puedan parecer menores, son clave para que el código sea fácil de entender para otros desarrolladores (o para uno mismo en el futuro).

### ***Nombres de identificadores***

- Funciones y variables: deben usar letras minúsculas con guiones bajos para separar palabras. Ejemplo: `calcular_promedio`.
- Clases: se recomienda usar el estilo *CamelCase*, donde cada palabra empieza con mayúscula y no hay guiones bajos. Ejemplo: `RobotAutonomo`.
- Constantes: deben escribirse completamente en mayúsculas, separando palabras con guiones bajos. Ejemplo: `PI_VALOR`.
- Archivos/módulos: seguir la misma regla que para funciones y variables: nombres en minúsculas con guiones bajos.

El uso coherente de nombres mejora la comprensión semántica del código.

Espaciado en expresiones

Correcto:

```
x = 5
total = x + 2
```

Incorrecto:

```
x=5
total=x+2
```

- Evitar espacios innecesarios:
  - No colocar espacios antes de paréntesis, corchetes o llaves. Por ejemplo: `funcion(valor)` está bien; `funcion (valor)` está mal.
  - No abusar de los espacios alrededor de operadores, salvo cuando mejore la legibilidad.

Estos pequeños detalles visuales ayudan a mantener el código ordenado.

### ***Comentarios y docstrings***

- Comentar el porqué del código, no solo el qué hace.
- Los comentarios deben ser claros, iniciar con mayúscula y estar precedidos por `#` seguido de un espacio:

`# Este es un buen comentario explicativo`

### **Uso de docstrings.**

Los *docstrings* son cadenas de texto que se colocan justo después de la definición de funciones, clases o módulos, encerradas entre triple comillas. Se utilizan para documentar el propósito y funcionamiento del bloque de código.

Ejemplo para una función:

```
def calcular_area_triangulo(base, altura):  
    """  
    Calcula el área de un triángulo dados su  
        base y altura.  
  
    Args:
```

```
        base (float): Longitud de la base.
        altura (float): Altura correspondiente.

Returns:
    float: Área del triángulo.
"""
return (base * altura) / 2

Ejemplo para una clase:
class Robot:
    """
    Representa un robot con nombre y energía.

    Attributes:
        nombre (str): Nombre del robot.
        energia (int): Nivel de energía.
    """
    def __init__(self, nombre):
        """ # Inicializa el robot con nombre y
            # energía por defecto."""
        self.nombre = nombre
        self.energia = 100
```

Se recomienda seguir estilos como Google, NumPy o reStructuredText para mantener consistencia y compatibilidad con herramientas como Sphinx.

## Importaciones

- Deben organizarse en bloques separados por una línea en blanco:
  1. Módulos de la biblioteca estándar
  2. Módulos de terceros
  3. Módulos propios del proyecto

Ejemplo:

```
import os
```

```
import sys
```

```
import numpy as np
from mi_paquete import mi_modulo
```

- No se recomienda usar `from modulo import *`, ya que esto introduce nombres de forma implícita y puede provocar errores o colisiones.

Una buena organización de importaciones es clave para mantener claridad.

### ***Estructura del archivo Python***

1. La estructura sugerida para un script Python es:
2. Comentario de módulo y docstring inicial (descripción general del archivo)
3. Bloque de importaciones
4. Definición de constantes globales
5. Declaración de funciones
6. Declaración de clases (si las hay)
7. Bloque principal con `if __name__ == "__main__":`



Este orden facilita la navegación y reutilización del código.

### ***Herramientas para verificar PEP 8***

Existen herramientas que automatizan la revisión y corrección del estilo PEP 8. Algunas de las más utilizadas son:

- Flake8: detecta errores de estilo, sintaxis y convenciones.
- Black: aplica formateo automático y uniforme a todo el código.
- Pylint: analiza el código con profundidad, detectando errores y sugerencias de mejora.

Ejemplo de uso:

```
flake8 mi_script.py
```

Estas herramientas ayudan a mantener la calidad del código de manera continua.

Seguir las recomendaciones de estilo del PEP 8 es una práctica altamente recomendada en el desarrollo en Python. No solo mejora la legibilidad del código y facilita el trabajo en equipo, sino que también refuerza la profesionalidad del programador.

Adoptar estas convenciones desde el inicio del aprendizaje promueve hábitos de codificación que serán valiosos a lo largo de la carrera profesional. Es una herramienta clave para lograr código limpio, mantenible y compatible.

## **ANEXO B. PROPUESTAS ADICIONALES DE PROYECTOS USANDO ABP**

### **Proyecto 1: Robot Seguidor de Línea con Estaciones Inteligentes**

#### ***Descripción del proyecto***

Los grupos de trabajo deberá diseñar y programar un robot LEGO EV3, usando programación en Python, que sea capaz de recorrer un circuito marcado con una línea negra sobre un fondo blanco, haciendo uso de los sensores disponible que permiten seguir trayectoria. Durante el recorrido, el robot encontrará “estaciones” que estarán marcadas con colores diferentes, en las cuales el robot deberá detenerse, sonar una alarma y mostrar un mensaje en la pantalla con el color de la estación.

#### ***Planeación ABP – Proyecto 1***

Título del proyecto: Robot Seguidor de Línea con Estaciones Inteligentes

#### ***Objetivo general***

Desarrollar un robot autónomo con LEGO EV3, programado en Python, que pueda recorrer un circuito siguiendo una línea negra y reconocer estaciones de distintos colores, ejecutando acciones específicas en cada una de ellas.

#### ***Objetivos específicos***

- Programar en Python las funciones necesarias para que el robot pueda seguir con precisión una línea negra sobre fondo blanco, aplicando conceptos de lógica y control.

- Configurar y utilizar los sensores de color del EV3 para que el robot pueda identificar diferentes estaciones en el recorrido y responder a ellas de manera adecuada.
- Diseñar acciones autónomas que permitan al robot detenerse, emitir una señal sonora y mostrar mensajes en pantalla según el color detectado.
- Fomentar el trabajo en equipo y la colaboración, distribuyendo roles y responsabilidades entre los integrantes para lograr un desarrollo integral del proyecto.

### ***Actividades de Desarrollo del Proyecto (ABP con LEGO EV3 y Python)***

1. Exploración y contextualización
  - Socialización del proyecto, explicación del circuito, estaciones de colores y metas del robot.
  - Conversatorio sobre aplicaciones reales de robots seguidores de línea en la industria y la vida cotidiana.
  - Formación de equipos de trabajo y asignación de roles (programador, documentador, integrador, evaluador).
2. Diseño inicial del robot
  - Selección de los sensores que se utilizarán (color, táctil, giroscopio, ultrasonido).
  - Construcción del modelo base del robot con LEGO EV3.
  - Discusión en grupo sobre posibles mejoras de diseño (estabilidad, posición de sensores, facilidad de programación).
3. Programación básica en Python
  - Introducción a la conexión del EV3 con Python.
  - Desarrollo de un programa sencillo para seguir una línea negra

sobre fondo blanco con el sensor de color.

- Pruebas iniciales en un pequeño tramo de circuito.
4. Reconocimiento de estaciones
    - Programación para identificar colores diferentes en las “estaciones”.
    - Implementación de las acciones para detenerse, sonar alarma y mostrar un mensaje en pantalla.
    - Ajuste de parámetros de detección para mejorar precisión.
  5. Integración y pruebas en el circuito completo
    - Ensamblaje del código que integra seguimiento de línea más el reconocimiento de estaciones.
    - Pruebas en el recorrido completo.
    - Ajuste fino de programación y calibración de sensores (ejemplo: niveles de luz para el sensor de color, distancias seguras con ultrasonido).
  6. Documentación del proceso
    - Elaboración de las bitácoras de clase y la bitácora de culminación del proyecto.
    - Registro en video o fotos de las fases de construcción y pruebas.
    - Redacción del informe final con objetivos, metodología, resultados y conclusiones.
  7. Socialización y evaluación
    - Presentación del robot en funcionamiento frente a los demás grupos.
    - Explicación de cómo resolvieron el reto, qué dificultades tuvieron y cómo las superaron.
    - Autoevaluación y coevaluación del trabajo en equipo.

## ***Evaluación***

- Rúbricas y bitácoras: 60 %
- Presentación del proyecto: 10 %
- Informe escrito: 15 %
- Autoevaluación y coevaluación: 15 %

## **Proyecto 2: Robot evasor de obstáculos**

### ***Descripción del proyecto***

El proyecto consiste en diseñar y programar un robot autónomo LEGO EV3 en Python, cuyo desafío será explorar de manera inteligente una pista rectangular de 2 m x 2 m, delimitada por bordes negros y poblada con distintos obstáculos como cubos de LEGO, cilindros de PVC o cartón, y pequeñas paredes que simulan muros.

La prueba inicia cuando el robot se coloca en una de las esquinas de la pista y se activa mediante un botón o comando de inicio, y finaliza al cumplirse 3 minutos de exploración o cuando el robot regrese a su zona de partida. Durante el recorrido deberá evitar colisiones, empleando sensores ultrasónicos, giroscópico, de color y táctil, y acompañando cada acción con señales sonoras y mensajes en pantalla, que informen la maniobra ejecutada.

En un escenario de competencia, el desempeño de cada robot será evaluado considerando tres aspectos principales:

- Autonomía o la capacidad para moverse y tomar decisiones sin intervención externa.
- Eficiencia en la evasión de obstáculos o número de choques y correcciones necesarias.
- Cobertura de exploración de la pista o porcentaje del espacio

recorrido sin errores de salida.

El mejor desempeño, corresponderá al robot que logre explorar de manera más amplia y segura el área en el tiempo establecido, mostrando precisión en sus giros, respuestas rápidas a los obstáculos y un control estable de la trayectoria.

## ***Planeación ABP – Proyecto 2***

Título del proyecto: Robot explorador autónomo con evasión de obstáculos.

### ***Objetivo general***

Diseñar, programar y evaluar un robot autónomo LEGO® MINDSTORMS® EV3 capaz de explorar una pista delimitada, detectar, y esquivar obstáculos de diferentes formas y tamaños, empleando sensores y alarmas para garantizar un desempeño eficiente, seguro y competitivo frente a otros robots en condiciones similares.

### ***Objetivos específicos***

Comprender y aplicar los fundamentos de la programación en Python para LEGO EV3, con énfasis en el control de sensores y motores, como base para el desarrollo del robot explorador.

Configurar y calibrar los sensores ultrasónicos, giroscópico, de color y táctil, de modo que permitan al robot percibir el entorno con precisión y reaccionar de manera adecuada ante obstáculos o límites de la pista.

- Diseñar e implementar estrategias de evasión de obstáculos, considerando distintos tipos de objetos (bloques, cilindros,

muros) y asegurando la continuidad del recorrido dentro del espacio delimitado.

- Integrar sistemas de retroalimentación sonora y visual, que comuniquen las decisiones y maniobras del robot durante su operación, mejorando su claridad y atractivo en el contexto de una competencia.
- Evaluar el desempeño del robot en una competencia controlada, aplicando criterios como tiempo de exploración, eficacia en la detección de obstáculos, precisión en la navegación y estabilidad en la trayectoria.

## **Actividades de Desarrollo del Proyecto (ABP con LEGO EV3 y Python)**

### ***Planeación del Proyecto (ABP con LEGO EV3 y Python).***

1. Exploración y contextualización.
  - Presentación del reto: construir un robot autónomo con LEGO EV3 programado en Python para recorrer un circuito, reconocer estaciones y ejecutar acciones específicas.
  - Indagación inicial: los estudiantes investigan sobre robots autónomos, sensores EV3 y aplicaciones en la vida real.
  - Discusión guiada: se formulan preguntas orientadoras (¿Cómo detecta un robot su entorno? ¿Qué significa que un robot sea autónomo?).
  - Conformación de equipos de trabajo y asignación de roles (líder, programador, diseñador mecánico, encargado de pruebas, documentador).
2. Diseño inicial del robot.
  - Lluvia de ideas sobre configuraciones mecánicas y selección

de la más adecuada.

- Construcción inicial del robot en LEGO EV3 (estructura básica con motores y ruedas).
  - Identificación de los sensores a utilizar (color, ultrasónico, giroscopio).
  - Boceto esquemático del robot y de la pista de prueba.
3. Programación básica en Python.
    - Configuración del entorno de programación (instalación de librerías y conexión con EV3).
    - Desarrollo de rutinas básicas: movimiento hacia adelante, giro y frenado.
    - Implementación inicial del seguimiento de línea con sensor de color.
    - Registro de avances en un cuaderno de laboratorio digital o físico.
  4. Reconocimiento de estaciones.
    - Programación del reconocimiento de colores para identificar estaciones en el circuito.
    - Definición de acciones asociadas a cada estación (detenerse, mostrar mensaje en pantalla, emitir sonido).
    - Validación de la precisión del sensor de color en diferentes condiciones de iluminación.
    - Prueba de integración parcial en tramos cortos de la pista.
  5. Integración y pruebas en el circuito completo
    - Ensamblaje final del robot con todos los sensores.
    - Programación de la lógica completa del recorrido (seguir línea + estaciones + acciones).
    - Pruebas de recorrido en la pista delimitada.



- Ajustes de calibración en velocidad, giros y reconocimiento de colores.
- Mini competencia interna entre equipos para comparar desempeño (tiempo de recorrido, cumplimiento de acciones en estaciones, estabilidad).

### ***Documentación del proceso.***

- Registro de cada fase del trabajo: diseño, programación, pruebas y resultados.
- Elaboración de un informe escrito con evidencias gráficas y análisis de dificultades.
- Preparación de un portafolio digital con videos cortos del robot en acción.

### ***Socialización y evaluación.***

- Presentación de resultados por parte de cada equipo (exposición con robot en funcionamiento).
- Debate y retroalimentación grupal: análisis de estrategias de diseño y programación.
- Evaluación del desempeño del robot en términos de:
  - Tiempo de recorrido
  - Precisión en el seguimiento de línea
  - Reconocimiento de estaciones y ejecución de acciones
- Autoevaluación y coevaluación del proceso de aprendizaje de cada estudiante.

### ***Evaluación.***

- Rúbricas y bitácoras: 60 %
- Presentación del proyecto: 10 %
- Informe escrito: 15 %
- Autoevaluación y coevaluación: 15 %

**ANEXO C. HERRAMIENTAS Y RECURSOS DEL PROYECTO “DISEÑO E IMPLEMENTACIÓN DE UN EXPLORADOR AUTÓNOMO CON EL ROBOT LEGO® MINDSTORMS® EV3”**

Finalmente, se presentan las rúbricas par avaluación del estudiante, bitácoras, plantillas y formatos propuestos anteriormente:

**Tabla 3.**  
*Rubrica de evaluación 1*

| <b>Objetivo:</b> Evaluar el programa que realiza el seguimiento de la línea. |                                                                     |                                                           |                                                  |                                                         |                                                      |             |      |
|------------------------------------------------------------------------------|---------------------------------------------------------------------|-----------------------------------------------------------|--------------------------------------------------|---------------------------------------------------------|------------------------------------------------------|-------------|------|
| Criterio                                                                     | Descripción                                                         | Nivel 1 (1-2 puntos)                                      | Nivel 2 (3 puntos)                               | Nivel 3 (4 puntos)                                      | Nivel 4 (5 puntos)                                   | Ponderación | Nota |
| Puntualidad en la entrega                                                    | Entrega del programa dentro del tiempo establecido.                 | No entrega o lo hace fuera del plazo sin justificación.   | Entrega fuera del plazo con justificación.       | Entrega dentro del plazo, pero con retraso menor a 24h. | Entrega en el plazo estipulado.                      | 30 %        |      |
| Funcionalidad del código                                                     | El programa debe permitir que el robot siga la línea correctamente. | El robot no sigue la línea o el código preseneta errores. | El robot sigue la línea de manera inconsistente. | El robot sigue la línea con algunos errores menores.    | El robot sigue la línea de manera precisa y estable. | 25 %        |      |
| Documentación del código                                                     | Uso de comentarios y buenas prácticas de codificación.              | No hay comentarios ni estructura en el código.            | Comentarios insuficientes o poco claros.         | Comentarios adecuados, pero podrían mejorarse.          | Comentarios claros y bien estructurados.             | 15 %        |      |

|                          |                                                                                    |                                                             |                                                        |                                                          |                                                              |      |  |
|--------------------------|------------------------------------------------------------------------------------|-------------------------------------------------------------|--------------------------------------------------------|----------------------------------------------------------|--------------------------------------------------------------|------|--|
| Integración de sensores  | Correcta configuración e integración de los sensores para el seguimiento de línea. | No se integraron sensores o su configuración es incorrecta. | Sensores integrados, pero con errores funcionales.     | Sensores bien integrados con ajustes menores requeridos. | Sensores correctamente integrados y funcionando óptimamente. | 15 % |  |
| Entrega de documentación | Presentación de un informe detallado del desarrollo del programa.                  | No se entrega documentación.                                | Documentación incompleta o con errores significativos. | Documentación adecuada, pero falta detalle.              | Documentación clara, completa y bien estructurada.           | 15 % |  |

**Tabla 4.**  
*Rúbrica de evaluación 2*

| <b>Objetivo:</b> Evaluar la integración entre los programas seguidor de línea y el programa que mueve en espiral al robot. |                                                                                     |                                                           |                                                                       |                                                                 |                                                                                    |             |      |
|----------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|-----------------------------------------------------------|-----------------------------------------------------------------------|-----------------------------------------------------------------|------------------------------------------------------------------------------------|-------------|------|
| Criterio de Evaluación                                                                                                     | Descripción                                                                         | Nivel 1 (1 - 2 puntos)                                    | Nivel 2 (3 puntos)                                                    | Nivel 3 (4 puntos)                                              | Nivel 4 (5 puntos)                                                                 | Ponderación | Nota |
| Puntualidad en la entrega                                                                                                  | Entrega del trabajo en la fecha y hora establecidas.                                | Entrega con más de 3 días de retraso.                     | Entrega con 1-3 días de retraso.                                      | Entrega con hasta 24 horas de retraso.                          | Entrega antes o en la fecha límite sin retrasos.                                   | 20 %        |      |
| Integración funcional de los programas                                                                                     | Capacidad de combinar correctamente el seguidor de línea y el movimiento en espiral | La integración no funciona o falla la mayoría del tiempo. | La integración presenta errores ocasionales que afectan el desempeño. | La integración funciona con ligeros retrasos o ajustes menores. | La integración es completa, fluida y sin fallos; el cambio entre modos es preciso. | 15 %        |      |

|                                      |                                                                                       |                                                        |                                                                  |                                                           |                                                                                               |      |  |
|--------------------------------------|---------------------------------------------------------------------------------------|--------------------------------------------------------|------------------------------------------------------------------|-----------------------------------------------------------|-----------------------------------------------------------------------------------------------|------|--|
|                                      | en una sola ejecución sin errores.                                                    |                                                        |                                                                  |                                                           |                                                                                               |      |  |
| Transición entre modos de movimiento | Suavidad y precisión al pasar de seguir la línea a realizar el movimiento en espiral. | La transición es lenta, imprecisa o no se logra.       | La transición presenta interrupciones notorias.                  | La transición es mayormente suave, con ligeros retrasos.  | La transición es inmediata, precisa y sin pérdida de control.                                 | 10 % |  |
| Funcionalidad del código             | Correcta ejecución de instrucciones y ausencia de errores.                            | El código no cumple la función o falla constantemente. | El código presenta errores que requieren intervención frecuente. | El código funciona con pequeños ajustes necesarios.       | El código ejecuta todas las funciones sin errores y con alta eficiencia.                      | 15 % |  |
| Documentación del código             | Claridad y detalle en la explicación del funcionamiento del programa.                 | No se presenta documentación.                          | Documentación incompleta o poco clara.                           | Documentación buena, pero con detalles menores faltantes. | Documentación completa con explicación clara de funciones, variables y lógica de integración. | 15 % |  |
| Uso e integración de sensores        | Correcta utilización de sensores para ambas funciones.                                | Sensores no utilizados o configurados incorrectamente. | Sensores mal calibrados o parcialmente funcionales.              | Sensores bien integrados, con fallos menores.             | Sensores correctamente configurados y usados en ambas funciones sin fallos.                   | 10 % |  |

|                                               |                                                                       |                              |                                    |                                                            |                                                                                                   |      |  |
|-----------------------------------------------|-----------------------------------------------------------------------|------------------------------|------------------------------------|------------------------------------------------------------|---------------------------------------------------------------------------------------------------|------|--|
| Presentación de la documentación del proyecto | Entrega de un informe final con formato claro, organizado y completo. | No se entrega documentación. | Documento incompleto o desordenado | Documento claro, pero con ausencia de uno o dos apartados. | Documento bien estructurado, con portada, objetivos, descripción técnica, pruebas y conclusiones. | 15 % |  |
|-----------------------------------------------|-----------------------------------------------------------------------|------------------------------|------------------------------------|------------------------------------------------------------|---------------------------------------------------------------------------------------------------|------|--|

**Tabla 5.**  
*Rubrica de evaluación 3*

| <b>Objetivo:</b> Evaluar el programa que realiza la evasión de obstáculos y su integración con el seguimiento de la línea. |                                                                              |                                                                     |                                                                        |                                                                           |                                                                                       |             |      |
|----------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|---------------------------------------------------------------------|------------------------------------------------------------------------|---------------------------------------------------------------------------|---------------------------------------------------------------------------------------|-------------|------|
| Criterio de Evaluación                                                                                                     | Descripción                                                                  | Nivel 1 (1 - 2 puntos)                                              | Nivel 2 (3 puntos)                                                     | Nivel 3 (4 puntos)                                                        | Nivel 4 (5 puntos)                                                                    | Ponderación | Nota |
| Puntualidad en la entrega                                                                                                  | Evalúa si el programa se entrega en la fecha establecida.                    | Entrega con más de dos días de retraso o no se entrega.             | Entrega con hasta dos días de retraso.                                 | Entrega con un día de retraso.                                            | Entrega antes o en la fecha estipulada, sin retrasos.                                 | 30%         |      |
| Funcionalidad del programa de evasión de obstáculos                                                                        | Mide si el robot detecta y esquiva obstáculos correctamente en su recorrido. | El robot no detecta o no logra evitar los obstáculos correctamente. | El robot detecta obstáculos, pero la evasión es incompleta o errática. | El robot detecta y evita la mayoría de los obstáculos, con pocos errores. | El robot detecta y evita todos los obstáculos de forma precisa y fluida, sin errores. | 15%         |      |

|                                                         |                                                                                                                                                            |                                                                                                                       |                                                                                                                                    |                                                                                                                      |                                                                                                                                                                                                 |     |  |
|---------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|--|
| Integra-<br>ción con<br>el segui-<br>miento<br>de línea | Evalúa<br>si el pro-<br>grama de<br>evasión fun-<br>ciona de<br>manera coor-<br>dina-<br>da con el<br>seguidor de<br>línea.                                | Integra-<br>ción defi-<br>ciente: el<br>robot no<br>retoma<br>la línea o<br>se pierde<br>después<br>de la<br>evasión. | Inte-<br>gración<br>parcial:<br>el robot<br>sigue<br>la línea<br>tras la<br>evasión,<br>pero con<br>dificul-<br>tades<br>notorias. | Inte-<br>gración<br>buena:<br>retoma<br>el segui-<br>miento<br>con pe-<br>queñas<br>correc-<br>ciones<br>necesarias. | Inte-<br>gración<br>perfecta:<br>el robot<br>retoma<br>el segui-<br>miento<br>de la<br>línea inmedia-<br>tamente<br>después<br>de es-<br>quivar ob-<br>stácu-<br>los, sin<br>desvia-<br>ciones. | 15% |  |
| Uso e<br>integra-<br>ción de<br>sensores                | Evalúa la<br>correcta<br>utiliza-<br>ción y com-<br>bina-<br>ción de<br>sensores<br>(color,<br>ultra-<br>sonido,<br>etc.) para<br>ambas<br>funcio-<br>nes. | Sensores<br>mal con-<br>figurados<br>o sin in-<br>tegración<br>efectiva.                                              | Sensores<br>funcio-<br>nales,<br>pero con<br>errores<br>frecuen-<br>tes de<br>lectura.                                             | Sensores<br>bien inte-<br>grados,<br>con pe-<br>queñas<br>impreci-<br>siones.                                        | Sensores<br>configu-<br>rados y<br>calibra-<br>dos de<br>forma<br>óptima;<br>inte-<br>gración<br>precisa y<br>sin fallos.                                                                       | 10% |  |
| Docu-<br>menta-<br>ción del<br>código                   | Revisa la<br>claridad,<br>estruc-<br>tura y<br>comen-<br>tarios del<br>código<br>fuente.                                                                   | Códi-<br>go sin<br>comen-<br>tarios<br>o con<br>desor-<br>den que<br>dificulta<br>su com-<br>prensión.                | Código<br>con<br>pocos<br>comen-<br>tarios o<br>explica-<br>ciones<br>poco<br>claras.                                              | Código<br>estructu-<br>rado, con<br>la ma-<br>yoría de<br>las fun-<br>ciones<br>comenta-<br>das.                     | Código<br>clara-<br>mente<br>estructu-<br>rado, con<br>comen-<br>tarios<br>útiles y<br>consis-<br>tentes<br>que<br>explican<br>la lógica.                                                       | 10% |  |

|                                       |                                                                                      |                                         |                                             |                                                                             |                                                                                 |     |  |
|---------------------------------------|--------------------------------------------------------------------------------------|-----------------------------------------|---------------------------------------------|-----------------------------------------------------------------------------|---------------------------------------------------------------------------------|-----|--|
| Entrega de documentación del proyecto | Evalúa si se entrega el documento técnico del proyecto con la información requerida. | Documentación ausente o muy deficiente. | Documentación incompleta o poco organizada. | Documentación completa, pero con pequeños detalles de forma o presentación. | Documentación completa, bien organizada, clara y acorde a las pautas del curso. | 20% |  |
|---------------------------------------|--------------------------------------------------------------------------------------|-----------------------------------------|---------------------------------------------|-----------------------------------------------------------------------------|---------------------------------------------------------------------------------|-----|--|

**Tabla 6.**  
*Rúbrica de evaluación 4*

| <b>Objetivo:</b> Evaluar la entrega del proyecto operando de acuerdo con las especificaciones que se indicaron al inicio del proceso. |                                                                              |                                                              |                                                            |                                                         |                                                                             |             |      |
|---------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|--------------------------------------------------------------|------------------------------------------------------------|---------------------------------------------------------|-----------------------------------------------------------------------------|-------------|------|
| Criterio de Evaluación                                                                                                                | Descripción                                                                  | Nivel 1 (1 - 2 puntos)                                       | Nivel 2 (3 puntos)                                         | Nivel 3 (4 puntos)                                      | Nivel 4 (5 puntos)                                                          | Ponderación | Nota |
| Puntualidad en la entrega                                                                                                             | Entrega del proyecto dentro del tiempo estipulado.                           | Entregado con retraso mayor a 24 horas o no entregado.       | Entregado con retraso de hasta 24 horas.                   | Entregado el mismo día con retraso menor a 2 horas.     | Entregado en la fecha y hora establecidas sin retrasos.                     | 20 %        |      |
| Seguimiento de línea inicial                                                                                                          | Capacidad de seguir la línea negra con curvas de 20°, 50° y 90° sin salirse. | No logra completar la fase o pierde la línea constantemente. | Completa con más de 2 salidas, pero logra llegar al final. | Completa el recorrido con 1 salida y rápida corrección. | Completa el recorrido sin salidas de la línea y mantiene velocidad estable. | 10 %        |      |



|                                    |                                                                                      |                                                             |                                                                     |                                                                      |                                                                                  |      |  |
|------------------------------------|--------------------------------------------------------------------------------------|-------------------------------------------------------------|---------------------------------------------------------------------|----------------------------------------------------------------------|----------------------------------------------------------------------------------|------|--|
| Evasión de obstáculos              | Capacidad de detectar y esquivar obstáculos en el segundo ambiente.                  | No logra esquivar y queda detenido o choca.                 | Esquiva parcialmente, con varias colisiones o pérdida de dirección. | Esquiva la mayoría con 1-2 colisiones leves.                         | Detecta y esquiva todos los obstáculos sin colisiones ni errores de trayectoria. | 10 % |  |
| Reintegración a la línea           | Capacidad de encontrar nuevamente la línea negra tras esquivar obstáculos.           | No logra encontrar la línea o sigue trayectoria incorrecta. | Encuentra la línea tras más de 10 s o con varias correcciones.      | Encuentra y retoma la línea en menos de 10 s, con ligera desviación. | Encuentra y retoma la línea en menos de 5 s, sin errores.                        | 10 % |  |
| Retorno al punto de partida        | Ejecución correcta de la trayectoria inversa hasta el inicio.                        | No completa el recorrido inverso.                           | Completa con múltiples salidas y correcciones.                      | Completa con 1 salida leve y rápida corrección.                      | Completa el recorrido inverso sin salidas ni errores.                            | 10 % |  |
| Movimiento en espiral              | Ejecución de un patrón en espiral al finalizar la ruta.                              | No realiza el movimiento en espiral.                        | Espiral incompleta o con pérdida de forma.                          | Espiral completa con ligeras irregularidades.                        | Espiral completa, simétrico y sin interrupciones.                                | 10 % |  |
| Calidad de integración del sistema | Funcionamiento coordinado de todas las fases del programa sin errores de transición. | Transiciones fallidas que impiden completar la tarea.       | Transiciones con fallos que afectan el tiempo total.                | Transiciones con ligeros retrasos, pero sin errores funcionales.     | Transiciones fluidas entre todas las fases, sin fallos.                          | 15 % |  |

|                                       |                                                                                      |                                         |                                             |                                                                             |                                                                                 |      |  |
|---------------------------------------|--------------------------------------------------------------------------------------|-----------------------------------------|---------------------------------------------|-----------------------------------------------------------------------------|---------------------------------------------------------------------------------|------|--|
| Entrega de documentación del proyecto | Evalúa si se entrega el documento técnico del proyecto con la información requerida. | Documentación ausente o muy deficiente. | Documentación incompleta o poco organizada. | Documentación completa, pero con pequeños detalles de forma o presentación. | Documentación completa, bien organizada, clara y acorde a las pautas del curso. | 15 % |  |
|---------------------------------------|--------------------------------------------------------------------------------------|-----------------------------------------|---------------------------------------------|-----------------------------------------------------------------------------|---------------------------------------------------------------------------------|------|--|

**Tabla 7.**  
*Bitácora de clase*

| Bitácora de clase                    |  |  |  |  |  |  |  |  |  |                        |  |  |  |  |  |  |  |  |  |
|--------------------------------------|--|--|--|--|--|--|--|--|--|------------------------|--|--|--|--|--|--|--|--|--|
| Fecha _____                          |  |  |  |  |  |  |  |  |  | Bitácora número: _____ |  |  |  |  |  |  |  |  |  |
| Nombre del equipo _____              |  |  |  |  |  |  |  |  |  |                        |  |  |  |  |  |  |  |  |  |
| Responsable _____                    |  |  |  |  |  |  |  |  |  |                        |  |  |  |  |  |  |  |  |  |
| Actividad realizada _____            |  |  |  |  |  |  |  |  |  |                        |  |  |  |  |  |  |  |  |  |
|                                      |  |  |  |  |  |  |  |  |  |                        |  |  |  |  |  |  |  |  |  |
| Recursos utilizados                  |  |  |  |  |  |  |  |  |  |                        |  |  |  |  |  |  |  |  |  |
|                                      |  |  |  |  |  |  |  |  |  |                        |  |  |  |  |  |  |  |  |  |
|                                      |  |  |  |  |  |  |  |  |  |                        |  |  |  |  |  |  |  |  |  |
|                                      |  |  |  |  |  |  |  |  |  |                        |  |  |  |  |  |  |  |  |  |
|                                      |  |  |  |  |  |  |  |  |  |                        |  |  |  |  |  |  |  |  |  |
| Descripción de la actividad de clase |  |  |  |  |  |  |  |  |  |                        |  |  |  |  |  |  |  |  |  |
|                                      |  |  |  |  |  |  |  |  |  |                        |  |  |  |  |  |  |  |  |  |
|                                      |  |  |  |  |  |  |  |  |  |                        |  |  |  |  |  |  |  |  |  |
|                                      |  |  |  |  |  |  |  |  |  |                        |  |  |  |  |  |  |  |  |  |
|                                      |  |  |  |  |  |  |  |  |  |                        |  |  |  |  |  |  |  |  |  |
|                                      |  |  |  |  |  |  |  |  |  |                        |  |  |  |  |  |  |  |  |  |
| Dificultades encontradas             |  |  |  |  |  |  |  |  |  |                        |  |  |  |  |  |  |  |  |  |
|                                      |  |  |  |  |  |  |  |  |  |                        |  |  |  |  |  |  |  |  |  |
|                                      |  |  |  |  |  |  |  |  |  |                        |  |  |  |  |  |  |  |  |  |
|                                      |  |  |  |  |  |  |  |  |  |                        |  |  |  |  |  |  |  |  |  |
|                                      |  |  |  |  |  |  |  |  |  |                        |  |  |  |  |  |  |  |  |  |
|                                      |  |  |  |  |  |  |  |  |  |                        |  |  |  |  |  |  |  |  |  |
| Solución aplicada                    |  |  |  |  |  |  |  |  |  |                        |  |  |  |  |  |  |  |  |  |
|                                      |  |  |  |  |  |  |  |  |  |                        |  |  |  |  |  |  |  |  |  |
|                                      |  |  |  |  |  |  |  |  |  |                        |  |  |  |  |  |  |  |  |  |
|                                      |  |  |  |  |  |  |  |  |  |                        |  |  |  |  |  |  |  |  |  |
|                                      |  |  |  |  |  |  |  |  |  |                        |  |  |  |  |  |  |  |  |  |
| Observaciones                        |  |  |  |  |  |  |  |  |  |                        |  |  |  |  |  |  |  |  |  |
|                                      |  |  |  |  |  |  |  |  |  |                        |  |  |  |  |  |  |  |  |  |
|                                      |  |  |  |  |  |  |  |  |  |                        |  |  |  |  |  |  |  |  |  |
|                                      |  |  |  |  |  |  |  |  |  |                        |  |  |  |  |  |  |  |  |  |
|                                      |  |  |  |  |  |  |  |  |  |                        |  |  |  |  |  |  |  |  |  |
|                                      |  |  |  |  |  |  |  |  |  |                        |  |  |  |  |  |  |  |  |  |

Tabla 8.  
Bitácora de culminación de fase de proyecto

| Bitácora de culminación etapa del proyecto                                                                                                                                       |  |  |  |  |                  |  |  |  |  |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|--|--|--|------------------|--|--|--|--|
| Fecha                                                                                                                                                                            |  |  |  |  | Bitácora número: |  |  |  |  |
| Nombre del equipo                                                                                                                                                                |  |  |  |  |                  |  |  |  |  |
| Responsable                                                                                                                                                                      |  |  |  |  |                  |  |  |  |  |
| Nombre de la etapa culminada                                                                                                                                                     |  |  |  |  |                  |  |  |  |  |
| Estado final de la etapa                                                                                                                                                         |  |  |  |  |                  |  |  |  |  |
| Culminada <input type="checkbox"/> No culminada <input type="checkbox"/> Suspendida <input type="checkbox"/> No requerida <input type="checkbox"/> Otro <input type="checkbox"/> |  |  |  |  |                  |  |  |  |  |
| Objetivo de la etapa                                                                                                                                                             |  |  |  |  |                  |  |  |  |  |
|                                                                                                                                                                                  |  |  |  |  |                  |  |  |  |  |
| Actividades realizada para la culminación de la etapa                                                                                                                            |  |  |  |  |                  |  |  |  |  |
|                                                                                                                                                                                  |  |  |  |  |                  |  |  |  |  |
| Dificultades encontradas                                                                                                                                                         |  |  |  |  |                  |  |  |  |  |
|                                                                                                                                                                                  |  |  |  |  |                  |  |  |  |  |
| Soluciones aplicadas                                                                                                                                                             |  |  |  |  |                  |  |  |  |  |
|                                                                                                                                                                                  |  |  |  |  |                  |  |  |  |  |
| Observaciones                                                                                                                                                                    |  |  |  |  |                  |  |  |  |  |
|                                                                                                                                                                                  |  |  |  |  |                  |  |  |  |  |

**Tabla 9.**  
*Formato de coevaluación*

| Formato de Coevaluación         |                                                                                            |               |
|---------------------------------|--------------------------------------------------------------------------------------------|---------------|
| Nombre del proyecto             |                                                                                            |               |
| Nombre del equipo               |                                                                                            |               |
| Nombre del evaluador            |                                                                                            |               |
| Escala de la evaluación:        | 1 - Deficiente                                                                             | 2 - Aceptable |
|                                 | 4 - Muy bueno                                                                              | 3 - Bueno     |
|                                 |                                                                                            | 5 - Excelente |
| Criterio de evaluación          | Criterio de evaluación                                                                     | Escala (1-5)  |
| Participación activa            | El compañero(a) trabajó en la construcción del robot, programación y pruebas.              |               |
| Observaciones                   |                                                                                            |               |
| Responsabilidad                 | Cumplió con sus compromisos y respetó los tiempos de entrega.                              |               |
| Observaciones                   |                                                                                            |               |
| Contribución técnica            | Aportó significativamente en programación, integración de sensores o pruebas.              |               |
| Observaciones                   |                                                                                            |               |
| Calidad de los aportes escritos | Colaboró en la memoria y en la redacción del informe.                                      |               |
| Observaciones                   |                                                                                            |               |
| Claridad en la comunicación     | Expresó sus ideas de manera clara, respetuosa y constructiva.                              |               |
| Observaciones                   |                                                                                            |               |
| Colaboración y apoyo            | Escuchó y apoyó a los demás miembros del equipo en la                                      |               |
| Observaciones                   |                                                                                            |               |
| Resolución de conflictos        | Mantuvo una actitud conciliadora ante diferencias o dificultades.                          |               |
| Observaciones                   |                                                                                            |               |
| Actitud y disposición           | Mostró motivación, iniciativa y compromiso en todas las fases del proyecto.                |               |
| Observaciones                   |                                                                                            |               |
| Creatividad e innovación        | Aportó ideas originales que enriquecieron el desarrollo del proyecto.                      |               |
| Observaciones                   |                                                                                            |               |
| Liderazgo compartido            | Ejerció liderazgo positivo, promoviendo la organización y la toma de decisiones en equipo. |               |
| Observaciones                   |                                                                                            |               |

**Tabla 10.**  
*Formato de autoevaluación*

| Formato de Autoevaluación                     |                                                                                                      |              |
|-----------------------------------------------|------------------------------------------------------------------------------------------------------|--------------|
| Nombre del proyecto                           |                                                                                                      |              |
| Nombre del equipo                             |                                                                                                      |              |
| Nombre del estudiante                         |                                                                                                      |              |
| Escala de la evaluación:                      | 1 - Deficiente    2 - Aceptable    3 - Bueno<br>4 - Muy bueno    5 - Excelente                       |              |
| Criterio de evaluación                        | Criterio de evaluación                                                                               | Escala (1-5) |
| Cumplimiento de las especificaciones técnicas | Mi robot logró seguir la línea, evadir obstáculos y realizar el movimiento en espiral.               |              |
| Observaciones                                 |                                                                                                      |              |
| Programación en Python                        | El código que desarrollé funciona correctamente, está comentado y organizado.                        |              |
| Observaciones                                 |                                                                                                      |              |
| Integración de sensores y actuadores          | Utilicé adecuadamente los sensores y motores garantizando la funcionalidad del sistema.              |              |
| Observaciones                                 |                                                                                                      |              |
| Calidad de documentación escrita              | Entregué la memoria final clara, estructurada y completa.                                            |              |
| Observaciones                                 |                                                                                                      |              |
| Presentación del trabajo                      | El informe y la entrega del proyecto fueron presentados de forma ordenada y profesional.             |              |
| Observaciones                                 |                                                                                                      |              |
| Trabajo en equipo                             | Aporté activamente en la construcción, programación y documentación, colaborando con mis compañeros. |              |
| Observaciones                                 |                                                                                                      |              |
| Resolución de problemas                       | Fui capaz de identificar y solucionar fallos durante el desarrollo del proyecto.                     |              |
| Observaciones                                 |                                                                                                      |              |
| Creatividad e innovación                      | Propuse ideas originales y mejoras adicionales al proyecto más allá de lo solicitado.                |              |
| Observaciones                                 |                                                                                                      |              |
| Gestión del tiempo y recursos                 | Cumplí con los plazos y aproveché eficientemente los recursos disponibles.                           |              |
| Observaciones                                 |                                                                                                      |              |
| Reflexión personal                            | Identifiqué qué aprendí, fortalezas y aspectos a mejorar en futuros trabajos.                        |              |
| Observaciones                                 |                                                                                                      |              |

**Tabla 11.**  
*Formato para el informe final del proyecto*

| Plantilla general del contenido en el informe final                                                                                                                                                                    |  |  |  |  |  |  |  |  |  |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|--|--|--|--|--|--|--|--|
| 1. Portada                                                                                                                                                                                                             |  |  |  |  |  |  |  |  |  |
| Título del proyecto.                                                                                                                                                                                                   |  |  |  |  |  |  |  |  |  |
| Nombre de los estudiantes.                                                                                                                                                                                             |  |  |  |  |  |  |  |  |  |
| Nombre del curso y docente.                                                                                                                                                                                            |  |  |  |  |  |  |  |  |  |
| 2. Resumen                                                                                                                                                                                                             |  |  |  |  |  |  |  |  |  |
| Breve descripción del proyecto (máx. 200 palabras), resaltando:                                                                                                                                                        |  |  |  |  |  |  |  |  |  |
| Objetivo general.                                                                                                                                                                                                      |  |  |  |  |  |  |  |  |  |
| Metodología aplicada.                                                                                                                                                                                                  |  |  |  |  |  |  |  |  |  |
| Resultados obtenidos.                                                                                                                                                                                                  |  |  |  |  |  |  |  |  |  |
| Conclusiones principales.                                                                                                                                                                                              |  |  |  |  |  |  |  |  |  |
| 3. Introducción                                                                                                                                                                                                        |  |  |  |  |  |  |  |  |  |
| Contexto del proyecto.                                                                                                                                                                                                 |  |  |  |  |  |  |  |  |  |
| Importancia del desarrollo de sistemas autónomos en robótica.                                                                                                                                                          |  |  |  |  |  |  |  |  |  |
| Justificación del trabajo.                                                                                                                                                                                             |  |  |  |  |  |  |  |  |  |
| 4. Objetivos                                                                                                                                                                                                           |  |  |  |  |  |  |  |  |  |
| Objetivo General: Desarrollar habilidades en programación mediante el desarrollo de un sistema de navegación autónomo que utiliza el robot LEGO® MINDSTORMS® EV3 como vehículo y Python como lenguaje de programación. |  |  |  |  |  |  |  |  |  |
| 4.1 Objetivos Específicos:                                                                                                                                                                                             |  |  |  |  |  |  |  |  |  |
| · Implementar secuencias de movimiento utilizando bucles y estructuras condicionales en Python.                                                                                                                        |  |  |  |  |  |  |  |  |  |
| · Diseñar algoritmos para trayectorias específicas como recorridos en cuadrados, espirales y movimientos aleatorios.                                                                                                   |  |  |  |  |  |  |  |  |  |
| · Aplicar principios de optimización en la programación de trayectorias.                                                                                                                                               |  |  |  |  |  |  |  |  |  |
| · Integrar sensores para mejorar la autonomía del robot.                                                                                                                                                               |  |  |  |  |  |  |  |  |  |
| 5. Metodología                                                                                                                                                                                                         |  |  |  |  |  |  |  |  |  |
| Descripción del enfoque ABP (Aprendizaje Basado en Proyectos).                                                                                                                                                         |  |  |  |  |  |  |  |  |  |
| Fases del proyecto:                                                                                                                                                                                                    |  |  |  |  |  |  |  |  |  |
| 1. Planeación y diseño.                                                                                                                                                                                                |  |  |  |  |  |  |  |  |  |
| 2. Programación y pruebas unitarias (seguidor de línea, espiral, evasión de obstáculos).                                                                                                                               |  |  |  |  |  |  |  |  |  |
| 3. Integración de sistemas.                                                                                                                                                                                            |  |  |  |  |  |  |  |  |  |
| 4. Validación en pista de prueba.                                                                                                                                                                                      |  |  |  |  |  |  |  |  |  |
| 5. Entrega y socialización.                                                                                                                                                                                            |  |  |  |  |  |  |  |  |  |
| Herramientas utilizadas: LEGO EV3, sensores (color, ultrasonido), software de programación en Python.                                                                                                                  |  |  |  |  |  |  |  |  |  |
| 6. Desarrollo y Resultados                                                                                                                                                                                             |  |  |  |  |  |  |  |  |  |
| Explicación del proceso paso a paso.                                                                                                                                                                                   |  |  |  |  |  |  |  |  |  |
| Evidencias gráficas (fotos, diagramas de flujo, pseudocódigo, capturas de pantalla).                                                                                                                                   |  |  |  |  |  |  |  |  |  |
| Resultados obtenidos:                                                                                                                                                                                                  |  |  |  |  |  |  |  |  |  |
| Funcionamiento del seguidor de línea.                                                                                                                                                                                  |  |  |  |  |  |  |  |  |  |
| Evasión de obstáculos.                                                                                                                                                                                                 |  |  |  |  |  |  |  |  |  |
| Retorno al recorrido inicial.                                                                                                                                                                                          |  |  |  |  |  |  |  |  |  |
| Movimiento en espiral.                                                                                                                                                                                                 |  |  |  |  |  |  |  |  |  |
| Análisis de dificultades presentadas y cómo se resolvieron.                                                                                                                                                            |  |  |  |  |  |  |  |  |  |
| 7. Discusión y Análisis Crítico                                                                                                                                                                                        |  |  |  |  |  |  |  |  |  |
| Fortalezas del sistema desarrollado.                                                                                                                                                                                   |  |  |  |  |  |  |  |  |  |
| Limitaciones encontradas.                                                                                                                                                                                              |  |  |  |  |  |  |  |  |  |
| Posibles mejoras en versiones futuras.                                                                                                                                                                                 |  |  |  |  |  |  |  |  |  |
| 8. Conclusiones                                                                                                                                                                                                        |  |  |  |  |  |  |  |  |  |
| Cumplimiento de los objetivos planteados.                                                                                                                                                                              |  |  |  |  |  |  |  |  |  |
| Aprendizajes adquiridos por el equipo.                                                                                                                                                                                 |  |  |  |  |  |  |  |  |  |
| Impacto del proyecto en la formación en robótica y programación.                                                                                                                                                       |  |  |  |  |  |  |  |  |  |
| 9. Recomendaciones                                                                                                                                                                                                     |  |  |  |  |  |  |  |  |  |
| Estrategias de mejora en el código y la integración de sensores.                                                                                                                                                       |  |  |  |  |  |  |  |  |  |
| Propuestas de ampliación (ejemplo: navegación en entornos más complejos, implementación de inteligencia artificial básica).                                                                                            |  |  |  |  |  |  |  |  |  |
| 10. Referencias Bibliográficas                                                                                                                                                                                         |  |  |  |  |  |  |  |  |  |
| Formato APA o el que la institución requiera.                                                                                                                                                                          |  |  |  |  |  |  |  |  |  |
| 11. Anexos                                                                                                                                                                                                             |  |  |  |  |  |  |  |  |  |
| Código fuente.                                                                                                                                                                                                         |  |  |  |  |  |  |  |  |  |
| Diagramas de conexión de sensores.                                                                                                                                                                                     |  |  |  |  |  |  |  |  |  |
| Evidencias fotográficas o de pruebas.                                                                                                                                                                                  |  |  |  |  |  |  |  |  |  |

**Tabla 12.**  
 Cronograma de presentación del proyecto

| Pasos  | Actividad                                                                                                                                                                                     | Semana 1 | Semana 2 | Semana 3 | Semana 4 | Semana 5 | Semana 6 | Semana 7 | Semana 8 | Semana 9 | Semana 10 | Semana 11 | Semana 12 | Semana 13 | Semana 14 | Semana 15 | Semana 16 |
|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|----------|----------|----------|----------|----------|----------|----------|----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|
| 1, 2   | Presentación del proyecto, conformación de grupos y lluvia de ideas inicial                                                                                                                   |          |          |          |          |          |          |          |          |          |           |           |           |           |           |           |           |
| 3,4,5  | Presentación del Cronograma por parte de cada grupo, para el desarrollo de las diferentes actividades que lleva el proyecto. Definición de las necesidades de conocimiento.                   |          |          |          |          |          |          |          |          |          |           |           |           |           |           |           |           |
|        | Búsqueda de información y presentación por parte del docente de los conceptos que fueran necesarios para que el estudiante desarrolle las actividades requeridas en la ejecución del proyecto |          |          |          |          |          |          |          |          |          |           |           |           |           |           |           |           |
| 6      | Desarrollo del programa que permita seguir la línea al robot LEGO                                                                                                                             |          |          |          |          |          |          |          |          |          |           |           |           |           |           |           |           |
|        | Evaluación del programa que realiza el seguimiento de la línea.                                                                                                                               |          |          |          |          |          |          |          |          |          |           |           |           |           |           |           |           |
|        | Desarrollo del programa que permita ejecutar un movimiento en espiral al robot LEGO                                                                                                           |          |          |          |          |          |          |          |          |          |           |           |           |           |           |           |           |
|        | Integración del programa seguidor de línea con el programa que mueve en espiral al robot.                                                                                                     |          |          |          |          |          |          |          |          |          |           |           |           |           |           |           |           |
|        | Evaluación de la integración entre los programas seguidor de línea y el programa que mueve en espiral al robot.                                                                               |          |          |          |          |          |          |          |          |          |           |           |           |           |           |           |           |
|        | Desarrollo del programa que permita evadir obstáculos al robot LEGO                                                                                                                           |          |          |          |          |          |          |          |          |          |           |           |           |           |           |           |           |
|        | Evaluación del programa que realiza la evasión de obstáculos y su integración con el seguimiento de la línea.                                                                                 |          |          |          |          |          |          |          |          |          |           |           |           |           |           |           |           |
|        | Pruebas y ajustes en la operación del robot sobre la pista final que se usará en la prueba                                                                                                    |          |          |          |          |          |          |          |          |          |           |           |           |           |           |           |           |
|        | Entrega del proyecto operando de acuerdo con las especificaciones que se indicaron al inicio                                                                                                  |          |          |          |          |          |          |          |          |          |           |           |           |           |           |           |           |
| 7      | Reunión con los estudiantes donde se realice una retrospectiva y se concluya sobre el desarrollo del proyecto.                                                                                |          |          |          |          |          |          |          |          |          |           |           |           |           |           |           |           |
| 8      | Presentación pública del proyecto mediante una competencia. En dicho evento, se premiará a los dos sistemas que mayor eficiencia presenten. (Recorrido de la pista vs tiempo)                 |          |          |          |          |          |          |          |          |          |           |           |           |           |           |           |           |
| 9      | Retroalimentación por parte de jurados externos invitados al evento.                                                                                                                          |          |          |          |          |          |          |          |          |          |           |           |           |           |           |           |           |
| Paso 1 | Generación de ideas                                                                                                                                                                           |          |          |          |          |          |          |          |          |          |           |           |           |           |           |           |           |
| Paso 2 | Necesidades cognitivas del alumno                                                                                                                                                             |          |          |          |          |          |          |          |          |          |           |           |           |           |           |           |           |
| Paso 3 | Búsqueda de información                                                                                                                                                                       |          |          |          |          |          |          |          |          |          |           |           |           |           |           |           |           |
| Paso 4 | Análisis y síntesis                                                                                                                                                                           |          |          |          |          |          |          |          |          |          |           |           |           |           |           |           |           |
| Paso 5 | Gestión/ Planeación por parte de los estudiantes de sus actividades y tiempos .                                                                                                               |          |          |          |          |          |          |          |          |          |           |           |           |           |           |           |           |
| Paso 6 | Actuar                                                                                                                                                                                        |          |          |          |          |          |          |          |          |          |           |           |           |           |           |           |           |
| Paso 7 | Conclusiones/perspectivas                                                                                                                                                                     |          |          |          |          |          |          |          |          |          |           |           |           |           |           |           |           |
| Paso 8 | Publicación/ Presentación pública del proyecto                                                                                                                                                |          |          |          |          |          |          |          |          |          |           |           |           |           |           |           |           |
| Paso 9 | Retroalimentación                                                                                                                                                                             |          |          |          |          |          |          |          |          |          |           |           |           |           |           |           |           |



## Bibliografía

- Aliane, N., & Bemposta, S. (2008). Una experiencia de aprendizaje basado en proyectos en una asignatura de robótica. *IEEE-RITA*, 3(2), 71–76.
- Botelho, S. S. C., Braz, L. G., & Rodrigues, R. N. (2012). Exploring creativity and sociability with an accessible educational robotic kit. *RiE*, 55–60. <http://www.silviacb.c3.furg.br>
- Bradski, G., & Kaehler, A. (2008). *Learning OpenCV: Computer vision with the OpenCV library*. O'Reilly Media. <https://books.google.com.co/books?hl=es&lr=&id=seAgiOfu2EIC>
- Bueno, P. M., Victoria, Y., & Fitzgerald, L. (2004). Aprendizaje basado en problemas. *Theoria*, 13, 145–157.
- Chavarría, M., & Saldaño, A. (2010). La robótica educativa como una innovativa interfaz educativa entre el alumno y una situación-problema. *Didasc@lia: Didáctica y Educación*, 2, 1–12.
- Coria Arreola, J. M. (2010). Aprendizaje por proyectos: Una metodología diferente. *Revista e-FORMADORES*.
- Cruz, J. A. G., Isusqui, J. C. P., Alvarez, V. M., Inga, C. V., Silva, M. O. C., & Villavicencio, I. E. S. (2023). Desafíos del aula invertida para la educación universitaria en América Latina. Editorial Mar Caribe de Josefrank. <https://doi.org/10.31219/osf.io/teqkh>
- da Silva Filgueira, M. G., & González González, C. S. (2017). PequeBot: Propuesta de un sistema ludificado de robótica educativa para la educación infantil. <http://riull.ull.es/xmlui/handle/915/6677>
- Downey, A. B. (2012). *Think Python (Vol. 1)*. O'Reilly Media.

<https://books.google.com.co/books?hl=es&lr=&id=1mZtP9H6OMQC>

Echeverría, V. L. (2022). Motivados por la robótica. *Campo de Prácticas*, 1(2), 69–81.

Espejo Leupin, R. M. (2016). ¿Pedagogía activa o métodos activos? El caso del aprendizaje activo en la universidad. *Revista Digital de Investigación en Docencia Universitaria*, 16–27. <https://doi.org/10.19083/ridu.10.456>

Estruch, V., & Silva, J. (2006). Aprendizaje basado en proyectos en la carrera de Ingeniería Informática. *Closas-Orcoyen*.

Fernández Martínez, M., García Sánchez, J.-N., Fuertes, A. de C., Redondo Fidalgo, R., & Arias Gundín, O. (2006). El aprendizaje basado en problemas: Revisión de estudios empíricos internacionales. *Revista de Educación*, 341, 397–418.

García, J. M. (2015). Robótica educativa: La programación como parte de un proceso educativo. *RED. Revista de Educación a Distancia*, 46. <http://www.um.es/ead/red/46>

García, R., Traver, J. A., & Candela, I. (2019). Aprendizaje cooperativo: Fundamentos, características y técnicas (Vol. 2). Publicaciones ICCE.

Gouaillier, D., Hugel, V., Blazevic, P., Kilner, C., Monceaux, J., Lafourcade, P., Marnier, B., Serre, J., & Maisonnier, B. (2009). Mechatronic design of NAO humanoid. In 2009 IEEE International Conference on Robotics and Automation (pp. 769–774). <https://doi.org/10.1109/ROBOT.2009.5152516>

Gouws, L. A., Bradshaw, K., & Wentworth, P. (2013). Computational thinking in educational activities. In *Proceedings of the 18th ACM*

Conference on Innovation and Technology in Computer Science Education (pp. 10–15). <https://doi.org/10.1145/2462476.2466518>

Karim, M. E., Lemaignan, S., & Mondada, F. (2015). A review: Can robots reshape K-12 STEM education? In 2015 IEEE International Workshop on Advanced Robotics and Its Social Impacts (ARSO) (pp. 1–8). <https://doi.org/10.1109/ARSO.2015.7428217>

Keyser, M. W. (2000). Active learning and cooperative learning: Understanding the difference and using both styles effectively. *Research Strategies*, 17(1), 35–44. [https://doi.org/10.1016/S0734-3310\(00\)00022-7](https://doi.org/10.1016/S0734-3310(00)00022-7)

Kolmos, A., Rodríguez, F. J., & Guerra, A. (2017). Aprendizaje basado en problemas en ingeniería: Teoría y práctica (F. Rodríguez-Mesa, Trad.). Aalborg University Press.

Ledo, M. V., Rivera Michelena, N., Cao, N. N., Morales Suárez, R., Niurka, M., & Vidal, V. (2016). Aula invertida, nueva estrategia didáctica. *Educación Médica Superior*, 30(3).

LEGO Education. (2016). Guía de uso: Lego Mindstorms Education EV3 (Versión 1.2.2, Vol. 3). The LEGO Group.

LEGO Education. (2021a). LEGO® MINDSTORMS® EV3 31313. <https://www.lego.com/es-es/product/lego-mindstorms-ev3-31313>

LEGO Education. (2021b, 5 de febrero). MINDSTORMS Education EV3 Retirement FAQs. <https://community.legoeducation.com/blogs/36/95>

Liu, M., & Pedersen, S. (1998). The effect of being hypermedia designers on elementary school students' motivation and learning of design.

- López, G., & Acuña, S. (2011). Aprendizaje cooperativo en el aula. *Inventio*, 7(14), 29–38. <https://inventio.uaem.mx/index.php/inventio/article/view/422>
- Lott, S., & Banffy, R. (2022). Functional Python programming. <https://books.google.com.co/books?hl=es&lr=&id=iWWkEAAQBAJ>
- Raspberry Pi Ltd. (2012). Raspberry Pi. <https://www.raspberrypi.com/>
- Lutz, M. (2013). Learning Python. O'Reilly Media.
- Martí, J. A., Heydrich, M., Rojas, M., & Hernández, A. (2010). Aprendizaje basado en proyectos: Una experiencia de innovación docente. *Revista Universidad EAFIT*, 46, 11–21.
- Microsoft. (2020). Visual Studio Code. <https://code.visualstudio.com/>
- Moscaiza Torres, V. A. (2022). Robótica educativa en el aprendizaje cooperativo del área de Matemática en niños y niñas del sexto grado de educación primaria (Tesis de Maestría). Universidad Nacional de Huancavelica.
- Oliver, J., Toledo, R., Pujol, J., Sorribes, J., & Valderrama, E. (2009). Un ABP basado en la robótica para las ingenierías informáticas. *JENUI 2009*, 331–338.
- OpenCV Team. (2008). OpenCV. <https://opencv.org/>
- Python, P. (2025, enero 2). PEP 8 – Style Guide for Python Code. <https://peps.python.org/pep-0008/>
- Python Software Foundation. (2021, enero 30). Python Brochure Vol. 1. <https://brochure.getpython.info/>
- Quigley, M., Gerkey, B., Conley, K., Faust, J., Foote, T., Leibs, J., Berger,

- E., Wheeler, R., & Ng, A. (2009). ROS: An open-source robot operating system. ICRA Workshop on Open Source Software, 3(3.2), 5–11.
- Quintero Vallejo, H. A., & Prado Martínez, W. A. (2020). Evaluación de la pertinencia del aprendizaje basado en proyectos (ABP)... (Tesis de Maestría). Universidad Tecnológica de Pereira.
- Richardson, M., Donat, W., & Wallace, S. (2021). Getting started with Raspberry Pi (4.a ed.). Make Community.
- Robot Framework Foundation. (2025). Robot Framework. <https://robotframework.org/>
- Robotics, O. (2021). ROS - Robot Operating System. <https://www.ros.org/>
- Rollins, M. (2014). Beginning LEGO MINDSTORMS EV3. Apress.
- Ruiz Vicente, F. (2017). Diseño de proyectos STEAM a partir del currículum actual de Educación Primaria.... Universidad CEU Cardenal Herrera.
- Seshan, S., & EV3Lessons. (2020). Robot designs. <https://ev3lessons.com/en/robotdesigns.html>
- SoftBank Robotics America. (2025). NAO: Personal Robot Teaching Assistant. <https://us.softbankrobotics.com/nao>
- Stellman, A., & Greene, J. (2024). Head First C# (5.a ed.). O'Reilly Media.
- Sweigart, A. (2020). Automate the boring stuff with Python. No Starch Press.

Trujillo, F. (2016). Aprendizaje basado en proyectos: Infantil, primaria y secundaria (1.a ed.). Ministerio de Educación.

Valk, L. (2021). Pybricks. <https://pybricks.com/>

Valk, L., & Pybricks. (2022). Pybricks. <https://pybricks.com/>

Vergara Ramírez, J. J. (2015). El aprendizaje basado en proyectos (ABP), paso a paso (Vol. 10). E. M. Ediciones.



## Autores

### **Francisco Alejandro Medina Aguirre**

(Pereira, Colombia 1975)

Magister en Instrumentación Física de la Universidad Tecnológica de Pereira. Ingeniero de Sistemas de la Universidad Antonio Nariño. Docente Asociado de la Universidad Tecnológica de Pereira, adscrito a la Facultad de Ingenierías. Ha publicado artículos en revistas especializadas nacionales e internacionales en las áreas de programación, robótica y desarrollo de software, contribuyendo a la formación y actualización del conocimiento en disciplinas asociadas a la ingeniería aplicada. Pertenece a los grupos de investigación Nyquist y Robótica Aplicada, ambos registrados en Minciencias.

**famedina@utp.edu.co**



## **Jimmy Alexander Cortés Osorio**

(Pereira, Colombia 1968)

Doctor en Ingeniería de la Universidad Nacional de Colombia (2020). Profesor Titular adscrito al Departamento de Física de la Facultad de Ciencias y a la Facultad de Ingenierías de la Universidad Tecnológica de Pereira. Investigador Senior (IS) reconocido por Minciencias y líder del grupo de investigación Robótica Aplicada, registrado en Minciencias. Es miembro activo del Institute of Electrical and Electronics Engineers (IEEE) y de la Society for Industrial and Applied Mathematics (SIAM), asociaciones internacionales de referencia en ingeniería y matemáticas aplicadas.

Ha participado como conferencista invitado en congresos y eventos académicos en Asia, Europa y América, consolidando una trayectoria internacional en instrumentación, visión por computador y aprendizaje automático. Sus publicaciones recientes incluyen artículos en revistas indexadas como Tecnura, Information and Computer Security, Revista Facultad de Ingeniería – Universidad de Antioquia, Scientia et Technica e INGE CUC. Así mismo, es autor de libros especializados en óptica, oscilaciones, circuitos eléctricos, circuitos lógicos y prototipos para la enseñanza de la física e ingeniería, publicados por Ecoe Ediciones y JM Calle Gráficas Buda.

Lidera el grupo de investigación Robótica Aplicada, registrado en Minciencias.

**[jacoper@utp.edu.co](mailto:jacoper@utp.edu.co)**

## **Hernán Alberto Quintero Vallejo**

(Santa Rosa de Cabal, Risaralda 1967)

Magister en Enseñanza de la Física de la Universidad Tecnológica de Pereira. Licenciado en Electricidad y Electrónica de la Universidad del Quindío. Docente Asociado de la Universidad Tecnológica de Pereira, adscrito a la Facultad de Tecnología.

Es autor de dos patentes de invención: “Equipo portátil para evitar la contaminación vehicular en las estaciones de servicio”, concedida en 2017 (Exp. N.º 14277014), y “Sistema y método de registro automático de condiciones de salud en animales”, concedida en 2024 (Exp. N.º NC2021/0005805). Ha publicado artículos en revistas especializadas nacionales en el área de la Ingeniería Mecatrónica.

Pertenece al grupo de investigación Mecabot, registrado en Minciencias.

**hernan.quintero@utp.edu.co**



La Editorial de la Universidad Tecnológica de Pereira tiene como política la divulgación del saber científico, técnico y humanístico para fomentar la cultura escrita a través de libros y revistas científicas especializadas.

Las colecciones de este proyecto son: Trabajos de Investigación, Ensayos, Textos Académicos y Tesis Laureadas.

Este libro pertenece a la colección Textos Académicos.

Programación en Python para Robótica: de la teoría a la práctica con LEGO MINDSTORMS® EV3 invita a descubrir el mundo de la robótica educativa. Con un estilo accesible y ejemplos paso a paso, guía al lector desde los fundamentos de Python hasta la programación de robots capaces de percibir y responder a su entorno mediante sensores y actuadores. Cada capítulo propone retos progresivos que estimulan la experimentación y el diseño de soluciones prácticas, consolidando conceptos de programación, control y automatización. Al mismo tiempo, fomenta la creatividad, la innovación y el pensamiento crítico, elementos esenciales en la formación de estudiantes y profesionales en la era digital. Más que un manual técnico, este libro es una herramienta didáctica y motivadora, ideal para estudiantes, docentes y entusiastas que buscan fortalecer sus habilidades en programación y pensamiento computacional. Su enfoque práctico convierte el aprendizaje en una experiencia dinámica, donde teoría y práctica se encuentran para dar vida a proyectos robóticos. En definitiva, es una invitación a construir, programar y crear, demostrando que la robótica es un puente entre el conocimiento y la imaginación aplicada al mismo tiempo que se aprende los conceptos básicos de programación en Python.